

**Row Pattern Matching Analytics: Bringing SQL
MATCH_RECOGNIZE to Pandas DataFrames**

by

Monier Ashraf Monier Lawande

A Thesis Submitted to the
School of Information Technology and Computer Science
at Nile University

In Partial Fulfillment of the
Requirements for the Degree of
Master of Science

In

Informatics

Date of submission

SEPTEMBER/2026

**Row Pattern Matching Analytics: Bringing SQL
MATCH_RECOGNIZE to Pandas DataFrames**

by

Monier Ashraf Monier Lawande

A Thesis Submitted to the
School of Information Technology and Computer Science
at Nile University

In Partial Fulfillment of the
Requirements for the Degree of
Master of Science

In

Informatics

Supervision Committee

Main Supervisor

**Professor Mohamed ElHelw, School of Information Technology and
Computer Science, Nile University**

Supervisor

**Associate Professor Ahmed Awad, The British University in Dubai,
UAE, and Cairo University, Egypt**

CERTIFICATION OF APPROVAL

Row Pattern Matching Analytics: Bringing SQL MATCH_RECOGNIZE to Pandas
DataFrames

by
Monier Ashraf Monier Lawande



Dr. Iman Helal
Associate Professor, Cairo University

2 / sept / 2026

Date



Dr. Ahmed El Nokrashy
Associate Professor, ITCS-NU

2-sept-2026

Date

on behalf of: 

Dr. Mohamed El Helw
Professor, ITCS-NU

2/9/2026

Date

Ahmed Awad

Dr. Ahmed Awad
Associate Professor, The British University in Dubai

2-9-2026

Date

COPYRIGHT

Attention is drawn to the fact that copyright of this thesis rests with the author and copyright of any previously published materials included may rest with third parties. A copy of this thesis has been supplied on condition that anyone who consults it understands that they must not copy it or use material from it except as licensed, permitted by law, or with the consent of the author or other copyright owners.

يجب التنويه إلى أن حقوق الطبع والنشر لهذه الأطروحة تعود إلى المؤلف وأن حقوق الطبع والنشر لأي مواد منشورة مسبقاً متضمنة قد تكون مع أطراف ثالثة. تم توفير نسخة من هذه الأطروحة بشرط أن يجب على أي شخص يستشيرها عدم نسخها أو استخدام مواد منها باستثناء ما هو مرخص أو يسمح به القانون أو بموافقة المؤلف أو مالكي حقوق الطبع والنشر الآخرين.

This thesis was prepared according to:

- Egyptian Standard No. 2609-2005, which is technically identical to ISO 7144:1986 for the preparation of scientific theses.
- References follow the IEEE citation style.

تم إعداد وإخراج هذه الرسالة وفقاً ل :

- المواصفة القياسية المصرية رقم 2005-2609 المتماثلة فنياً مع المواصفة الدولية رقم ISO 7144:1986 الخاصة بإعداد الأطروحات العلمية
- تم صياغة الاستشهادات المرجعية المقتن وفقاً لأسلوب (IEEE Style)

Acknowledgement

I would like to express my deepest gratitude to Professor Ahmed Awad for his continuous academic, technical, and personal support throughout this work. His guidance, encouragement, and unwavering support at every stage meant more to me than words can express, and I am sincerely thankful for everything he has done. I am also deeply grateful to Professor Mohamed ElHelw for his valuable guidance, constant support, and understanding, particularly during difficult and unusual circumstances.

My heartfelt gratitude goes to my family (my mother, my father, and my brother Rafek Ashraf) for their endless love, patience, encouragement, and support throughout all my years of study. Their belief in me has always been a source of strength and motivation.

I would also like to thank my friends and colleagues for their support, encouragement, and kindness throughout this journey.

Declaration

I hereby declare that this thesis is my own original intellectual work and has been conducted in accordance with the principles of academic integrity and ethical research practices. The conceptual framework, critical analyses, and syntheses of knowledge are all my own contributions. This work has not been submitted for a degree at any other university or institution. I also confirm that all sources have been properly acknowledged and cited in the references.

Any use of artificial intelligence (AI) tools was strictly limited to technical assistance and did not replace the author's independent intellectual contribution, analysis, or academic judgment.

Student's Name: Monier Ashraf Monier Lawande

Date: _____

Signature: _____

Abstract

SQL:2016 introduced the `MATCH_RECOGNIZE` clause to search ordered rows for user-defined patterns. It is useful when the information is a sequence, not one row. It expresses V-, W-, and U-shaped patterns (one dip, two dips, a long dip) and repeated behavior, as in fraud detection, IoT monitoring, and clickstreams. Oracle and Trino provide it, but Pandas does not, so analysts fall back on shifted columns, loops, and state handling that are harder to verify.

The standard defines row-pattern recognition as three independent optional features: R010 places the clause in the `FROM` clause, so a pattern reads like a table, and the other two cover windows and aggregates. A vendor may implement any or none, so support differs widely.

This thesis addresses that gap with an in-process engine for a subset of R010 that it defines, implements, and tests. The engine parses a query, compiles its row pattern into finite automata, and executes it over ordered `DataFrame` partitions. The evaluation covers correctness, feature support, usability, performance, and scalability against Trino and Oracle. All 709 regression tests passed.

The main benchmark ran five patterns over six Amazon UK Products 2023 sizes, 100 K to 2.22 million rows (30 pattern-size cells per system), and all three agreed on every result. Under one core and 32 GB, mean time was 0.17 s for the engine, 1.04 s for Oracle, and 1.98 s for Trino. A scalability study on Amazon Reviews 2023 ran the same five patterns over seven sizes to 227.9 million rows (35 cells per system) under one core and 58 GB; the engine and Oracle completed all 35, Trino 33. Engine time stayed close to linear, while query-time memory grew with input size. Row-pattern recognition

can therefore connect declarative SQL with in-process Pandas analysis while keeping sequential queries readable.

Keywords: MATCH_RECOGNIZE; Row Pattern Recognition; Pandas; SQL:2016; Finite Automata; Pattern Matching.

Table of Contents

Acknowledgement	ii
Declaration	iii
Abstract	iv
Table of Contents	vi
List of Figures	ix
List of Tables	x
List of Symbols, Abbreviations and Nomenclature	xii
List of Publications out of This Thesis	xiv
Chapter 1: Introduction	1
1.1 Row Patterns in Ordered Data	1
1.2 Problem Statement	3
1.3 Research Objectives and Contributions	5
1.4 Organization of the Thesis	6
1.5 Publications in the Course of this Thesis	7
1.6 Chapter Summary	7
Chapter 2: Background and Concepts	9
2.1 Pattern Matching in Sequential Data Analysis	9
2.2 The SQL:2016 MATCH_RECOGNIZE Clause	10
2.2.1 Clause Structure and Components	10
2.2.2 Illustrative Example: Price V-Pattern	13
2.3 Chapter Summary	13
Chapter 3: Related Work	14
3.1 Existing Systems and Approaches	14
3.1.1 Comparative Overview of Existing Implementations	14
3.1.2 Oracle	15
3.1.3 Trino	15
3.1.4 Snowflake	15
3.1.5 Apache Flink	16
3.1.6 Bringing MATCH_RECOGNIZE to DuckDB	16
3.1.7 Optimization Research	16
3.1.8 Implementation Restrictions Across Systems	17
3.2 Pandas and the Python Ecosystem Gap	17
3.3 Contributions of This Work	17
3.4 Chapter Summary	18

Chapter 4: System Design and Methodology	19
4.1 Architecture Overview	19
4.2 SQL Parsing	22
4.3 Semantic Validation	26
4.4 Logical Plan Construction	27
4.5 Chapter Summary	28
Chapter 5: Implementation and Optimization	29
5.1 Pattern Compilation with Finite Automata	29
5.2 Pandas DataFrame Execution Model	31
5.2.1 Exact Search for State-Dependent Conditions	32
5.2.2 Skip Modes, Output Modes, and Measures	32
5.3 Performance Optimization Techniques	34
5.4 Chapter Summary	36
Chapter 6: Experimental Evaluation	38
6.1 Evaluation Methodology	38
6.1.1 Evaluation Criteria and Metrics	38
6.1.2 Environment and Configuration	39
6.2 Dataset and Test Patterns	40
6.3 Validation of Concept	42
6.3.1 Example 1: Dip Pattern on the orders Dataset	42
6.3.2 Example 2: Stock-Price Analysis on the stock_price Dataset	42
6.3.3 Example 3: V-Shape Recovery on the stock_price Dataset	48
6.3.4 Usability and Developer Effort	50
6.4 Coverage and Complexity Results	51
6.4.1 SQL Feature Coverage	51
6.4.2 Complexity Analysis	53
6.5 Empirical and Cross-System Analysis	56
6.5.1 Pattern Definitions	56
6.5.2 Cross-System Comparison Under a Common Outer Resource Limit	58
6.5.3 Throughput by Pattern, Size, and System	60
6.5.4 Memory Metrics by Pattern, Size, and System	60
6.5.5 Cross-System Summary	63
6.5.6 Sensitivity and Boundary Analysis	65
6.5.7 Scalability	68
6.5.8 Evaluation Validity	71
6.5.9 Key Findings	72
6.6 Chapter Summary	72
Chapter 7: Conclusions and Future Work	73
7.1 Discussion	73
7.2 Claims and Evidence	74
7.3 Limitations	75
7.4 Conclusion	76

7.5	Future Work	77
7.6	Chapter Summary	77
	References	78
	Appendix A: Detailed Benchmark Tables	83
A.1	Measurement Provenance and Reproducibility	83
A.2	Match Counts	84
A.3	Performance Summary by Pattern and by Dataset Size	84
A.4	Detailed Cross-System Benchmark Matrices	85
A.5	Ordered vs. Shuffled Input (Sorting Scenarios)	89
A.6	Stress-Test Results	90

List of Figures

Figure 1-1:	The V, W, and U shapes.	2
Figure 4-1:	Five-stage architecture of the proposed engine.	19
Figure 4-2:	Worked orders pipeline.	21
Figure 4-3:	Visitor pipeline: from SQL to an AST	24
Figure 4-4:	Typed AST for the worked orders query	25
Figure 4-5:	Resolving RUNNING and FINAL semantics.	28
Figure 5-1:	Language-equivalent NFA for $A^+B?$	29
Figure 5-2:	Planning, compilation, and DataFrame execution	31
Figure 6-1:	Columns of the two synthetic tables	38
Figure 6-2:	Data-size complexity of the compiled benchmark path	55
Figure 6-3:	Performance summary by pattern	59
Figure 6-4:	Performance summary by dataset size	59
Figure 6-5:	Execution time by pattern, size, and system	60
Figure 6-6:	Throughput by pattern, size, and system	61
Figure 6-7:	Query memory by pattern, size, and system	62
Figure 6-8:	Footprint memory by pattern, size, and system	62
Figure 6-9:	Overall benchmark statistics by system	63
Figure 6-10:	Average query memory by dataset size	64
Figure 6-11:	Average footprint memory by dataset size	64
Figure 6-12:	Proposed engine relative to each SQL engine	65
Figure 6-13:	Ordered vs shuffled input by system	66
Figure 6-14:	Ordered and shuffled engine time	67
Figure 6-15:	Pattern-driven worst case, all systems	67
Figure 6-16:	Volume scaling on real data to 227.9M rows	69
Figure 6-17:	Throughput scaling on real data	69
Figure 6-18:	Per-pattern time at the full 227.9M-row corpus	70
Figure 6-19:	Diagnostic query-memory counters on real data	71
Figure A-1:	Per-pattern execution-time scaling	91

List of Tables

Table 1-1:	Six ordered rows, each labelled by a condition.	1
Table 3-1:	Scope and deployment of selected row-pattern systems . . .	15
Table 4-1:	Principal fields in the logical execution configuration	27
Table 5-1:	The four AFTER MATCH SKIP modes and the resulting next scan position	33
Table 5-2:	Output modes and their row-count and measure-semantics behavior	33
Table 5-3:	Performance optimizations, purpose and activation	35
Table 6-1:	Sample orders dataset (customer_id, order_date, price).	40
Table 6-2:	Stock price table	41
Table 6-3:	Dip-pattern result	42
Table 6-4:	Stock-price result	48
Table 6-5:	V-shape result	50
Table 6-6:	Lines-of-code comparison for the three worked examples. . .	51
Table 6-7:	Support status of MATCH_RECOGNIZE features in the proposed engine	52
Table 6-8:	Pattern-type coverage	53
Table 6-9:	Theoretical and empirical views of complexity analysis . . .	54
Table 6-10:	SQL Pattern Definitions and Detection Goals	57
Table 7-1:	Claims, evidence, and scope	74
Table A-1:	Returned rows by pattern and size under ONE ROW PER MATCH	84
Table A-2:	Performance summary by pattern across the three systems .	84
Table A-3:	Performance summary by dataset size across the three systems	85
Table A-4:	Execution time by pattern, size, and system (seconds; mean $\pm$ sample standard deviation)	85
Table A-5:	Throughput (rows/s) by pattern, dataset size, and system . .	86
Table A-6:	Incremental query memory by pattern, size, and system (MB)	86
Table A-7:	Absolute footprint memory by pattern, size, and system (MB)	87
Table A-8:	Overall benchmark statistics by system	87
Table A-9:	Average query memory by dataset size, all three systems . .	88
Table A-10:	Average footprint memory by dataset size, all three systems .	88
Table A-11:	Cross-system correctness by pattern across six dataset sizes .	88
Table A-12:	Relative difference between the proposed engine and each SQL engine	89
Table A-13:	Average execution time for ordered and shuffled input . . .	89
Table A-14:	Execution time and slowdown by size for ordered (“ord”) and shuffled (“shuf”) input	89

Table A-15:	Observed ordered–shuffled time difference for the proposed engine by dataset size	90
Table A-16:	Mean execution time for the state-dependent aggregate pattern	90
Table A-17:	Stress-test execution time (seconds; mean $\pm$ SD)	92
Table A-18:	Stress-test absolute peak memory (GB)	92
Table A-19:	Stress-test throughput (million rows/s)	93
Table A-20:	Stress-test incremental peak memory above the pre-query baseline (GB)	93
Table A-21:	Database-native per-query memory counters (GB)	94
Table A-22:	Stress-test returned rows by pattern and size under ONE ROW PER MATCH	94

List of Symbols, Abbreviations and Nomenclature

Nomenclature

Symbol	Meaning
n	Number of input rows
m	Number of pattern symbols, with a bounded quantifier $\{i, j\}$ counted as j copies; a PERMUTE of m variables has $m!$ orders
i, j	Lower and upper bounds of a quantifier $\{i, j\}$
d	Number of DEFINE variables
k	Number of matches
ℓ	Average number of rows per match materialized in the output
p	Number of partitions
$O(f(n))$	Upper bound: the cost grows no faster than $f(n)$
R^2	Share of the variation in measured time that a fitted line explains; 1 is a perfect fit
$\bar{x}$	Arithmetic mean of a metric across the 30 benchmark cells
$q_0, q_1, \dots$	States of a non-deterministic finite automaton
$D_0, D_1, \dots$	States of a deterministic finite automaton

Greek Letters

Symbol	Meaning
ε	Empty transition of an automaton, which consumes no row
$\Omega(f(n))$	Lower bound: the cost grows at least as fast as $f(n)$
$\Theta(f(n))$	Tight bound: both $O(f(n))$ and $\Omega(f(n))$ hold

Subscripts

Subscript	Meaning
$0, 1, 2, \dots$	Index of an automaton state, as in q_0 or D_0
h	Index of a partition; n_h is the number of rows in partition h
proposed	The proposed engine, as in $\bar{x}_{\text{proposed}}$
reference	The SQL engine it is compared with, as in $\bar{x}_{\text{reference}}$

Abbreviations

Abbreviation	Full name or meaning
ANTLR	ANother Tool for Language Recognition
ANTLR4	ANother Tool for Language Recognition, version 4
API	Application Programming Interface
ASC	Ascending (SQL ordering direction)
AST	Abstract Syntax Tree
CEP	Complex Event Processing
CPU	Central Processing Unit
CSV	Comma-Separated Values
CTE	Common Table Expression
DESC	Descending (SQL ordering direction)
DFA	Deterministic Finite Automaton
EE	Enterprise Edition (Oracle)
GB	Gigabyte
GBP	Pound sterling (currency code)
GiB	Gibibyte (2^{30} bytes)
IEEE	Institute of Electrical and Electronics Engineers
IoT	Internet of Things
IQR	Interquartile Range
JIT	Just-In-Time (compilation)
JVM	Java Virtual Machine
$LL(*)$	Left-to-right parsing with a leftmost derivation and variable lookahead
LOC	Lines of Code
LRU	Least Recently Used
LTS	Long-Term Support
MB	Megabyte
MiB	Mebibyte (2^{20} bytes)
MIT	Massachusetts Institute of Technology (license name)
NFA	Nondeterministic Finite Automaton
PGA	Program Global Area (Oracle)
RAM	Random Access Memory
SD	Standard Deviation
SGA	System Global Area (Oracle)
SQL	Structured Query Language
UK	United Kingdom

List of Publications out of This Thesis

1. M. Lawande, M. El-helw, and A. Awad, “Row Pattern Recognition for Pandas: Bringing MATCH_RECOGNIZE to Python DataFrames,” in *2026 IEEE 23rd Mediterranean Electrotechnical Conference (MELECON)*, IEEE, 2026, doi: 10.1109/MELECON64486.2026.11418885.
2. A manuscript extending this work, submitted to a journal (under review).

Chapter 1: Introduction

Row-pattern recognition searches an ordered series of table rows for relationships that span several events. It can detect shapes such as V, W, and U, as well as trends and repeated behavior. These patterns support analysis in many domains. Examples include money-laundering transaction chains [1], failures in IoT sensor data, and user journeys through e-commerce clickstreams.

This chapter starts from the idea itself. Section 1.1 explains what a row pattern is, what the V, W, and U shapes mean, and how the existing families of languages express such patterns. Section 1.2 then states the problem this thesis addresses, and Section 1.3 turns it into objectives, contributions, and the evidence for them. The remaining sections give the structure of the thesis and the publications produced during it.

1.1 Row Patterns in Ordered Data

A row pattern describes a shape that appears across several consecutive rows. Building one takes three steps.

First, the rows are divided into independent groups (one customer, one sensor, one bank card), and each group is put in order, usually by time. Second, each kind of row that matters is given a name, and that name is given a meaning by a condition. A row whose price is lower than the row before it can be called `DOWN`; a row whose price is higher can be called `UP`. Third, a pattern says in which order those names may appear.

The third step is the same idea used to search text. A search for a word followed by any digits does not look for one fixed string; it describes a shape that many different strings can take. A row pattern does this for rows instead of characters [2], [3]. The idea is shown on six days of one customer's prices (See Table 1-1).

Table 1-1: Six ordered rows, each labelled by a condition.

Day	Price	Label	Why
1	100	START	the row the shape starts from
2	80	DOWN	lower than day 1
3	60	DOWN	lower than day 2
4	55	DOWN	lower than day 3
5	70	UP	higher than day 4
6	90	UP	higher than day 5

The price falls for three days and then rises for two. In words: one `START` row, then one or more `DOWN` rows, then one or more `UP` rows. That sentence is written as a pattern like this:

1 `PATTERN (START DOWN+ UP+)`

Each name in the line is one of the row labels (See Table 1-1), and the order of the names is the order the rows must appear in. The symbol + after a name means *one or more rows of that kind*. So START matches day 1, DOWN+ matches days 2 to 4, and UP+ matches days 5 and 6.

Because + does not fix a number, the same line also matches a fall lasting two days or ten. It describes the shape rather than one exact length, which is what makes it a pattern. This is the V shape (See Figure 1-1). Writing DOWN{3} instead would demand exactly three falling days, and DOWN? would make the fall optional.

The difference from an ordinary filter is the unit of interest. A filter asks a question about one row: is this price below ten? A row pattern asks a question about a run of rows: does the price fall three times and then recover? No single row answers the second question. The answer lives in the order and in the relationship between neighbors, so the information disappears the moment the rows are treated as an unordered set [4].

Analysts often name these patterns by the outline they trace when the measured value is plotted against time. Three names recur, shown side by side (See Figure 1-1).

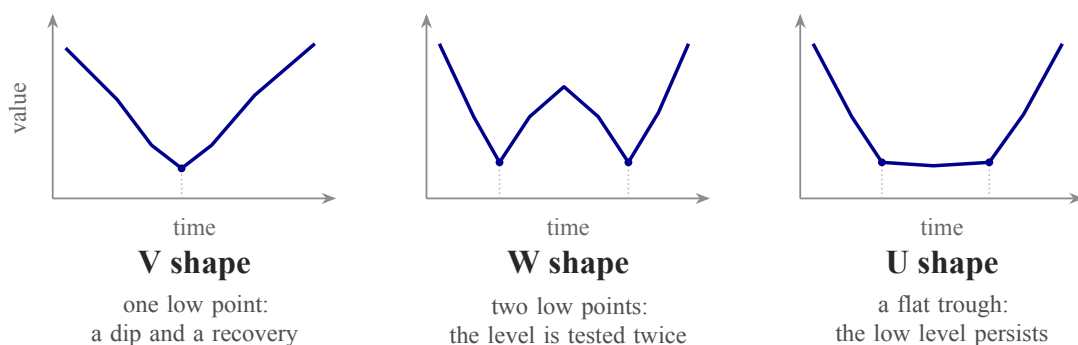


Figure 1-1: The V, W, and U shapes.

V shape. The value falls to a single low point and then rises again. It marks a dip followed by a recovery: a share price that drops and rebounds, a temperature that falls and returns, or a customer whose spending declines for a few months and then picks up. The V is the smallest interesting shape, because it already needs two opposite trends and the turning point between them.

W shape. Two V shapes in sequence. The value falls, recovers only partly, falls a second time, and then rises. In market analysis this double bottom is read as a stronger signal than one dip, since the level was tested twice. In monitoring it often means a fault that was not really fixed the first time.

U shape. The value falls, stays low for a while, and then rises. The trough is flat rather than sharp. The distinction from a V matters in practice: a V is a brief disturbance, whereas a U is a sustained condition such as an outage, a seasonal lull, or a slow recovery.

The shapes matter because they carry meaning that summary statistics lose. An average price hides whether the series dipped once, twice, or stayed low. Detecting

the shape, and reporting where it started and ended, is what turns an ordered table into an explanation of what happened [5]. The same reasoning extends beyond these three outlines to trends, repeated behavior, and recovery after a failure.

Sequential patterns are not new, and several families of languages already describe them. They differ in how directly the pattern can be stated, and in what has to be deployed to run it.

Procedural code keeps the data in the host language. An analyst writes a loop, or shifts columns and combines Boolean masks, and inspects the rows directly. Nothing new has to be installed, and any condition can be expressed. The cost is that the pattern is never written down as such: it is spread across index arithmetic, bounds checks, and overlap handling, where a single off-by-one silently changes the result [6].

Window functions and self-joins stay inside SQL. A query can reach the previous or next row with LAG and LEAD, or join a table to itself once per step. This is declarative and portable. But the pattern must be re-encoded as fixed offsets, so the query grows with the length of the sequence, a variable-length repetition is awkward to express, and the plan may scan the table several times [7].

Complex-event-processing languages were designed for this problem in streams. Systems such as Apache Flink provide pattern operators over unbounded event sequences [8]. They are expressive, but they assume a streaming runtime and a separate deployment, which is a large step for an analyst holding a finished table.

Row-pattern recognition in SQL takes the regular-expression idea and applies it to rows. The pattern is written once, as a pattern, next to the data it queries. This is the MATCH_RECOGNIZE clause, and it is the approach this thesis follows. Consider a card that a thief first tests with small charges and then empties. A fraud analyst writes that sequence directly as the shape it is:

```
1 PATTERN (PROBE+ (TRANSFER | WITHDRAW){2,5} DRAIN)
```

It is built the same way as PATTERN (START DOWN+ UP+) above, with two additions. PROBE+ again means one or more test charges. The bar in (TRANSFER | WITHDRAW) means *or*, so that row may be either kind, and {2,5} allows between two and five of them. DRAIN matches the final large movement. As with the row labels (See Table 1-1), the names are not keywords of the language: the analyst invents them and gives each one a meaning through a condition, so PROBE might mean an amount below a small threshold. Chapter 2 presents the full clause and its sub-clauses.

The cost of this approach is scope: the clause is optional in the standard, so it is not available everywhere. Chapter 3 examines how far each system implements it.

1.2 Problem Statement

SQL:2016 standardized row-pattern recognition through the MATCH_RECOGNIZE clause [9], [10]. The clause lets a query define and search patterns within ordered rows. Oracle had already introduced it in Oracle Database 12c in 2013 [11], [12], [13]. Support nevertheless remains uneven.

Trino [14], [15], Snowflake [16], and Apache Flink [8] provide their own implementations. However, support for the optional SQL:2016 features R010, R020, and R030 varies, as do practical restrictions. PostgreSQL 18 lists these features as unsupported, while DuckDB lists MATCH_RECOGNIZE as planned work [17], [18].

This gap matters because many analytical workflows begin in Python. Users prepare data, test hypotheses, and build models before moving to a larger database or cloud platform. For data that already fits in memory, a separate SQL engine adds deployment, integration, and data-transfer work. This thesis does not aim to replace an enterprise database. Its main objective is to bring a practical subset of declarative row-pattern recognition to the Pandas environment where these workflows often begin. Section 1.3 states that objective precisely and breaks it into three parts.

Without this declarative primitive, SQL users must combine window functions, self-joins, or loops outside the database. Beyond two or three steps, the logic accumulates row-number calculations, timestamp checks, and JOIN conditions. It is difficult to maintain and may require repeated table scans [7].

The same limitation appears in Python. Pandas [19] is a popular tabular-data library, but it has no built-in SQL row-pattern primitive. Analysts instead create shifted columns such as `df['value'].shift(-1)` and `shift(-2)`, combine Boolean masks, use `groupby().apply()`, and then pivot or rename intermediate columns. Explicit loops are also common. Listing 1-1 shows this approach for a three-row price decrease (A B C).

```

1 import pandas as pd
2
3 df['order_date'] = pd.to_datetime(df['order_date'])
4 df = df.sort_values(['customer_id', 'order_date']).reset_index(drop=True)
5
6 def match_pattern_ABC(group: pd.DataFrame):
7     matches = []
8     for i in range(len(group) - 2):
9         A, B, C = group.iloc[i], group.iloc[i+1], group.iloc[i+2]
10        if (B['price'] < A['price']) and (C['price'] < B['price']):
11            matches.append({
12                'customer_id': A['customer_id'],
13                'start_date': A['order_date'],
14                'end_date': C['order_date'],
15                'bottom_price': C['price']
16            })
17    return matches
18
19 all_matches = []
20 for _, grp in df.groupby('customer_id'):
21     all_matches.extend(match_pattern_ABC(grp))
22
23 result_df = pd.DataFrame(all_matches)

```

Listing 1-1: Detecting a three-step price-decrease pattern in Pandas without MATCH_RECOGNIZE

Even this fixed three-row pattern needs several procedural steps. The hand-written logic is vulnerable to alignment errors and does not extend cleanly to longer patterns or patterns with more conditions. Such cases also require custom indexing, overlap handling, and output logic.

The same query expressed with MATCH_RECOGNIZE is shown in Listing 1-2: a single declarative statement that specifies the partition, ordering, pattern grammar, variable definitions, and output measures.

The declarative form is not only shorter. It separates what to find from how the engine finds it. The query is easier to audit, compose, and extend. A four- or five-step sequence

```

1 import pandas as pd
2 from pandas_match_recognize import match_recognize
3
4 df['order_date'] = pd.to_datetime(df['order_date'])
5
6 query = """
7 SELECT customer_id, start_date, end_date, bottom_price
8 FROM orders
9 MATCH_RECOGNIZE (
10     PARTITION BY customer_id
11     ORDER BY      order_date
12     MEASURES
13         A.order_date AS start_date,
14         C.order_date AS end_date,
15         C.price      AS bottom_price
16     ONE ROW PER MATCH
17     AFTER MATCH SKIP TO NEXT ROW
18     PATTERN (A B C)
19     DEFINE
20         B AS B.price < A.price,
21         C AS C.price < B.price
22 )
23 """
24
25 output_df = match_recognize(query, df)

```

Listing 1-2: The same pattern expressed with MATCH_RECOGNIZE on a Pandas DataFrame

with conditional alternation normally changes the PATTERN and DEFINE clauses, not the surrounding Python traversal code [5], [20]. The implementation can then compile the pattern into automata, cache repeated pattern structures, and process ordered partitions without exposing those details to the user. Chapter 2 introduces the full structure and sub-clauses of MATCH_RECOGNIZE.

1.3 Research Objectives and Contributions

Main objective. To bring a practical R010-style subset of SQL:2016 row-pattern recognition to Pandas, so that a declarative pattern query runs directly on a DataFrame without a database server.

This connects relational pattern queries with in-memory analytics. It also asks a question that the rest of the thesis answers: whether such queries can run in process, correctly and fast enough to be useful. Three specific objectives follow from it.

Specific objectives. Each one is stated so that the thesis can be judged against it.

1. **Determine whether the subset can run in process.** The standard defines row-pattern recognition for a database engine, which owns its storage, planner, and execution operators. A DataFrame runtime owns none of these. The first objective is to design and realize an engine that parses a MATCH_RECOGNIZE query, compiles its pattern into automata, and executes it over an ordered DataFrame with no database server, and so to show whether the standard's execution model survives that move.
2. **Define the supported subset and verify its correctness.** A partial implementation is only useful if its edges are known. The second objective is to state the evaluated R010-style subset precisely: the main clauses, the output and skip modes,

navigation and aggregate functions, and the pattern constructs from quantifiers and alternation to exclusions and PERMUTE. It is then to establish that its results agree with independent production implementations, rather than with the engine's own expectations.

3. **Characterize the cost and the limits.** Feasibility alone does not justify the approach. The third objective is to measure execution time, throughput, and memory against established engines under equal resource limits, and to test how the engine scales as the input grows.

Objective 1 is addressed by the design in Chapter 4 and the implementation in Chapter 5. Objectives 2 and 3 are answered by the five evaluation criteria of Chapter 6: correctness and feature coverage for the second, and usability, performance, and scalability for the third.

Contributions. Together, these objectives produce three contributions: an in-process Pandas engine with tested row-pattern semantics, an optimized and guarded execution design, and a theoretical and empirical evaluation. Section 3.3 explains these contributions and positions them against related systems. Chapter 4 describes the engine's five core stages.

Evidence at a glance. At the time of final validation, the regression suite contained 709 passing test cases [21]. The cases cover pattern families, constructs, output modes, navigation, and aggregation behavior. The controlled cross-system benchmark adds direct comparison with Trino 473 and Oracle 21c EE. It uses five pattern families and six nested prefixes of the Amazon UK Products dataset, from 100 K rows to the full 2,222,742 rows. All 90 system–pattern–size cells completed. For each of the 30 pattern–size combinations, all three systems returned the same normalized result set. Under the shared one-core, 32 GB memory limit, mean time was 0.17 s for the engine, 1.04 s for Oracle, and 1.98 s for Trino. Engine time grew almost linearly. Its measured query-time memory was higher than either database, because the compiled path builds whole-column masks and intermediate arrays instead of evaluating row by row. Its total footprint was nevertheless lower, because it runs as a single Python process rather than a resident server that keeps the table in its own managed memory: a Java Virtual Machine (JVM) heap for Trino, or a System Global Area (SGA) for Oracle.

The larger Amazon Reviews test extends the range to 227.9 million rows, about $100\times$ the main dataset. Under a shared one-core, 58 GB memory limit, the engine and Oracle 21c Enterprise Edition completed all 35 cells. Trino 473 completed 33. Its complex nested cases at 160 and 227.9 million rows reached the 600 s limit. Engine time remained close to linear. Four pattern families were completed by all three systems at every size. Averaging those four at each size, Oracle's mean time was $5.1\text{--}10.1\times$ the engine's and Trino's was $11.4\text{--}13.8\times$ the engine's. Oracle's multiple grew with the input, while Trino's stayed roughly flat. At the largest size the four-family means were 28.9 s for the engine, 291.7 s for Oracle, and 375.6 s for Trino. Chapter 6 reports the complete method, trade-offs, and limits.

1.4 Organization of the Thesis

The rest of the thesis is organized as follows.

Chapter 2 — Background and Concepts presents the theoretical and technical foundations of the work. It introduces sequential pattern matching and its applications, explains the eight `MATCH_RECOGNIZE` sub-clauses, and presents the SQL:2016 R010–R030 feature classification used in this thesis.

Chapter 3 — Related Work reviews Oracle, Trino, Snowflake, Apache Flink, and DuckDB. It compares their feature scope, execution strategies, deployment models, and practical restrictions. It then examines the Python and Pandas gap and positions this work within the row-pattern literature.

Chapter 4 — System Design and Methodology describes the architecture and methodology. It covers the SQL grammar, parser, abstract syntax tree, semantic validation, logical plan, and the design that connects a declarative query to automata-based execution.

Chapter 5 — Implementation and Optimization explains how a pattern is compiled into automata, how those automata run over Pandas partitions, and how output and measures are built. It also presents the performance layer: vectorized conditions, compiled fast paths, pattern caching, compact output construction, and memory management.

Chapter 6 — Experimental Evaluation presents the methodology, datasets, and results. It evaluates correctness, feature and pattern coverage, usability, pattern complexity, performance, memory, and scalability. The chapter compares five pattern families across six Amazon UK Products sizes and then presents a larger Amazon Reviews scalability test. Trino and Oracle provide the cross-system references.

Chapter 7 — Conclusions and Future Work summarizes the contributions and current limitations. It also identifies directions for future research and extension.

1.5 Publications in the Course of this Thesis

This thesis produced two scientific outputs in data systems and sequential pattern recognition: one published paper and one submitted manuscript.

First Publication. The first paper was published in the *2026 IEEE 23rd Mediterranean Electrotechnical Conference (MELECON)*. The paper, entitled “Row Pattern Recognition for Pandas: Bringing `MATCH_RECOGNIZE` to Python DataFrames” [22], addresses the lack of native row-pattern recognition in Python data-analysis frameworks. It presents an SQL-in-Pandas engine for running `MATCH_RECOGNIZE`-style queries directly on DataFrames. The paper describes its supported row-pattern features, its automaton-based matching method, and initial tests. Those tests show that the declarative interface reduces procedural Pandas code while preserving the expected results in the studied cases.

Second Publication. A second manuscript extending this work has been submitted to a journal and is currently under review.

1.6 Chapter Summary

This chapter introduced row-pattern recognition and the lack of a native, declarative equivalent in Pandas. It explained why hand-written multi-row solutions become verbose, fragile, and difficult to extend. It then stated the research objectives, summarized the

contributions and evidence, outlined the thesis, and listed the research outputs. Chapter 2 next introduces the theory and SQL semantics used by the system.

Chapter 2: Background and Concepts

This chapter introduces the foundations needed to understand the `MATCH_RECOGNIZE` engine for Pandas DataFrames. Section 2.1 explains pattern matching in sequential data. Section 2.2 introduces the SQL:2016 clause and the terms used in this thesis. A V-pattern example then connects these concepts to the proposed system. The next chapter reviews related work and identifies the research gap.

2.1 Pattern Matching in Sequential Data Analysis

Sequential pattern detection identifies ordered events or states in a dataset. A conventional filter tests each row against one or more predicates. A row-pattern query also tests the order and relationships between rows. It returns a group when the rows follow the requested sequence. This makes it possible to find shapes such as V, W, and U (See Figure 1-1), as well as repeated or changing behavior.

The important distinction is that the “symbols” in row-pattern recognition are not characters in a string. They are data rows that become associated with pattern variables such as A, B, or DOWN when their predicates hold. A row sequence can therefore be read as a sequence of variable assignments. This explains why regular-expression-like syntax and finite automata fit the problem. The combination gives the best of both. The syntax is a readable way to write the pattern, and the automaton is an efficient way to run it, one row at a time. Later chapters use this connection to compile row patterns into automata and execute them over ordered Pandas partitions.

Relational databases are strong at set-based operations such as selection, projection, joins, and aggregation. Sequence analysis was traditionally harder. Analysts often exported data to a stream processor, used cursors and temporary tables, or wrote the logic in an application. SQL:2016 introduced row-pattern recognition so that this logic could be expressed directly in SQL [23].

Possible applications that benefit from this thesis work span several domains [11]. The following examples show where the need arises.

Financial Market Analysis. Detecting a V-shaped price pattern, a decline followed by a recovery, requires an ordered sequence of price movements. Successive higher troughs, for example, can indicate an upward trend. The worked example later in this chapter applies a V-pattern to ordered price records [11].

Fraud Detection. Credit card fraud can appear as a sequence, such as several small test charges before a large purchase. Another example is a series of transactions in locations that are too far apart to be realistic. Writing these rules in SQL can make them easier to manage than procedural code [1].

IoT and Sensor Data. Manufacturing systems monitor sensor readings for signs of equipment degradation. One example is a gradual temperature rise followed by a sudden drop. Pattern queries over ordered sensor readings are a natural use of `MATCH_RECOGNIZE` [11].

Clickstream and Funnel Analysis. E-commerce platforms study the paths that lead to a purchase or an exit. A funnel query may look for “view → add-to-cart → remove → exit.” With `MATCH_RECOGNIZE`, one query can count repeated actions, return their timestamps, and measure the elapsed time. The same analysis otherwise needs several subqueries or custom session logic [24].

Network Security and Intrusion Detection. Cyber-attacks often appear as a series of events rather than one isolated event. A multi-stage intrusion may start with port scanning, continue with failed logins, and end with a successful login and privilege escalation. One row-pattern query can describe this sequence and return the relevant session, timestamps, port, and account [1].

Healthcare and Patient Monitoring. Early-warning systems can look for a sequence of changes in clinical measurements. An alert may combine a rising heart rate, falling blood pressure, and a later laboratory change. A row-pattern query can describe this ordered sequence. The actual alerting rule depends on the clinical domain and system.

Supply Chain and Logistics. One late shipment may lead to partial deliveries and then a stockout. A row-pattern query over a fulfillment log can find this sequence for each supplier or warehouse. It can also measure the time from the first delay to the final stockout.

Financial Compliance and Trade Surveillance. Trade-surveillance systems look for sequences that may indicate market manipulation. In a layering example, a trader places a large resting order, trades on the other side, and then cancels the first order. `MATCH_RECOGNIZE` can express this timed sequence as an auditable SQL query.

2.2 The SQL:2016 `MATCH_RECOGNIZE` Clause

The SQL:2016 standard introduced `MATCH_RECOGNIZE` [9], [23]. It finds ordered rows in which each part of a pattern is defined by a row condition. The regular-expression-like pattern describes event sequences in a clear, declarative form [25]. Similar logic can be written with joins, window functions, recursion, or procedural code. The clause makes that logic shorter and easier to maintain.

The SQL standard distinguishes three optional features for row pattern recognition [2]. **Feature R010** covers row pattern recognition in the `FROM` clause. **Feature R020** covers row patterns in `WINDOW` specifications. **Feature R030** covers full aggregate support. They are distinct parts of the standard, not successive levels [4], [10], [20].

This thesis uses the SQL:2016 terminology as the conceptual frame, but the implementation claims are intentionally scoped. The proposed engine targets core R010-style behavior in the `FROM` clause and evaluates selected row-pattern constructs that are relevant to in-memory Pandas analytics. Features outside this scope, including full window-clause row-pattern recognition and complete vendor-specific extensions, are treated as related work or future work rather than as requirements of the implemented system.

2.2.1 Clause Structure and Components

The clause works with a table or a derived table. It is commonly described through the eight specifications shown in Listing 2-1. The standard requires `PATTERN` and `DEFINE`. The square brackets in the listing mark the optional specifications. Together, they control

partitioning, order, output, row classification, and overlap between matches. The clause thus mixes two languages: ordinary SQL for partitioning, ordering, and expressions, and a regular-expression-like language for the pattern.

```

1 SELECT <columns>
2 FROM <table>
3 MATCH_RECOGNIZE (
4   [PARTITION BY <partition_columns>] -- 1. Data organization
5   [ORDER BY <ordering_columns>] -- 2. Row sequencing
6   [MEASURES <expressions>] -- 3. Output computations
7   [<rows_per_match>] -- 4. Result granularity
8   [AFTER MATCH SKIP <option>] -- 5. Overlap handling
9   PATTERN ( <pattern_regex> ) -- 6. Sequence specification
10  [SUBSET <union_variable_defs>] -- 7. Union variables
11  DEFINE <variable_definitions> -- 8. Variable predicates
12 ) [AS <alias>]

```

Listing 2-1: General clause structure used in this thesis

1. **PARTITION BY.** Splits the input into independent partitions, similar to a window function. Matching then runs within each customer, sensor, or other partition. Independent partitions may also support parallel processing. If it is omitted, the input forms one partition.
2. **ORDER BY.** This determines the row order within each partition. The matcher processes rows in that sequence. An explicit and deterministic order is normally needed for repeatable results. Several sort keys may be combined, each with its own ASC or DESC direction and its own NULLS FIRST or NULLS LAST placement. The evaluated engine implements these forms.
3. **MEASURES.** Defines computed expressions that extract values from matched row sequences using pattern variables and navigation functions (FIRST, LAST, PREV, NEXT). Each measure expression becomes a column in the output, allowing a match to be summarized without returning every raw input row.
4. **ROWS PER MATCH.**
 - ONE ROW PER MATCH: one summary row per complete match. This is the default when the option is omitted.
 - ALL ROWS PER MATCH: one output row for each included row in a match. It takes one optional modifier: SHOW EMPTY MATCHES adds an output row when a pattern completes without consuming an input row, OMIT EMPTY MATCHES suppresses such rows, and WITH UNMATCHED ROWS also returns rows that no match covers.
5. **AFTER MATCH SKIP.** Controls where the matcher resumes after finding a match:
 - SKIP TO NEXT ROW – resume from the row after match start (maximum overlaps).

- SKIP PAST LAST ROW – resume from the row after match end (non-overlapping matches). This is the default.
 - SKIP TO FIRST *var* – resume from the first row classified as *var*.
 - SKIP TO LAST *var* – resume from the last row classified as *var*. Written without FIRST or LAST, SKIP TO *var* means the same thing.
6. **PATTERN.** Specifies the sequence using regular-expression-like syntax with pattern variables as terminals:
- Concatenation: A B C (A followed by B followed by C).
 - Alternation: A | B (either A or B).
 - Quantifiers: A* (zero or more), A+ (one or more), A? (zero or one), A{i} (exactly *i*), A{i, j} (between *i* and *j*). A quantifier is *greedy* by default: it takes as many rows as it can while the rest of the pattern still matches. A trailing question mark makes it *reluctant*, so it takes as few as possible: A*?, A+?, A??. or A{i, j}?.
 - Grouping: (A B)+ (one or more AB pairs).
 - Anchors: ^A B C\$ (pattern must span the entire partition).
 - Exclusions: {- A+ B+ -} (rows matching the enclosed sub-pattern are excluded from row-level output).
 - Permutations: PERMUTE(A, B, C) (any ordering of A, B, and C).
7. **SUBSET.** Defines a union variable from variables already used in PATTERN. It does not change the match. Instead, it gives one name to rows classified by any of its component variables. It can then be referenced in MEASURES, DEFINE, and supported skip targets. Of the eight specifications, it is the one used least often in practice.
8. **DEFINE.** Assigns Boolean predicates to pattern variables. The matcher tests a candidate row in the current match context to decide whether it can be classified as a variable. Predicates may use current-row columns, navigation functions, aggregations over matched rows, and Boolean or comparison operators [2]. A variable without a DEFINE predicate uses the implicit true predicate. The standard still requires the clause itself. The evaluated engine relaxes that point and also accepts a query with no DEFINE, in which case every pattern variable is true.

The clause forms a table expression inside a SELECT query. An outer query can therefore filter, group, aggregate, or order its result where the SQL dialect allows [2].

These specifications separate the sequence to detect from the way matches are classified and returned. PATTERN defines the sequence, while DEFINE gives the row-classification rules. The output and skip options decide how matches are shown and where the next search begins. This separation guides the system design in later chapters: parsing creates a structured query, validation checks its variables and columns, and compilation builds an automaton for execution.

2.2.2 Illustrative Example: Price V-Pattern

Listing 2-2 shows a `MATCH_RECOGNIZE` query that detects a V-shaped sequence in customer order prices.

```

1 SELECT customer_id, start_price, bottom_price, final_price,
2        start_date, final_date
3 FROM   orders
4 MATCH_RECOGNIZE (
5     PARTITION BY customer_id
6     ORDER BY    order_date
7     MEASURES
8         START.price           AS start_price,
9         LAST(DOWN.price)      AS bottom_price,
10        LAST(UP.price)        AS final_price,
11        START.order_date      AS start_date,
12        LAST(UP.order_date)   AS final_date
13     ONE ROW PER MATCH
14     AFTER MATCH SKIP PAST LAST ROW
15     PATTERN (START DOWN+ UP+)
16     DEFINE
17         DOWN AS price < PREV(price),
18         UP   AS price > PREV(price)
19 );

```

Listing 2-2: Detecting V-shaped price patterns with `MATCH_RECOGNIZE`

The query groups rows by `customer_id` and orders each group by `order_date`. A match starts with any row, because `START` has no predicate in `DEFINE`. It then has one or more price decreases followed by one or more price increases. For each match, `MEASURES` returns the starting value and date, the value at the last `DOWN` row, and the final `UP` value and date.

2.3 Chapter Summary

This chapter explained sequential pattern matching and why ordinary row-by-row predicates are difficult to use for ordered events. It covered the main `MATCH_RECOGNIZE` specifications, including partitioning, ordering, measures, output modes, skip modes, patterns, subsets, and variable predicates.

The V-pattern example showed how these parts fit together in one query and why the abstraction works with automata-based execution. Chapter 3 next compares existing implementations, their deployment models, and their practical limits.

Chapter 3: Related Work

This chapter positions the thesis within existing work on SQL row-pattern recognition and Python-based tabular analytics. Section 3.1 reviews systems that provide native `MATCH_RECOGNIZE` or related row-pattern support. Section 3.2 discusses the gap in Pandas and similar DataFrame libraries. Section 3.3 summarizes how this thesis addresses that gap.

Row pattern recognition was first proposed as a SQL standard feature by Zemke et al. [3] and was officially adopted in SQL:2016 [9], [23]. Later standards guidance and technical literature documented its semantics in more detail [4], [10], [20]. Further work studied practical implementations and applications [1], [5].

The `MATCH_RECOGNIZE` clause is supported natively by several SQL and stream-processing engines, including Oracle [11], [12], [13], [26], Snowflake [16], and Trino [14], [15]; Apache Flink also exposes row pattern recognition through Flink SQL [8]. By contrast, PostgreSQL 18 still lists the related SQL features as unsupported, while DuckDB lists native `MATCH_RECOGNIZE` as planned work [17], [18], [27], [28]. In systems without native support, users typically emulate row-pattern matching using window functions, recursive queries, user-defined aggregates, procedural layers, or query translation [7], [29], [30], [31]. These alternatives can express many of the same ideas, but they are usually more verbose and harder to maintain than a declarative row pattern clause.

3.1 Existing Systems and Approaches

Oracle [11], [12], [13], Trino [14], [15], Snowflake [16], and Flink [8] provide SQL-based row-pattern recognition. Their scope and execution models differ. Oracle and Snowflake target database or warehouse workloads. Trino is a distributed query engine, while Flink targets streams. This variety shows the value of the abstraction, but it does not give Pandas users an in-process implementation.

The systems reviewed here are representative rather than exhaustive. They were selected because they provide semantic references, performance baselines, streaming alternatives, or in-process designs that are relevant to this thesis. Other platforms also provide the clause. BigQuery documents `MATCH_RECOGNIZE`, and Databricks introduced beta support in Runtime 19.0 [32], [33].

3.1.1 Comparative Overview of Existing Implementations

Section 2.2 introduced the three optional row-pattern features in the SQL standard: R010 (FROM-clause patterns), R020 (window patterns), and R030 (full aggregate support). In practice, vendor documentation normally describes the supported syntax instead of declaring conformance by feature code. The comparison therefore describes the documented scope directly (See Table 3-1). It does not claim that all systems provide identical SQL semantics.

Table 3-1: Documented scope and deployment of selected row-pattern systems and approaches

System	Documented scope	Deployment	License
Oracle	FROM-clause row patterns	On-premises / cloud	Proprietary
Trino	FROM and window row patterns	Server / cluster	Apache 2.0
Snowflake	FROM-clause row patterns	Cloud service	Proprietary
Apache Flink	Streaming SQL subset over a complex-event-processing engine	Streaming cluster	Apache 2.0
DuckDB	No native clause; external research transpiler	In-process	MIT
This work	Evaluated R010-style subset	In-process Python	MIT

3.1.2 Oracle

Oracle introduced SQL row-pattern recognition in Database 12c [11]. Oracle 21c, the version used in this thesis, is a main reference for relational MATCH_RECOGNIZE semantics [12], [13]. Its documentation places MATCH_RECOGNIZE in a table expression; it does not document the R020 window form. It also prohibits some nesting and correlated-subquery forms in MEASURES and DEFINE. Its documented aggregate support is limited to COUNT, SUM, AVG, MAX, and MIN, without DISTINCT [12]. Oracle uses a proprietary license and requires a separate database deployment. This differs from an in-process, open-source package.

3.1.3 Trino

Trino is an open-source distributed query engine designed for parallel, federated query execution across large data sources [14], [15]. Trino 473, the version used in this thesis, is a semantic reference for selected results and a performance baseline. Its documentation covers MATCH_RECOGNIZE in the FROM clause [14] and also documents row pattern recognition in window structures [34]. This gives Trino broad documented coverage among the reviewed open-source systems. It is still a query server rather than an in-process DataFrame library.

3.1.4 Snowflake

Snowflake is a managed cloud data platform. It documents MATCH_RECOGNIZE support, with rich DEFINE and MEASURES expressions for row-pattern queries [16], [24]. Its documentation also notes implementation-specific behavior. The engine uses backtracking, and some pattern and data combinations can take a long time to execute. Some row-pattern functions are also restricted to particular clauses. Snowflake therefore recommends selective conditions and bounded quantifiers when possible. Because it is a managed cloud warehouse, computational cost also depends on warehouse size and query workload [16].

3.1.5 Apache Flink

Apache Flink is designed for real-time stream processing. It exposes `MATCH_RECOGNIZE` through Flink SQL, the SQL interface to Flink’s streaming engine, backed by its Complex Event Processing (CEP) runtime library [8], [35], [36]. Its documented implementation is a SQL subset oriented toward streaming use cases. The current Flink documentation lists unsupported features including pattern groups, subsequence alternation, `PERMUTE`, anchors, exclusions, `ALL ROWS PER MATCH`, `SUBSET`, physical offsets (`PREV/NEXT`), and `DISTINCT` aggregations [8].

Flink fits continuous event streams, but it assumes a streaming runtime rather than a lightweight, batch-oriented DataFrame workflow.

When pattern output is later used in Python-based machine learning, server and cluster systems may require additional integration between SQL execution and modeling tools [37], [38]. In many practical pipelines, this can introduce data movement, duplicated intermediate data, or more complicated orchestration steps [39].

3.1.6 Bringing `MATCH_RECOGNIZE` to DuckDB

DuckDB is an in-process analytical database [40]. It needs no separate server, and its Python API can query a Pandas DataFrame directly from SQL [41].

At the time of review, `MATCH_RECOGNIZE` has not yet been implemented in DuckDB itself; its roadmap lists the clause as planned work [18]. The database group at the University of Tübingen instead translates row-pattern queries into SQL that DuckDB already supports. Their paper *Democratize `MATCH_RECOGNIZE`!* presents this route, and a bachelor’s thesis and further student research projects in the same group develop it [28], [29], [30], [31]. A transpiler is a source-to-source compiler: it rewrites code into another language at the same level of abstraction, rather than down to machine code [42]. Here both sides are SQL. The input is a `MATCH_RECOGNIZE` query, and the output is SQL that DuckDB already runs. Their transpiler produces non-recursive and recursive common table expressions (CTEs) and then executes the translated SQL in DuckDB. A CTE is a named temporary result set, written with `WITH` at the start of a query and referenced later in the same statement. A recursive CTE may also refer to itself, which is how plain SQL can walk a row sequence step by step. This approach expresses pattern variables, ordering, skip behavior, and measures without adding a native operator to DuckDB. It still requires translation and execution through a second engine.

3.1.7 Optimization Research

Optimization work on row patterns shares one broad goal: reduce unnecessary work before or during the automaton scan without changing the query result. The approaches differ in where they place that work.

Zhu et al. study how row-pattern queries over historical relational data can be optimized [43]. Their approach finds pattern variables whose conditions can be checked independently. It uses relational joins to remove rows that cannot take part in a match, then runs a non-deterministic finite automaton (NFA) matcher on the smaller input. Cost estimates for the joins and the automaton guide this choice.

This thesis addresses a different execution environment. It compiles conditions and match plans for in-memory arrays, then uses the general execution path when an optimization does not apply.

3.1.8 Implementation Restrictions Across Systems

The comparison shows three recurring differences [5], [8], [12], [14], [16]. First, some systems support only the FROM-clause form, while others also support window patterns. Second, function and aggregate coverage differs. Third, empty matches, exclusions, reluctant quantifiers, and skip modes are not uniform across implementations.

Oracle, Trino, Snowflake, and Flink also require a database, cluster, stream runtime, or cloud service. That setup is reasonable for workloads that already live in those environments. For a smaller dataset that fits in the memory of one machine, it is unnecessary cost: the system must be installed and configured, and the data must be moved into it before the first pattern query can run [44], [45], [46].

3.2 Pandas and the Python Ecosystem Gap

Pandas is a popular open-source Python library for data analysis [19]. Easy installation, broad adoption, and tight integration with the Python data-science stack make it a common choice for tabular analysis without a separate database. It provides primitives for sorting, filtering, grouping, rolling windows, shifting columns, and time-series analysis.

But those are low-level building blocks, not a declarative row-pattern language. Expressing a multi-step sequence usually means stitching together `groupby`, `shift`, `rolling`, custom loops, or intermediate Boolean columns. The code becomes verbose and error-prone as patterns lengthen, need overlap control, or refer to already-matched rows.

The Pandas 2.3.3 API used in this thesis has no native SQL:2016 `MATCH_RECOGNIZE`-style primitive [47]. The logic must therefore be assembled by hand, as above, or sent to another engine. Other DataFrame libraries, including Dask, Polars, and Modin, are outside the Pandas-specific implementation and evaluation scope of this thesis [45].

Research gap in the Pandas ecosystem. The reviewed systems expose a deployment and workflow gap. Each fits when the data already lives in its environment or needs distributed or stream processing. None is the direct execution path for a Pandas DataFrame already held in a notebook or script.

DuckDB reduces the setup cost because it runs in-process. The reviewed row-pattern approach still translates the query and executes it inside DuckDB. The gap addressed here is more specific: a Pandas-native interface that keeps the declarative SQL structure without a second engine or a transpilation step.

That sets the scope of this work. The engine targets in-process, offline DataFrame analytics and an evaluated R010-style subset with selected SQL:2016 constructs. It is not intended to replace a distributed database or stream processor.

3.3 Contributions of This Work

The three objectives in Section 1.3 lead to three connected contributions: the engine and its tested semantics, its optimized execution design, and the evaluation.

1. A **Pandas-native engine** that executes an R010-style subset of SQL:2016 MATCH_RECOGNIZE directly on a DataFrame. It supports the main clauses, output modes, skip modes, navigation functions, and aggregates in the forms this thesis evaluates. Its pattern constructs include quantifiers, alternation, grouping, anchors, exclusions, and PERMUTE. It runs in-process and needs no database server or cloud warehouse. Chapter 4 presents its five-stage design.
2. An **optimized and guarded execution design** with compiled fast paths, vectorized predicate evaluation, compact output construction, bounded caching, and partition-scoped data handling. One rule governs the layer: a faster path may never change the answer. Each path declares an activation condition and runs only when that condition is proven to hold. Row-local predicates become column masks, an already-ordered input skips the sort, eligible queries fuse matching with output construction, and compiled automata are reused across queries. Separate budgets bound cache capacity, automaton construction, and exact search. Conditions outside a fast path use the general evaluator, and differential tests compare each eligible fast path with the general path on the same input and output mode. Chapter 5 presents the compilation, execution, and optimization layers.
3. **Theoretical and empirical evaluation** of the engine. The theoretical analysis gives time and space bounds for ordering, matching, output construction, and pattern complexity. The empirical study covers correctness, SQL feature coverage, usability, execution time, throughput, memory, and scalability. It runs on synthetic reference cases, Amazon UK Products 2023 up to 2.22 million rows, and Amazon Reviews 2023 up to 227.9 million rows [48], [49], and compares the engine directly with Trino 473 and Oracle 21c EE. Chapter 6 gives the full method and results.

3.4 Chapter Summary

Oracle, Trino, Snowflake, and Flink show how row patterns work in database and streaming environments. DuckDB research shows a translation-based, in-process alternative. Optimization research shows how early filtering can reduce matching work.

Pandas 2.3.3 still has no direct row-pattern interface. Users must write procedural DataFrame logic or call another engine. This leaves the focused gap addressed by the thesis: declarative row-pattern recognition inside an in-memory Pandas workflow.

Chapter 4: System Design and Methodology

This chapter describes how the engine turns a `MATCH_RECOGNIZE` query into an executable plan for a Pandas DataFrame [50]. It also explains the main design decisions. The focus is the front end: the stages that turn raw SQL into a validated execution configuration for the implemented row-pattern subset [9]. Chapter 5 continues with automata compilation, execution, and optimization.

4.1 Architecture Overview

The engine takes two inputs: an SQL query and a Pandas DataFrame. It returns a result DataFrame. The work has five stages (See Figure 4-1). *Parsing* creates an abstract syntax tree (AST). *Semantic validation* checks references and supported clause combinations while the tree is built. *Logical-plan construction* resolves the execution configuration. *Pattern compilation* builds the automata. Finally, *execution* combines the compiled pattern with the input and builds the result.

The second stage is about meaning, not shape. The grammar has already proved the text well formed, so what remains is whether every reference resolves and whether the requested clauses may be combined. `AFTER MATCH SKIP TO LAST Z over PATTERN (A B)` parses correctly, but `Z` is not a pattern variable.

These stages are design boundaries. They do not require five separate runtime objects: parsing and validation share one traversal, and the logical plan is a group of resolved attributes. The separation still keeps errors close to their source and lets each boundary be tested before matching begins.

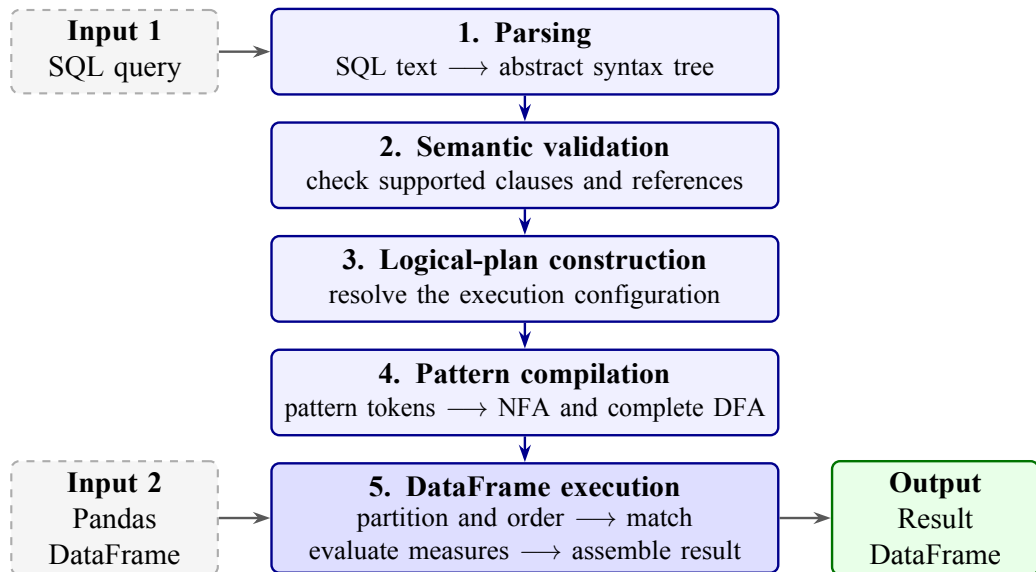


Figure 4-1: Five-stage architecture of the proposed engine.

The compiler converts the validated pattern into a non-deterministic finite automaton (NFA) and then a complete deterministic finite automaton (DFA) [51]. An NFA may be in several states at once, so it represents alternation, optional parts, and repetition directly. A DFA has exactly one next state for each input, so matching becomes a lookup instead of a search. The conversion therefore trades construction cost for a predictable scan. If complete construction exceeds a resource limit, compilation fails explicitly; it never executes a partial automaton. Chapter 5 explains compilation and DataFrame execution in detail.

Worked pipeline example. The architecture is applied here to the orders example (See Figure 4-2). The figure carries both inputs, an SQL query and an eight-row DataFrame, and traces them through the five stages (See Figure 4-1). Chapter 6 reuses the same example for the usability comparison.

The input table accounts for all eight rows. Stage 4 shows the four reachable DFA states produced for this pattern. Each transition is guarded by the predicate of its labeled variable. During execution, `cust_1` satisfies $A \rightarrow B \rightarrow C$ with prices $200 > 100 > 50$ and produces the output row. The `cust_2` candidate fails at C because $6 < 4$ is false.

The next three sections follow stages 1 to 3. Section 4.2 covers parsing, Section 4.3 semantic validation, and Section 4.4 logical-plan construction. Chapter 5 continues with Stage 4, pattern compilation (Section 5.1), and Stage 5, DataFrame execution (Section 5.2). The figure shows all five because the two chapters form one pipeline.

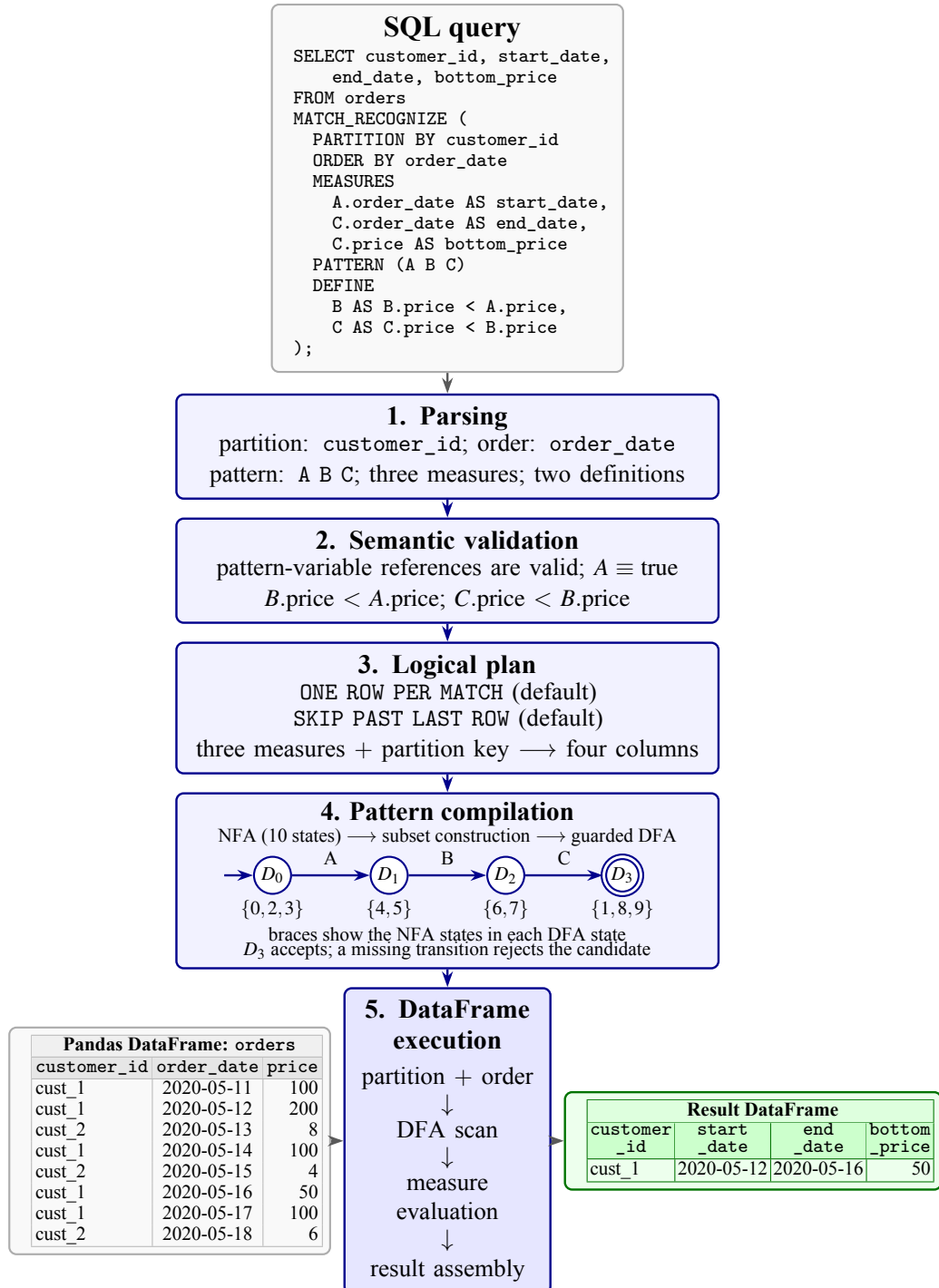


Figure 4-2: Worked orders pipeline.

4.2 SQL Parsing

Section 2.2.1 described the clause structure and the two languages it mixes. The parser must handle both at once. Listing 4-1 gives the form the core path accepts.

```

1 SELECT <expression> [, ...]
2 FROM   <table>
3 MATCH_RECOGNIZE (
4     [ PARTITION BY <column> [, ...] ]
5     [ ORDER BY <column> [ASC|DESC] [NULLS FIRST|NULLS LAST] [, ...] ]
6     [ MEASURES <expression> AS <alias> [, ...] ]
7     [ ONE ROW PER MATCH
8       | ALL ROWS PER MATCH
9         [ SHOW EMPTY MATCHES
10          | OMIT EMPTY MATCHES
11            | WITH UNMATCHED ROWS ] ]
12     [ AFTER MATCH SKIP
13       { PAST LAST ROW
14         | TO NEXT ROW
15         | TO FIRST <variable>
16         | TO LAST  <variable> } ]
17     PATTERN ( <row_pattern> )
18     [ SUBSET <subset_name> = ( <variable> [, ...] ) [, ...] ]
19     [ DEFINE <variable> AS <condition> [, ...] ]
20 ) [ AS <alias> ]
21 <rest>;

```

Listing 4-1: Clause structure used by the supported MATCH_RECOGNIZE path

The core DataFrame path targets one MATCH_RECOGNIZE clause inside a top-level SELECT, with an optional outer ORDER BY. A narrow separate path supports multiple independent clauses over self-contained inline relations. It is not a general relational planner. The DEFINE clause is optional, and a pattern variable without a predicate is treated as always true. Ordering keys support per-key ASC/DESC direction and NULLS FIRST/NULLS LAST; omitted null placement defaults to NULLS LAST. A key may also be an expression over the input columns, such as ABS(ts) or price + ts. Conditional and subquery ordering keys, rewriting of WITH clauses (CTEs), set-operation planning, and general nested subqueries remain outside the supported path.

Choosing a parser. The requirement is a typed tree for both languages at once. At the time of design, none of the reviewed Python SQL libraries covered the needed row-pattern syntax. sqlparse [52] produces only a shallow token stream. sqlglot [53] and mo_sql_parsing [54] expose a typed AST but omit constructs such as exclusions, anchors, PERMUTE, and bounded quantifiers. pglast [55] cannot parse MATCH_RECOGNIZE at all. Rather than extend one of these, the engine reuses the community grammar for the Trino SQL dialect written for ANTLR4, a parser generator that builds a parser from a written grammar [56], [57]. Its patternRecognition rule already covers all eight

sub-clauses. ANTLR4 generates the lexer, parser, and traversal scaffolding directly from that grammar, so no grammar changes were needed.

Two ANTLR4 properties matter here. Its adaptive *LL(*)* prediction examines more input when a decision needs it [58]. It also rewrites direct left-recursive expression rules internally. This lets the Trino grammar keep its natural Boolean and arithmetic structure.

Syntax errors go through a custom *error listener* instead of ANTLR's console default. Each error carries a line, column, and source snippet. A hand-written recursive-descent parser would duplicate a large SQL grammar. Regular expressions also cannot model nested pattern groups.

From parse tree to a typed AST. ANTLR4's parse tree is a *concrete syntax tree*. It contains the full grammatical structure, including keywords and punctuation that execution does not need. The engine converts this tree to a simpler abstract syntax tree (AST) using the *Visitor* pattern [57], [59]. This behavioral design pattern separates an operation from the objects it acts on. The same tree can then support different tasks, such as building an AST or checking a query, without changing its node classes.

The *client* starts the task. The *visitor interface* defines an operation for each node type. A *concrete visitor* provides the code for those operations. The *element interface* allows a node to accept a visitor. The *concrete elements* are the different node types in the tree, such as query and clause nodes.

The visitor passes itself to a node through an *accept* method. The node then calls the visitor operation for its own type. The node type selects the operation, while the concrete visitor determines what that operation does. This two-part selection is called *double dispatch*.

This process is shown as a pipeline using `SELECT price FROM orders` (See Figure 4-3). The visitor records `price` as the selected column and `orders` as the source table. The keywords guide parsing, but are not kept as separate token nodes in the AST. A different visitor could display the same query parts instead. The nodes stay the same; only the visitor's task changes.

ANTLR4 generates the visitor base class. Two visitors extend it to build the engine's compact, typed AST. One handles the outer query. The other handles the row-pattern clause. The generated node classes can remain unchanged. The AST root holds the outer `SELECT` and `FROM` parts and the row-pattern clause. Each sub-clause is stored in a dataclass, a structured record with named fields. Later stages can work with these query parts instead of reading the raw SQL again. Expression leaves remain textual by design. `DEFINE` predicates and `MEASURES` expressions are extracted as strings and parsed later by the restricted evaluator. This keeps the outer query AST compact.

This structure is shown for the worked `orders` query (See Figure 4-4). The outer clauses and the row-pattern clause share one root. Each measure stores its expression and output alias as separate fields. The orange leaves show the expressions kept as text, rather than expanded into operator trees at this stage. Names and ordering options are stored values. The pattern text goes to the pattern compiler, not the expression evaluator.

Safe expression evaluation. These predicate and measure strings come from the user, so the engine never runs them as Python code. It reads each one into a small tree of operations and walks that tree itself. This is the *Interpreter* pattern: a parser turns the text into a tree, and an evaluator walks that tree, interprets each node, and returns a value.

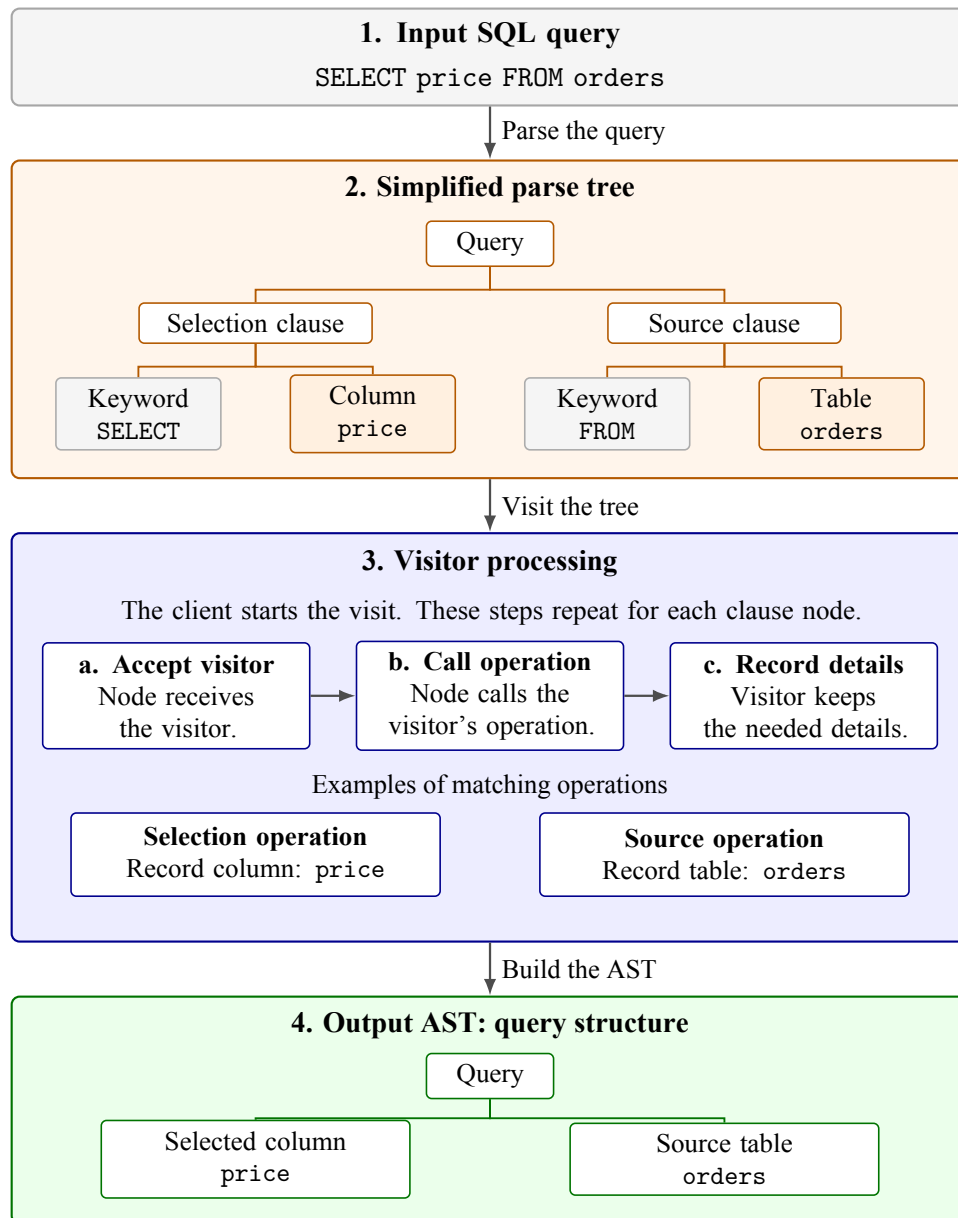
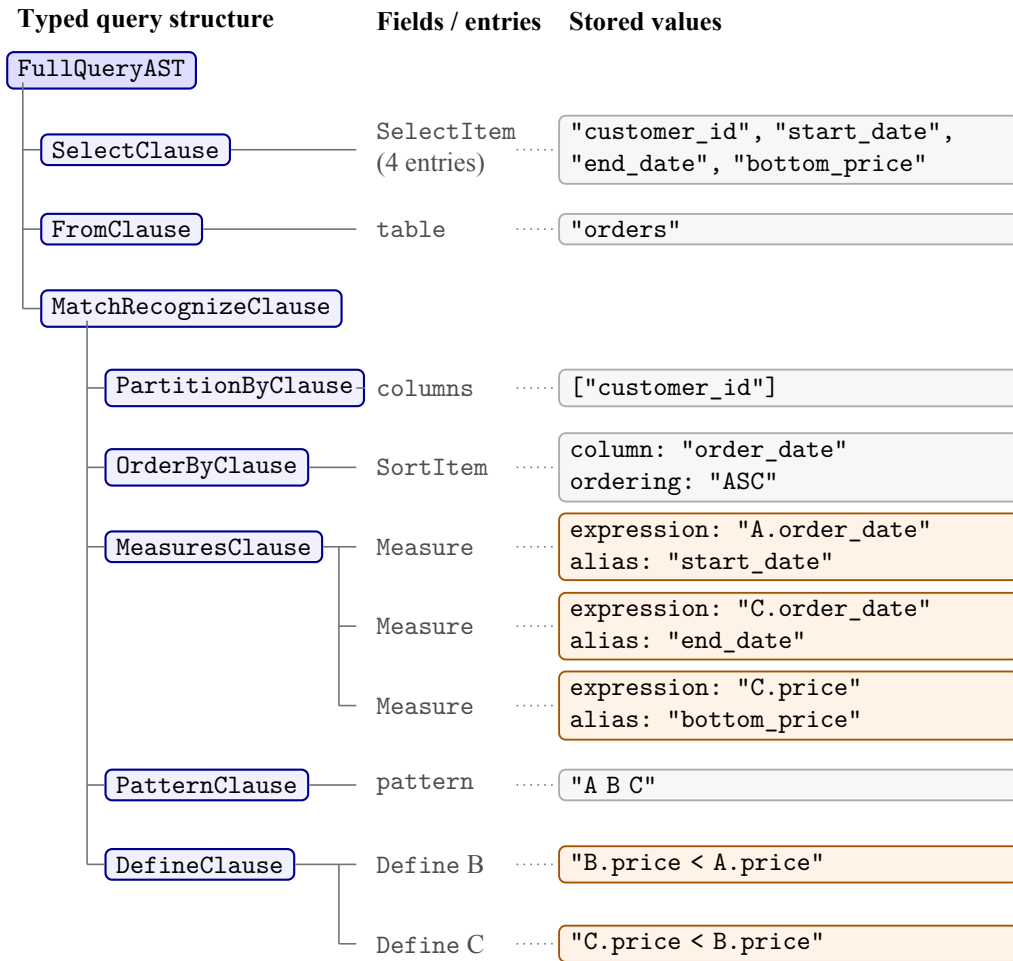


Figure 4-3: Pipeline for *SELECT price FROM orders*. Steps a and b show double dispatch: the node selects the operation, and the visitor supplies its behavior. Keyword tokens are not kept as separate AST nodes. The output describes the query; it does not contain query results.



Blue boxes: typed records. Gray leaves: names, options, and pattern text.

Orange leaves: expressions retained as text; measure aliases are shown separately.

Figure 4-4: Typed AST for the worked orders query in Figure 4-2. The tree separates the outer query from its row-pattern clause. Expression strings remain unexpanded. Metadata and absent optional clauses are omitted.

The walk uses an *allowlist*: an operation is evaluated only when the engine has a handler for it. Safety therefore comes from the allowlist itself, not from blocking unsafe cases one at a time.

The allowlist includes comparisons, Boolean and arithmetic operators, literals, IN lists, conditional expressions, qualified references such as `A.price`, navigation functions, `CLASSIFIER`, `MATCH_NUMBER`, and the supported aggregates. Everything else falls outside it: imports, assignments, and access to Python built-ins are never evaluated. An expression sees only its explicit evaluation context: ordered rows, pattern bindings, and match metadata.

The front end is exercised by dedicated tests. They include adversarial inputs such as undefined variables, prohibited function forms, invalid skip targets, and incompatible output-mode combinations. They also include end-to-end scenarios adapted from Trino's row-pattern tests.

Outside this path. General subquery planning and SQL features outside the supported row-pattern subset are not handled here. Vendor extensions may also need parser and evaluator changes.

4.3 Semantic Validation

Parsing proves that a query is well formed, but not that it is semantically valid. For example, `PATTERN (A B) with DEFINE B AS B.price > Q.price` parses correctly, but `Q` is not declared by the pattern.

A validation pass runs while the AST is completed and before any automaton is built. It rejects invalid references and unsupported clause combinations. The next stage then supplies defaults and builds the execution configuration. At this point no input row has been matched, so errors can still identify the clause that caused them.

Every reference must resolve. This is ordinary *name resolution*, with the pattern as its scope. For ordinary patterns, the front end checks qualified references in `DEFINE` and `MEASURES` against the pattern variables and `SUBSET` aliases. Specialized `PERMUTE` validation continues during pattern handling. A subset such as `SUBSET U = (A, B)` lets `U.price` refer to rows matched by either member. A variable without `DEFINE` is valid because its predicate is implicitly true. DataFrame column names are resolved later, when the execution configuration is applied to the input.

Calls must use a supported form. The front end checks the syntax of navigation functions and selected aggregates. Detailed aggregate argument checks also run before an aggregate is evaluated. For example, `COUNT_IF` takes one condition, while `SUM_IF` and `AVG_IF` take a value and a condition. This validation does not try to prove that a predicate can become true; a valid predicate may simply produce no match.

Settings must work together. Some combinations ask for contradictory output. An exclusion, written `{- ... -}`, keeps matched rows out of the result, while the `WITH UNMATCHED ROWS` form of `ALL ROWS PER MATCH` asks for every row to appear. One removes what the other requires, so the standard forbids the pair. The validator therefore rejects it. An `AFTER MATCH SKIP` target must also name a pattern variable or subset. Whether the target occurs in a particular match is known only at runtime. The matcher therefore rejects an absent target or a skip position that would make no progress. The

output mode, skip mode, implicit predicates, and measure modes then pass to logical-plan construction (Section 4.4).

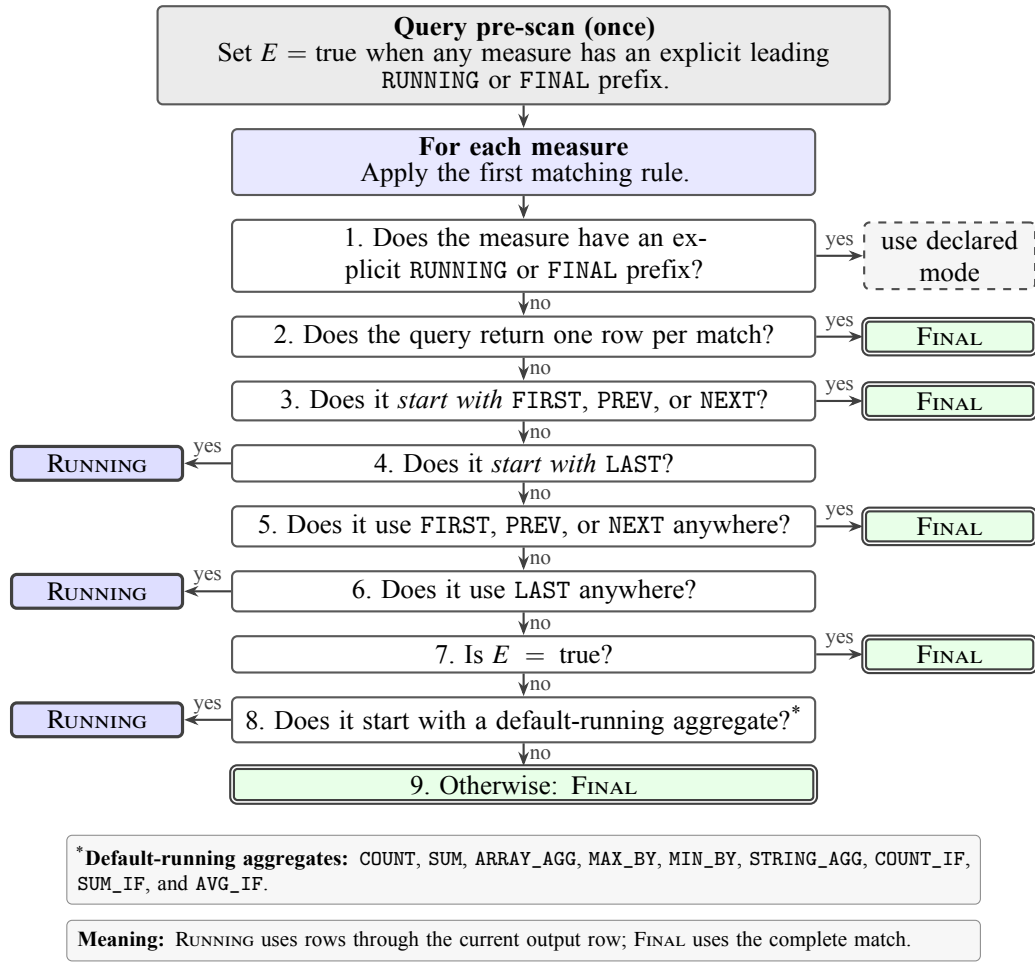
4.4 Logical Plan Construction

The logical plan is resolved as an execution configuration. It collects the typed AST fields pulled from the `MatchRecognizeClause` that compilation and matching need (See Table 4-1). Later stages do not reparse the full SQL query. Pattern, predicate, and measure text remain available to their dedicated compilers and evaluators.

Table 4-1: Principal fields in the logical execution configuration

Attribute	Type	Semantics
<code>partition_by</code>	<code>List[str]</code>	Columns for DataFrame partitioning; empty means a single global partition
<code>order_by</code>	<code>List[str]</code>	Column names or supported expressions used for intra-partition ordering
<code>order_ascending</code>	<code>List[bool]</code>	Per-key ascending or descending direction
<code>order_nulls</code>	<code>List[str]</code>	Per-key null placement; omitted placement defaults to <code>NULLS LAST</code>
<code>pattern_text</code>	<code>str</code>	Normalized pattern string forwarded to the tokenizer
<code>define</code>	<code>Dict[str, str]</code>	Explicit variable $\rightarrow$ predicate mappings; absent mappings are treated as implicit true during matching
<code>subset_dict</code>	<code>Dict[str, List[str]]</code>	Subset alias $\rightarrow$ constituent variables
<code>measures</code>	<code>Dict[str, str]</code>	Measure alias $\rightarrow$ expression string
<code>measure_semantics</code>	<code>Dict[str, str]</code>	Measure alias $\rightarrow$ "RUNNING"/"FINAL", per the priority rules (See Figure 4-5)
<code>rows_per_match</code>	<code>RowsPerMatch</code>	Output-row multiplicity
<code>skip_mode</code>	<code>SkipMode</code>	After-match skip strategy
<code>skip_var</code>	<code>Optional[str]</code>	Target variable/alias for <code>TO FIRST/TO LAST</code>

Two defaults matter. A variable without `DEFINE` uses an implicit true predicate. Measure semantics follow a set of priority rules (See Figure 4-5). An explicit `RUNNING` or `FINAL` prefix wins. `ONE ROW PER MATCH` uses final values. Under `ALL ROWS PER MATCH`, navigation rules are resolved before aggregate defaults. `RUNNING` sees rows through the current output row; `FINAL` sees the complete match.

Figure 4-5: Resolving *RUNNING* and *FINAL* semantics.

With these attributes resolved, the execution configuration is handed to the pattern-compilation phase of Chapter 5.

4.5 Chapter Summary

This chapter presented the five-stage design and its front-end boundaries. Parsing and validation create a checked AST. Logical-plan construction resolves the execution configuration, including output, skip, and measure modes. The design reuses the Trino ANTLR4 grammar and evaluates user expressions through a restricted AST evaluator. Chapter 5 continues with automata compilation, Pandas execution, and optimization.

Chapter 5: Implementation and Optimization

This chapter shows how the validated logical plan from Chapter 4 becomes an executable query. A pattern is compiled into a finite automaton and applied to ordered Pandas partitions. Guarded fast paths reduce repeated Python work without changing the supported semantics. Section 5.1 covers Stage 4: pattern compilation. Section 5.2 covers Stage 5: DataFrame execution. Section 5.3 explains the optimization layer.

5.1 Pattern Compilation with Finite Automata

This section covers Stage 4 of the architecture (See Figure 4-1). Stage 3 resolves the execution settings. Stage 4 turns the pattern into the automata used for matching. These are separate tasks, completed before the rows are scanned in Stage 5.

Row-pattern matching is a sequence-recognition problem. A row may satisfy one or more pattern variables. A match assigns consecutive rows to those variables in an order accepted by the pattern. Finite automata provide a natural representation for this ordered recognition task [51].

A finite automaton is a small machine with a fixed set of states. It starts in one state and reads the rows one at a time. A row it accepts moves it along a transition to another state. If it arrives at an accepting state, the rows it has read form a match. Nothing else is remembered: the current state carries everything the machine knows so far. That is what *finite* means here. The engine therefore compiles the pattern before it scans the data.

From pattern to NFA. The first automaton is a *non-deterministic finite automaton* (NFA). It may follow several branches and take ϵ -transitions, which consume no row. These properties represent alternation, optional parts, and repetition directly. The builder follows Thompson’s construction. Each pattern part becomes a small fragment, and the fragments are joined to form the whole pattern [60], [61]. The implementation also carries SQL-specific metadata for conditions, exclusions, anchors, and preference order. A simplified NFA for $A^+ B^?$ accepts the same row sequences as the compiled pattern (See Figure 5-1). Intermediate wrapper states used during construction are not shown.

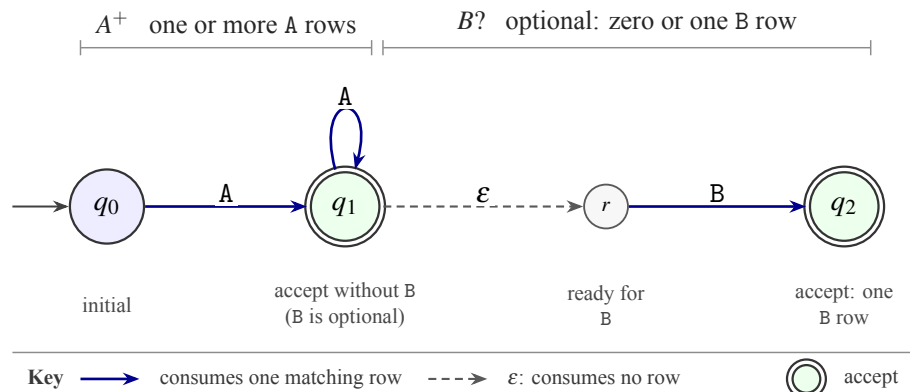


Figure 5-1: Language-equivalent NFA for $A^+ B^?$. The B row is optional: after one or more A rows, q_1 accepts without B , while q_2 accepts after one B row.

An A or B transition consumes one row for which that variable's DEFINE predicate is true. State q_1 is reached after one or more A rows and is accepting because B is optional. From q_1 , the NFA may accept, repeat A, or enter the optional fragment through the ε -transition. The transition into the connector state r consumes no row. Consuming one B row then reaches the second accepting state, q_2 . These alternatives show nondeterminism. The default greedy preference is stored as transition metadata and is not represented by the simplified state graph.

Reading the pattern. A left-to-right tokenizer reads variables, alternation, grouping, anchors, exclusions, and PERMUTE. It normalizes $*$, $+$, $?$, and $\{i, j\}$ into minimum and maximum counts together with a greedy or reluctant preference. The builder can then handle every quantifier through one internal representation.

Building the automaton. Each token contributes an NFA fragment. When several continuations are possible, transition metadata preserves their pattern order. PERMUTE over m variables represents all $m!$ orders. Construction is therefore guarded at eight variables and an estimated 50,000 branches. The two numbers are one decision: eight is the largest m whose $m!$ still fits under the branch ceiling, since $8! = 40,320$ stays below it while $9! = 362,880$ passes it seven times over. Both are checked before any fragment is built: the variables are counted, and the branches are estimated from the pattern. Exceeding either guard raises an explicit complexity error, so the engine never starts a construction it cannot finish. Rows inside $\{- \dots -\}$ still take part in matching; only their output is excluded.

From NFA to DFA. The NFA is converted into a deterministic finite automaton (DFA) by subset construction. Each DFA state represents a complete set of NFA states. Its ε -closure includes every state reachable without consuming a row. Reusing a previously seen set prevents duplicate DFA states [51].

The resulting automaton has no ε -transitions. For eligible row-local pattern structures, the matcher can advance through the ordered partition without keeping the complete NFA frontier at every row, that is, the full set of states the NFA could be in. Predicates or structural choices that depend on tentative assignments use the exact search described in Section 5.2.1.

Subset construction is exponential in the worst case. The engine therefore applies a memory-aware state budget, an iteration limit, and a structural ceiling of 50,000 states. This ceiling counts DFA states, and it is checked after the construction guards above, so a pattern that passes those can still be stopped here. The memory budget may lower that ceiling on a constrained system. Reaching any limit raises a construction error. The engine never executes or caches a partial DFA, and it never truncates an NFA subset. None of the evaluated patterns reached these limits.

After construction, unreachable states are removed and transitions are normalized. The current reduction is conservative; it does not perform full DFA minimization, which would also merge states that behave identically for every remaining input. Transition metadata is preserved so the runtime can apply the required greedy and leftmost preference.

5.2 Pandas DataFrame Execution Model

This section covers Stage 5. It uses the execution settings from Stage 3 and the compiled pattern from Stage 4 (Section 5.1). Logical planning decides how the query should run; pattern compilation prepares the automata; execution applies them to the input rows.

Pandas [19] stores each DataFrame column as an array. This suits both row-by-row matching and the *vectorized* fast paths in Section 5.3, which test a condition on a whole column at once instead of one row at a time. The system boundary appears in the architecture overview (See Figure 4-1); Stage 5 is expanded here together with the preceding planning and compilation stages (See Figure 5-2). Stage 5 takes the execution settings with the compiled pattern and the Pandas DataFrame. Its five logical execution steps are labelled 5a–5e: partitioning, ordering, match enumeration with predicate evaluation, measure evaluation, and result materialization. They are operations within Stage 5, not another set of architecture stages. An optimized physical path may fuse some operations without changing this logical flow.

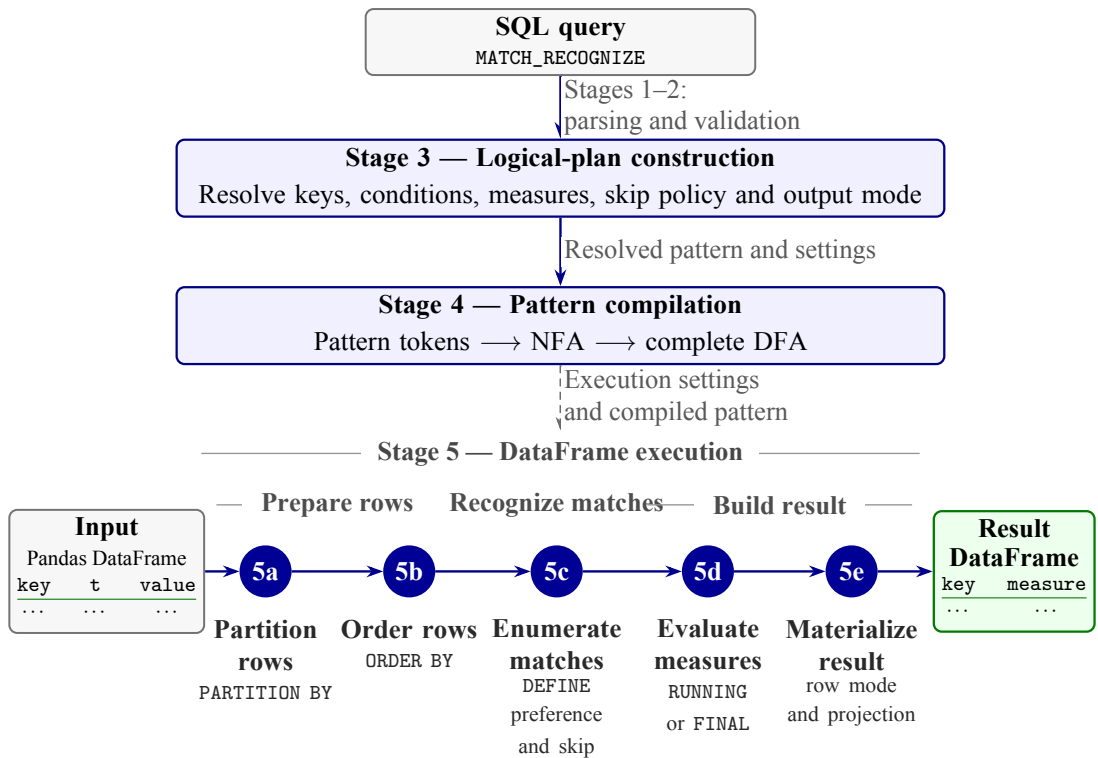


Figure 5-2: Logical planning (Stage 3), pattern compilation (Stage 4), and DataFrame execution (Stage 5). Steps 5a–5e are operations within Stage 5.

Preparing the data. A match never crosses a partition boundary, so the DataFrame is first split on the PARTITION BY columns, or treated as one partition when there are none. The engine checks the ORDER BY columns. It keeps a partition that already follows the complete ordering specification and sorts it otherwise. Each key may use ASC or DESC and its own null placement. The sort is stable, so rows with equal keys keep their

DataFrame order. This avoids repeating a sort whose result is already present. Because matches do not cross partitions, the remaining steps can process one ordered partition at a time.

Each DEFINE condition is turned into a small function once, at the start of the query. A variable used in PATTERN but not in DEFINE receives an always-true condition, as required by the supported semantics [14]. Row-local conditions may be evaluated for the whole partition. Other conditions are evaluated only when a candidate assignment needs them.

Finding the matches. Matching walks the rows in order. At each position, it tests the possible variable transitions and applies the pattern's preference. Greedy quantifiers prefer a longer valid match. Reluctant quantifiers prefer a shorter one. Alternation follows declaration order. The automata carry this ordering information, while the runtime checks that the preferred path can still complete.

Each completed match records its start, end, variable assignments, and excluded rows. This state drives skip handling and measure evaluation. Exclusion changes only the rows emitted; it does not change the rows used by conditions or measures.

5.2.1 Exact Search for State-Dependent Conditions

The deterministic walk is used only when the predicates and pattern structure can be decided without revisiting tentative assignments. Other conditions depend on the tentative match. For example, accepting a row changes a running average used by a later condition. Some nested quantified alternatives also need a choice to be tested before their preference can be decided.

These cases use a preference-ordered depth-first search with backtracking. The search tentatively assigns rows, evaluates the affected conditions, and tries the next legal choice when a later step fails. It is iterative, so Python's recursion depth does not limit the match length. A state budget and a search budget bound the explored states and steps. If either is exhausted, the query raises an explicit error instead of returning a partial or different match. Eligible row-local patterns remain on deterministic fast paths. Section 6.5.6 measures one supported state-dependent case.

5.2.2 Skip Modes, Output Modes, and Measures

After recording a match the engine must decide where to resume, which controls whether matches can overlap. SQL:2016 defines four AFTER MATCH SKIP modes, summarized with their next-start computation (See Table 5-1). Empty matches always advance by one row. A variable-based target that does not occur in the match, or that points back to the current start, raises an error. This prevents a silent loop or a changed match sequence.

The engine supports ONE ROW PER MATCH and ALL ROWS PER MATCH. The latter includes SHOW EMPTY MATCHES, OMIT EMPTY MATCHES, and WITH UNMATCHED ROWS [14]. Empty matches are shown by default; omitting them removes their output rows, while unmatched mode also emits input rows not covered by a non-empty match. CLASSIFIER() can expose the variable assigned to an output row. The row counts and measure contexts are summarized together (See Table 5-2).

Table 5-1: The four AFTER MATCH SKIP modes and the resulting next scan position

Mode	Next-start computation
PAST LAST ROW (default)	$i \leftarrow best.end + 1$. No overlap is possible; the scan resumes immediately after the last matched row.
TO NEXT ROW	$i \leftarrow best.start + 1$. Overlapping matches are produced because only a single row is skipped regardless of match length.
TO FIRST V	$i \leftarrow \min(bindings[V])$. The scan resumes at the <i>first</i> row assigned to variable V (V may also be a SUBSET alias).
TO LAST V	$i \leftarrow \max(bindings[V])$. The scan resumes at the <i>last</i> row assigned to variable V (or SUBSET alias).

Table 5-2: Output modes and their row-count and measure-semantics behavior

Mode	Rows out	Measure evaluation context
ONE ROW PER MATCH	1 per match	Measures use the completed variable bindings.
ALL ROWS PER MATCH	One per visible matched row; SHOW EMPTY adds one row for each empty match	Measures use their resolved RUNNING or FINAL semantics for each emitted row.
ALL ROWS ... WITH UNMATCHED	Visible matched rows, plus each unmatched input row; empty matches follow the selected empty-match option	Matched rows carry measure values. Unmatched rows emit NULL for the measure columns.

Measure semantics are resolved before matching. Common navigation and aggregate forms are compiled once; other supported expressions use the general evaluator. A qualified aggregate uses rows assigned to its variable or SUBSET. An unqualified aggregate uses the applicable RUNNING match prefix or the complete FINAL match. Direct navigation plans use bounds-checked lookups into the ordered partition, and an out-of-range reference returns NULL. MATCH_NUMBER() numbers matches within each partition, while CLASSIFIER() reports the variable assigned to the output row. Under ALL ROWS PER MATCH, a RUNNING measure sees the match prefix through the current row. A FINAL measure sees the complete match.

5.3 Performance Optimization Techniques

One rule governs this whole layer: a faster path may never change the answer. Each technique below is therefore switched on only when the engine can be sure it applies, and the ordinary path takes over otherwise. Differential tests compare eligible fast paths with the general path on the same input and output mode.

The techniques remove repeated predicate work, avoid unnecessary sorting and materialization, and reuse compiled structures. Their activation boundaries are listed together, and each technique below carries the name used there (See Table 5-3).

Ordered-input check. Sorting is the one cost that can be removed outright. Before sorting a partition, the engine tests whether it already follows the requested ORDER BY sequence. If it does, the sort and the DataFrame copy it would produce are both skipped. Section 6.5.6 measures the engine with and without that shortcut.

Vectorized condition masks. The main cost in the general path is repeated condition evaluation. Predicates that depend only on the current row, literals, and eligible fixed-offset PREV/NEXT forms can be evaluated as column operations. Their Boolean results are stored once per partition. The matcher then reads a mask instead of calling the expression evaluator for every candidate row.

The planner is conservative. FIRST, LAST, CLASSIFIER(), aggregates, and cross-variable references depend on the tentative match and cannot use a row-local result. A mixed AND predicate may still use its proven row-local conjuncts as a rejection mask; the remaining conjuncts are evaluated exactly. Supported expressions outside the compiled forms use the general evaluator. Unsupported forms are rejected.

The running-average condition used in the worst-case experiment receives a separate incremental plan. The aggregate is updated as the matched prefix grows instead of being recomputed from the beginning. This removes the quadratic repeated-aggregation cost for that supported form. It does not make every state-dependent predicate vectorizable (Section 6.5.6).

Row-local DFA decision tables. When all variable predicates have masks, the engine builds a compact transition index for the DFA states. Matching becomes array lookup and integer comparison rather than repeated dictionary and function work. Assignments can be stored as row segments instead of one Python object per row.

Linear run-length plan. A linear pattern such as A B{2,4} C+ allows a second plan. Run-length arrays record how many consecutive rows satisfy each variable. Failed token-position pairs are memoized, so the same impossible suffix is not explored again.

Table 5-3: Performance optimizations: purpose, activation condition, and benefit. The technique names match the bold paragraph headings that follow.

Technique	Purpose	Activation condition	Primary benefit
Ordered-input check	Avoid copies	Requested ORDER BY sequence already present	Avoids an unnecessary sort and DataFrame copy
Vectorized condition masks	Avoid rework	Row-local expression, or safe row-local conjuncts in a mixed AND	Replaces repeated Python evaluation with column operations and exact prefilters
Row-local DFA decision tables	Avoid rework	All predicates are vectorized, with no empty-match, anchor, exclusion, PERMUTE, SUBSET, reluctant, or state-dependent requirement	Replaces repeated transition dispatch with compact indexing and segment assignments
Linear run-length plan	Avoid rework	Default skip, a plain linear variable sequence, supported quantifiers, and row-local masks	Skips complete runs and memoizes failed suffixes
Fused output drivers	Avoid copies	Default skip, no unmatched rows, anchors, or empty matches, an eligible matcher, and fully compiled measures	Avoids intermediate match and row dictionaries
Compiled measure and prefix plans	Avoid rework	Supported direct navigation or aggregate form	Reuses prepared array state; avoids repeated prefix aggregation
LRU pattern cache	Reuse	Caching enabled and the complete automaton fits the cache budget	Eliminates redundant NFA/DFA construction across repeated queries
Lazy rows and output projection	Avoid copies	Partition path supports lazy access; selected source columns can be resolved safely	Reduces row copies, retained Python objects, and output width

Patterns with alternation use the row-local DFA path when eligible. State-dependent choices use the exact matcher. Neither case enters the linear run-length plan.

Fused output drivers. For eligible output modes, matching and materialization are fused. One path emits ONE ROW PER MATCH results directly. Another publishes compact segments for ALL ROWS PER MATCH. Both require the default skip mode and fully compiled measures. Any anchor, empty-match rule, measure, or output option outside their proof boundary returns to the complete path.

Compiled measure and prefix plans. Common measures are compiled once into direct plans. Simple SUM, AVG, COUNT, statistical aggregates, and direct navigation can then use arrays or prefix state. A RUNNING aggregate over a long match is updated incrementally instead of being recomputed for every output row. The compiled layer also covers selected arithmetic combinations and distinct aggregates. Filters and expressions outside these proven forms use the general measure evaluator.

LRU pattern cache. Compiled automata are stored in a bounded, thread-safe *least-recently-used* (LRU) cache. The key includes the pattern, its DEFINE conditions, and compilation-relevant SUBSET data. Repeating the same structure therefore avoids NFA and DFA construction. Condition caches are also bounded. For a large partition, a small row sample warms common evaluation paths.

Lazy rows and output projection. The executor avoids copying all grouped partitions before matching. For eligible default-skip, non-PERMUTE queries, it exposes one partition through a lazy row view with a bounded row cache. Other paths materialize partition rows and may retain them until output assembly. Match membership uses compact masks and segments where the semantics allow it. A conservative final-output projection gathers only source columns needed by the result; a wildcard or an expression it cannot prove safe keeps all columns. Eligible fast paths assemble output columns directly before creating the result DataFrame. These choices lower temporary memory, but they do not make execution streaming: it retains the input and result, and some paths also retain materialized rows or whole-column working arrays.

Budgets. The current implementation uses separate resource limits, called budgets. The compiled-pattern cache limits the number and estimated size of entries kept for reuse. It removes cached entries when space is needed and leaves an entry uncached if it is too large to fit. DFA construction limits both the number of states built and the construction iterations. Exact search limits the exploration steps from each candidate start row. If construction or search cannot finish within its budget, the engine raises an explicit error rather than returning an incomplete result. These checks do not cap total process memory, which also includes the input, temporary data, and output.

5.4 Chapter Summary

This chapter followed a query from a compiled pattern to a result. The pattern becomes an NFA and then a complete, guarded DFA. The input is partitioned and ordered before matching. Row-local queries use the deterministic path only when their predicates and structure are eligible. Predicates or structural choices that depend on tentative assignments use exact search and fail explicitly if its search budget is exhausted.

Major speed gains come from safe column-wise condition evaluation, linear run-length plans, compiled measures, fused output, and pattern caching. Lazy row access, projection, compact masks, and the cache and state budgets reduce temporary memory. Every fast path is guarded. If its conditions are not met, the complete path remains responsible for the result. Chapter 6 measures the resulting correctness, performance, scalability, and memory behavior.

Chapter 6: Experimental Evaluation

This chapter evaluates the implementation on five criteria: correctness against Trino and Oracle, SQL:2016 feature coverage, performance, scalability, and usability. Section 6.1 defines the methodology, Section 6.2 the datasets and patterns, Section 6.3 validates the concept example by example, Section 6.4 reports coverage and complexity results, and Section 6.5 presents the cross-system performance analysis.

6.1 Evaluation Methodology

Section 6.1.1 defines the five criteria and how each is measured; Section 6.1.2 describes the hardware and software environment.

6.1.1 Evaluation Criteria and Metrics

Correctness. The result DataFrame is compared against two independent SQL references, Trino 473 and Oracle 21c EE, by exact row-level equality (same rows, values, and order where deterministic). This covers the two small synthetic tables introduced in Section 6.2, `orders` and `stock_price`, and the 30 pattern–size combinations of the Amazon benchmark: five pattern families run at each of six sizes, so 30 cells per system and 90 in total. Each of the two tables has three columns: a key that partitions the rows, a date that orders them, and a price the pattern tests (See Figure 6-1). The SQL engines are the correctness references. The hand-written Pandas functions provide an additional output cross-check and the usability baseline (Section 6.3.4).

orders		
customer_id	→ PARTITION BY	
order_date	→ ORDER BY	
price	→ DEFINE	

Example 1

stock_price		
Company	→ PARTITION BY	
Price_date	→ ORDER BY	
Price	→ DEFINE	

Examples 2 and 3

Figure 6-1: Columns of the two synthetic tables and the clause that uses each one. The tables are independent; each serves its own worked examples.

SQL feature coverage. Two views are recorded, both reported in Section 6.4. The same boundary is written up for users in the package guide, which gives a status table for the main clauses and pattern constructs, names what is not supported, and lists the errors the API raises [62]. *Feature coverage* marks each main `MATCH_RECOGNIZE` clause as *supported*, *partially supported*, or *unsupported* against the reference tests. The clauses are partitioning, ordering, `PATTERN`, `DEFINE`, `MEASURES`, `SUBSET`, the output and skip modes, and navigation functions [9], [10], [50]. *Pattern-type coverage* groups the test cases by row-pattern type (simple sequences, quantified repetitions, trends, recovery, alternation, exclusions, anchors, and `PERMUTE`); a type is covered when at least one case matches the reference output.

Performance. Execution time is the client-observed end-to-end latency per pattern-size run on the Amazon UK Products dataset (Section 6.5): five pattern families, six sizes, result transfer included. Each cell runs five warmups, then twenty measured runs, each preceded by a five-second idle window that sets the pre-query memory baseline.

The warmups matter because a first run measures start-up rather than steady state. Trino runs on the Java Virtual Machine (JVM), which compiles its own bytecode to machine code while the early queries execute; that *just-in-time* (JIT) step makes them slower than later runs. Oracle fills its caches, and the engine compiles the pattern once.

Outliers are then removed per metric with the $1.5\times$ interquartile range (IQR) rule: the IQR is the spread between the 25th and 75th percentiles, and a run lying more than $1.5\times$ that spread outside them is dropped. It discarded 99 of the 1,800 measured time runs. Cells report the mean of the retained runs with its standard deviation. Throughput is computed per run, as dataset size divided by that run’s time, and then averaged the same way, so it is the mean of the rates rather than the rate of the mean. The two differ because a rate is one over a time, and averaging that inverse gives a slow run less weight; on the main-matrix averages the gap is 0.12% or less. For the main benchmark, the two memory metrics use one instrument for all three systems. *Query memory* is the rise above the idle baseline while the query runs, and *footprint memory* is the highest total the process or container reaches. Their scope is defined in Section 6.5.4.

Scalability. For each pattern family, the cell means are fitted with a straight line against row count by least squares. The fit is summarised by R^2 , the share of the variation in measured time that the line accounts for, where 1 is a perfect fit. It is reported for both the six-size benchmark and the larger stress test. A high R^2 describes the measured range only. It does not prove the algorithm’s asymptotic complexity, because over a range this narrow an $O(n\log n)$ cost is itself close to straight (Section 6.4.2).

Usability (developer effort). For each worked example, the line count of the idiomatic Pandas solution is compared with the single `MATCH_RECOGNIZE` statement in Section 6.3.4, counting only the detection logic on both sides. Line count is a proxy for developer effort rather than a measurement of it. It reflects how much procedural detail the author must write and later maintain, and says nothing about reading time, error rate, or learning cost [4], [6], [9], [10].

6.1.2 Environment and Configuration

The experiments ran on a workstation with an Intel Core i7-11800H processor and 64 GB of RAM under Ubuntu 24.04.4 LTS. For the cross-system benchmark, each system had the same outer control-group limit of one CPU core and 32 GB. Section 6.5.1 details the enforcement mechanisms.

The scalability stress test (Section 6.5.7) instead uses a 58 GB memory limit. Reaching hundreds of millions of rows on equal terms also meant changing the Trino deployment: it reads from a disk-backed Hive/Parquet connector rather than its in-memory connector. All database containers run on the host’s native Docker daemon, so container limits map directly to host control groups. A control group, or *cgroup*, is the Linux kernel facility that caps and reports the CPU and memory of a set of processes, and it is the same instrument used for the engine.

The software stack comprised the two baseline SQL engines, Trino 473 and Oracle 21c EE, each run as a Docker container. It also included the Java 11.0.25 LTS and Python 3.12.7 runtimes and the core Python libraries pandas 2.3.3, numpy 2.1.3, antlr4-python3-runtime 4.13.2, antlr4-tools 0.2.1, and pytest 8.3.4. The implementation itself uses ANTLR4 to parse Trino SQL, Pandas for DataFrame management, and custom AST/NFA/DFA modules for pattern detection (Chapter 4).

Unless stated otherwise, all experiments use this configuration. For correctness comparisons, identical data are loaded into all three systems and result sets are compared by row order and measure values (after normalizing case-only column-name differences).

6.2 Dataset and Test Patterns

Four datasets are used. Two are small synthetic tables that can be verified by hand, `orders` and `stock_price` (See Figure 6-1). The real-world Amazon UK Products dataset, up to 2.2 million rows, carries the cross-system timing and memory benchmark. The Amazon Reviews 2023 corpus, up to 227.9 million rows, carries the scalability test (Section 6.5.7).

Dataset 1: the `orders` table. The `orders` table supports the first worked example. It comprises three attributes [15]: `customer_id` (VARCHAR), which identifies the customer (two distinct values, `cust_1` and `cust_2`); `order_date` (DATE), the date the order was placed (ranging from 2020-05-11 to 2020-05-18); and `price` (INTEGER), the purchase amount on that date. Each customer appears multiple times in chronological order (`cust_1` spans five dates and `cust_2` spans three), and the two customers are interleaved by date. The query partitions by `customer_id`, so matching runs separately inside each customer’s rows and no match may cross from one customer to the other. The partitioning carries weight here: `cust_1`’s falling prices are consecutive only within that customer, so dropping the `PARTITION BY` leaves no match at all. The table therefore gives focused validation of partitioning, ordering, and pattern detection. The eight records used are listed in full (See Table 6-1).

Table 6-1: Sample orders dataset (`customer_id`, `order_date`, `price`)

<code>customer_id</code>	<code>order_date</code>	<code>price</code>
<code>cust_1</code>	2020-05-11	100
<code>cust_1</code>	2020-05-12	200
<code>cust_2</code>	2020-05-13	8
<code>cust_1</code>	2020-05-14	100
<code>cust_2</code>	2020-05-15	4
<code>cust_1</code>	2020-05-16	50
<code>cust_1</code>	2020-05-17	100
<code>cust_2</code>	2020-05-18	6

Dataset 2: the `stock_price` table. The `stock_price` table supports the second worked example. It represents a history of daily stock prices for market analysis and includes three columns: `company` (the company name), `price_date` (the trading date),

and price (the stock’s value on that day) [5]. Two companies, A and B, are recorded over ten trading days, yielding twenty rows that support per-company trend detection. The data are listed in full (See Table 6-2).

Table 6-2: The tabular form of the `stock_price` table

Company	Price_date	Price
A	2020-10-01	50
B	2020-10-01	89
A	2020-10-02	36
B	2020-10-02	24
A	2020-10-03	39
B	2020-10-03	37
A	2020-10-04	42
B	2020-10-04	63
A	2020-10-05	30
B	2020-10-05	65
A	2020-10-06	47
B	2020-10-06	56
A	2020-10-07	71
B	2020-10-07	50
A	2020-10-08	80
B	2020-10-08	54
A	2020-10-09	75
B	2020-10-09	30
A	2020-10-10	63
B	2020-10-10	32

Dataset 3: the Amazon UK Products dataset. The Amazon UK Products dataset [48] holds roughly 2.2 million product records from Amazon’s UK marketplace as one CSV file. The benchmark uses four of its columns: `stars` (average rating, 1–5), `reviews` (review count), `price` (GBP), and `categoryName`; the remaining columns (IDs, titles, URLs, flags) are not needed.

It supplies the data volume for the performance benchmark; the derived category column and the six dataset sizes are described in Section 6.5.1.

Dataset 4: the Amazon Reviews 2023 corpus. The scalability test (Section 6.5.7) needs far more rows than the product tables provide, so it uses the Amazon Reviews 2023 corpus [49] from the McAuley Lab at UC San Diego, roughly 571 million genuine review events across product categories. Only the pattern-relevant fields are taken: `user_id`, `parent_asin`, `rating`, and `timestamp`. Review text, images, and product metadata are excluded. Five categories are merged in global timestamp order into nested prefixes of 5 M to 227.9 M rows. Each review’s 1–5 rating is mapped to the same A–E category alphabet as the benchmark. The same five pattern families are used. A pattern family

is one `MATCH_RECOGNIZE` query, not a partition of the data: the five queries differ only in their `PATTERN`, and each is run over the whole prefix in turn. Four keep the main benchmark structure with different rating labels; the stress test’s optional family uses `ED? C+ B*`. No query in the benchmark or the stress test uses `PARTITION BY`, so every run scans one partition of the full prefix in timestamp order. This is a deliberate worst case for the engine, since a single partition removes the parallel and per-group work that partitioning would allow. Unlike a tiled copy of a smaller table, every row is a distinct real event, which makes this the more honest test of scaling behavior.

6.3 Validation of Concept

Each of the three worked examples compares the proposed engine, Trino, Oracle, and an equivalent hand-written Pandas function. The comparison covers output, exercised clauses, and developer effort. Trino and Oracle are the correctness references throughout, and both ran every benchmark case as well, not only these three examples. The hand-written functions are the usability baseline, not a fourth correctness reference. Beyond the examples, the engine is checked by the regression suite (709 passing test cases [21]), and the 30 benchmark pattern–size combinations in Section 6.5.2 agree across the three systems.

6.3.1 Example 1: Dip Pattern on the orders Dataset

The first example detects a ‘dip’, three consecutive orders with successively lower prices, on the `orders` dataset (Dataset 1; See Table 6-1). The same detection is expressed four ways: as a Trino SQL query (Listing 6-1), as an Oracle SQL query (Listing 6-2), as a call to the proposed engine (Listing 6-3), and as a hand-written Pandas function (Listing 6-4). The two SQL listings carry the same `MATCH_RECOGNIZE` clause; only the table creation and load differ, because the servers use different dialects [63].

For the *correctness* criterion, all four implementations return the same single match (See Table 6-3). The query also exercises the `PARTITION BY`, `ORDER BY`, `PATTERN`, `DEFINE`, and `MEASURES` clauses, and covers the simple-sequence pattern type.

Table 6-3: Dip-pattern result: all four systems return the identical match

Implementation	customer_id	start_date → end_date	bottom_price
Trino 473 (SQL)	cust_1	2020-05-12 → 2020-05-16	50
Oracle 21c EE (SQL)	cust_1	2020-05-12 → 2020-05-16	50
Proposed engine	cust_1	2020-05-12 → 2020-05-16	50
Pandas function	cust_1	2020-05-12 → 2020-05-16	50

Source: [50]

6.3.2 Example 2: Stock-Price Analysis on the stock_price Dataset

The second example detects bullish momentum on the `stock_price` table: a three-row window where the second and third rows each rise.

```

1 CREATE TABLE memory.default.orders (
2     customer_id VARCHAR,
3     order_date DATE,
4     price INTEGER
5 );
6
7 INSERT INTO memory.default.orders VALUES
8     ('cust_1', DATE '2020-05-11', 100),
9     ('cust_1', DATE '2020-05-12', 200),
10    ('cust_2', DATE '2020-05-13', 8),
11    ('cust_1', DATE '2020-05-14', 100),
12    ('cust_2', DATE '2020-05-15', 4),
13    ('cust_1', DATE '2020-05-16', 50),
14    ('cust_1', DATE '2020-05-17', 100),
15    ('cust_2', DATE '2020-05-18', 6);
16
17 SELECT customer_id, start_date, end_date, bottom_price
18 FROM memory.default.orders
19 MATCH_RECOGNIZE (
20     PARTITION BY customer_id
21     ORDER BY order_date
22     MEASURES
23         A.order_date AS start_date,
24         C.order_date AS end_date,
25         C.price AS bottom_price
26     PATTERN (A B C)
27     DEFINE
28         B AS B.price < A.price,
29         C AS C.price < B.price
30 );

```

Listing 6-1: Trino SQL: create and load the orders table, then detect the dip pattern with MATCH_RECOGNIZE.

```

1 CREATE TABLE orders (
2     customer_id VARCHAR2(50),
3     order_date DATE,
4     price NUMBER
5 );
6
7 INSERT ALL
8     INTO orders VALUES ('cust_1', DATE '2020-05-11', 100)
9     INTO orders VALUES ('cust_1', DATE '2020-05-12', 200)
10    INTO orders VALUES ('cust_2', DATE '2020-05-13', 8)
11    INTO orders VALUES ('cust_1', DATE '2020-05-14', 100)
12    INTO orders VALUES ('cust_2', DATE '2020-05-15', 4)
13    INTO orders VALUES ('cust_1', DATE '2020-05-16', 50)
14    INTO orders VALUES ('cust_1', DATE '2020-05-17', 100)
15    INTO orders VALUES ('cust_2', DATE '2020-05-18', 6)
16 SELECT * FROM dual;
17
18 SELECT customer_id, start_date, end_date, bottom_price
19 FROM orders
20 MATCH_RECOGNIZE (
21     PARTITION BY customer_id
22     ORDER BY order_date
23     MEASURES
24         A.order_date AS start_date,
25         C.order_date AS end_date,
26         C.price AS bottom_price
27     PATTERN (A B C)
28     DEFINE
29         B AS B.price < A.price,
30         C AS C.price < B.price
31 );

```

Listing 6-2: Oracle SQL: the same clause as Listing 6-1 over an Oracle copy of the orders table.

```

1 import pandas as pd
2 from pandas_match_recognize import match_recognize
3 orders = pd.DataFrame({
4     "customer_id": ["cust_1", "cust_1", "cust_2", "cust_1",
5     "cust_2", "cust_1", "cust_1", "cust_2"],
6     "order_date": ["2020-05-11", "2020-05-12", "2020-05-13", "2020-05-14",
7     "2020-05-15", "2020-05-16", "2020-05-17", "2020-05-18"],
8     "price": [100, 200, 8, 100, 4, 50, 100, 6],
9 })
10
11 query = """
12 SELECT customer_id, start_date, end_date, bottom_price
13 FROM orders
14 MATCH_RECOGNIZE (
15     PARTITION BY customer_id
16     ORDER BY order_date
17     MEASURES
18         A.order_date AS start_date,
19         C.order_date AS end_date,
20         C.price AS bottom_price
21     PATTERN (A B C)
22     DEFINE
23         B AS B.price < A.price,
24         C AS C.price < B.price
25 ); """
26 result = match_recognize(query, orders)
27 print(result)

```

Listing 6-3: Proposed engine: the same query as a MATCH_RECOGNIZE string over an orders DataFrame.

```

1 import pandas as pd
2
3 def detect_dip(df, cust_col="customer_id",
4               date_col="order_date", price_col="price"):
5     g = df.copy()
6     g[date_col] = pd.to_datetime(g[date_col])
7     g = g.sort_values([cust_col, date_col]).reset_index(drop=True)
8     rows = []
9     for cust, grp in g.groupby(cust_col, sort=False):
10         grp = grp.reset_index(drop=True)
11         i = 0
12         while i + 2 < len(grp):
13             a, b, c = (grp.loc[i, price_col], grp.loc[i + 1, price_col],
14                       grp.loc[i + 2, price_col])
15             if b < a and c < b: # A B C: successively lower prices
16                 rows.append({
17                     "customer_id": cust,
18                     "start_date": grp.loc[i, date_col].date(),
19                     "end_date": grp.loc[i + 2, date_col].date(),
20                     "bottom_price": c,
21                 })
22                 i += 3 # AFTER MATCH SKIP PAST LAST ROW
23             else:
24                 i += 1
25     return pd.DataFrame(rows)
26
27 result = detect_dip(orders)
28 print(result)

```

Listing 6-4: Hand-written Pandas function for the same 'dip' detection.

It partitions by company, orders by date, and uses PATTERN (A B C). DEFINE then requires B to be priced above the row before it and C above the row before it in turn, where PREV steps back one row in the ordered partition rather than one row of the same variable. A carries no condition, so it is implicitly true and any row may open a match. On the stock_price data (See Table 6-2) this returns three matches: 36 → 39 → 42 and 30 → 47 → 71 for company A, and 24 → 37 → 63 for company B.

The same detection is written as Trino SQL (Listing 6-5), as Oracle SQL (Listing 6-6), as a call to the proposed engine (Listing 6-7), and as a hand-written Pandas function (Listing 6-8). The two SQL listings carry the same clause over their own copies of the table [63]. All four implementations return the identical three matches (See Table 6-4).

```

1 CREATE TABLE memory.default.stock_price (
2     Company    VARCHAR,
3     Price_date DATE,
4     Price      INTEGER
5 );
6
7 INSERT INTO memory.default.stock_price (Company, Price_date, Price) VALUES
8     ('A', DATE '2020-10-01', 50), ('B', DATE '2020-10-01', 89),
9     ('A', DATE '2020-10-02', 36), ('B', DATE '2020-10-02', 24),
10    ('A', DATE '2020-10-03', 39), ('B', DATE '2020-10-03', 37),
11    ('A', DATE '2020-10-04', 42), ('B', DATE '2020-10-04', 63),
12    ('A', DATE '2020-10-05', 30), ('B', DATE '2020-10-05', 65),
13    ('A', DATE '2020-10-06', 47), ('B', DATE '2020-10-06', 56),
14    ('A', DATE '2020-10-07', 71), ('B', DATE '2020-10-07', 50),
15    ('A', DATE '2020-10-08', 80), ('B', DATE '2020-10-08', 54),
16    ('A', DATE '2020-10-09', 75), ('B', DATE '2020-10-09', 30),
17    ('A', DATE '2020-10-10', 63), ('B', DATE '2020-10-10', 32);
18
19 SELECT *
20 FROM memory.default.stock_price
21 MATCH_RECOGNIZE (
22     PARTITION BY Company
23     ORDER BY Price_date
24     MEASURES
25         FIRST(A.Price) AS start_price,
26         LAST(C.Price)  AS end_price,
27         MATCH_NUMBER() AS match_id
28     PATTERN (A B C)
29     DEFINE
30         B AS B.Price > PREV(B.Price),
31         C AS C.Price > PREV(C.Price)
32 );

```

Listing 6-5: Trino SQL: create and load the stock_price table, then detect the increasing-window pattern with MATCH_RECOGNIZE.

All four implementations return the same three matches (See Table 6-4), covering partitioning, ordering, implicit DEFINE, PREV navigation, MATCH_NUMBER(), and measure materialization. MATCH_NUMBER() counts the matches 1, 2, ... within each partition and restarts at 1 in the next, so company B's only match is 1.

```

1 CREATE TABLE stock_price (
2     Company    VARCHAR2(10),
3     Price_date DATE,
4     Price      NUMBER
5 );
6
7 INSERT ALL
8     INTO stock_price VALUES ('A', DATE '2020-10-01', 50)
9     INTO stock_price VALUES ('B', DATE '2020-10-01', 89)
10    INTO stock_price VALUES ('A', DATE '2020-10-02', 36)
11    INTO stock_price VALUES ('B', DATE '2020-10-02', 24)
12    INTO stock_price VALUES ('A', DATE '2020-10-03', 39)
13    INTO stock_price VALUES ('B', DATE '2020-10-03', 37)
14    INTO stock_price VALUES ('A', DATE '2020-10-04', 42)
15    INTO stock_price VALUES ('B', DATE '2020-10-04', 63)
16    INTO stock_price VALUES ('A', DATE '2020-10-05', 30)
17    INTO stock_price VALUES ('B', DATE '2020-10-05', 65)
18    INTO stock_price VALUES ('A', DATE '2020-10-06', 47)
19    INTO stock_price VALUES ('B', DATE '2020-10-06', 56)
20    INTO stock_price VALUES ('A', DATE '2020-10-07', 71)
21    INTO stock_price VALUES ('B', DATE '2020-10-07', 50)
22    INTO stock_price VALUES ('A', DATE '2020-10-08', 80)
23    INTO stock_price VALUES ('B', DATE '2020-10-08', 54)
24    INTO stock_price VALUES ('A', DATE '2020-10-09', 75)
25    INTO stock_price VALUES ('B', DATE '2020-10-09', 30)
26    INTO stock_price VALUES ('A', DATE '2020-10-10', 63)
27    INTO stock_price VALUES ('B', DATE '2020-10-10', 32)
28 SELECT * FROM dual;
29
30 SELECT *
31 FROM stock_price
32 MATCH_RECOGNIZE (
33     PARTITION BY Company
34     ORDER BY Price_date
35     MEASURES
36         FIRST(A.Price) AS start_price,
37         LAST(C.Price)  AS end_price,
38         MATCH_NUMBER() AS match_id
39     PATTERN (A B C)
40     DEFINE
41         B AS B.Price > PREV(B.Price),
42         C AS C.Price > PREV(C.Price)
43 );

```

Listing 6-6: Oracle SQL: the same clause as Listing 6-5 over an Oracle copy of the stock_price table.

```

1 import pandas as pd
2 from pandas_match_recognize import match_recognize
3 data = {
4     "Company": ["A","B","A","B","A","B","A","B","A","B",
5                "A","B","A","B","A","B","A","B","A","B"],
6     "Price_date": ["2020-10-01","2020-10-01","2020-10-02","2020-10-02",
7                   "2020-10-03","2020-10-03","2020-10-04","2020-10-04",
8                   "2020-10-05","2020-10-05","2020-10-06","2020-10-06",
9                   "2020-10-07","2020-10-07","2020-10-08","2020-10-08",
10                  "2020-10-09","2020-10-09","2020-10-10","2020-10-10"],
11     "Price": [50,89,36,24,39,37,42,63,30,65,47,56,71,50,80,54,75,30,63,32],
12 }
13 df = pd.DataFrame(data)
14 df["Price_date"] = pd.to_datetime(df["Price_date"])
15
16 query = """
17 SELECT *
18 FROM stock_price
19 MATCH_RECOGNIZE (
20     PARTITION BY Company
21     ORDER BY Price_date
22     MEASURES
23         FIRST(A.Price) AS start_price,
24         LAST(C.Price) AS end_price,
25         MATCH_NUMBER() AS match_id
26     PATTERN (A B C)
27     DEFINE
28         B AS B.Price > PREV(B.Price),
29         C AS C.Price > PREV(C.Price)
30 ); """
31 result = match_recognize(query, df)
32 print(result)

```

Listing 6-7: Proposed engine: the same query as a MATCH_RECOGNIZE string over a stock_price DataFrame.

```

1 import pandas as pd
2
3 def detect_two_consecutive_increases(df, company_col="Company",
4                                     date_col="Price_date", price_col="Price"):
5     gdf = df.copy()
6     gdf[date_col] = pd.to_datetime(gdf[date_col])
7     gdf = gdf.sort_values([company_col, date_col]).reset_index(drop=True)
8     out_rows = []
9     for company, g in gdf.groupby(company_col, sort=False):
10         g = g.reset_index(drop=True)
11         inc = g[price_col].diff() > 0
12         start_mask = inc.shift(-1).fillna(False) & inc.shift(-2).fillna(False)
13         candidates = [int(i) for i, ok in enumerate(start_mask)
14                       if ok and i + 2 < len(g)]
15         match_id, next_start_pos = 0, 0
16         for i in candidates:
17             if i < next_start_pos:
18                 continue
19             match_id += 1
20             out_rows.append({
21                 "company": company,
22                 "start_price": g.loc[i, price_col],
23                 "end_price": g.loc[i + 2, price_col],
24                 "match_id": match_id,
25             })
26         next_start_pos = i + 3 # AFTER MATCH SKIP PAST LAST ROW
27     return pd.DataFrame(out_rows)
28
29 result = detect_two_consecutive_increases(df)
30 print(result)

```

Listing 6-8: Hand-written Pandas function for the same increasing-window detection.

Table 6-4: Stock-price result: all four systems return these three identical matches

Match	company	start_price	end_price	match_id
1	A	36	42	1
2	A	30	71	2
3	B	24	63	1

Source: [5], [63]

6.3.3 Example 3: V-Shape Recovery on the stock_price Dataset

The third example detects a *V-shape recovery* on the same table: a strictly falling run followed by a strictly rising one. Unlike the fixed three-row patterns above, PATTERN (A B+ C+) uses the + quantifier twice, with the fall and rise defined through PREV. The measures report the start, bottom, and recovered prices. As before, the same detection is expressed as a Trino SQL query (Listing 6-9), as an Oracle SQL query (Listing 6-10), as a call to the proposed engine (Listing 6-11), and as a hand-written Pandas function (Listing 6-12). The two SQL listings again carry the same clause [63]. All four return the same matches (See Table 6-5).

```

1 SELECT *
2 FROM memory.default.stock_price
3 MATCH_RECOGNIZE (
4   PARTITION BY Company
5   ORDER BY Price_date
6   MEASURES
7     FIRST(A.Price) AS start_price,
8     LAST(B.Price) AS bottom_price,
9     LAST(C.Price) AS end_price,
10    MATCH_NUMBER() AS match_id
11   ONE ROW PER MATCH
12   PATTERN (A B+ C+)
13   DEFINE
14     B AS B.Price < PREV(B.Price),
15     C AS C.Price > PREV(C.Price)
16 );

```

Listing 6-9: Trino SQL: detect the V-shape recovery on the stock_price table (already created and loaded in Listing 6-5).

For the *correctness* criterion, all four implementations return the same three matches (See Table 6-5). Beyond the clauses of the earlier examples, this query additionally exercises the + quantifier (one-or-more) on two pattern variables, confirming that the engine's greedy consecutive-quantifier handling agrees with the reference engines.

```

1 SELECT *
2 FROM stock_price
3 MATCH_RECOGNIZE (
4   PARTITION BY Company
5   ORDER BY Price_date
6   MEASURES
7     FIRST(A.Price) AS start_price,
8     LAST(B.Price) AS bottom_price,
9     LAST(C.Price) AS end_price,
10    MATCH_NUMBER() AS match_id
11 ONE ROW PER MATCH
12 PATTERN (A B+ C+)
13 DEFINE
14   B AS B.Price < PREV(B.Price),
15   C AS C.Price > PREV(C.Price)
16 );

```

Listing 6-10: Oracle SQL: the same clause as Listing 6-9 on the `stock_price` table created in Listing 6-6.

```

1 from pandas_match_recognize import match_recognize
2
3 query = """
4 SELECT *
5 FROM stock_price
6 MATCH_RECOGNIZE (
7   PARTITION BY Company
8   ORDER BY Price_date
9   MEASURES
10    FIRST(A.Price) AS start_price,
11    LAST(B.Price) AS bottom_price,
12    LAST(C.Price) AS end_price,
13    MATCH_NUMBER() AS match_id
14 ONE ROW PER MATCH
15 PATTERN (A B+ C+)
16 DEFINE
17   B AS B.Price < PREV(B.Price),
18   C AS C.Price > PREV(C.Price)
19 ); """
20 result = match_recognize(query, df)
21 print(result)

```

Listing 6-11: Proposed engine: the same V-shape query over the `stock_price` DataFrame `df` of Listing 6-7.

```

1 import pandas as pd
2
3 def detect_v_shapes(df, key="Company", ts="Price_date", val="Price"):
4     g = df.copy()
5     g[ts] = pd.to_datetime(g[ts])
6     g = g.sort_values([key, ts]).reset_index(drop=True)
7     out = []
8     for company, grp in g.groupby(key, sort=False):
9         v = grp[val].to_numpy()
10        n, i, match_id = len(v), 0, 0
11        while i < n - 2:
12            j = i
13            while j + 1 < n and v[j + 1] < v[j]: # B+: strictly falling
14                j += 1
15            if j == i: # need at least one fall
16                i += 1
17                continue
18            k = j
19            while k + 1 < n and v[k + 1] > v[k]: # C+: strictly rising
20                k += 1
21            if k == j: # need at least one rise
22                i += 1
23                continue
24            match_id += 1
25            out.append({
26                "company": company,
27                "start_price": int(v[i]),
28                "bottom_price": int(v[j]),
29                "end_price": int(v[k]),
30                "match_id": match_id,
31            })
32            i = k + 1 # AFTER MATCH SKIP PAST LAST ROW
33        return pd.DataFrame(out)
34
35 result = detect_v_shapes(df)
36 print(result)

```

Listing 6-12: Hand-written Pandas function for the same V-shape detection.

Table 6-5: V-shape result: all four systems return these three identical matches

Match	company	start_price	bottom_price	end_price
1	A	50	36	42
2	B	89	24	65
3	B	56	50	54

Source: [5], [63]

6.3.4 Usability and Developer Effort

Each worked example expresses the same detection three ways: SQL query, engine call, and hand-written Pandas function. The detection logic is counted on both sides in non-blank, non-comment lines of code (LOC): the MATCH_RECOGNIZE statement against the Pandas detection function (See Table 6-6). Data setup, the imports, and the printing are shared, so they are left out of both. The declarative form takes 14 vs 23, 14 vs 25, and 16 vs 31 lines, and the gap widens with pattern complexity.

Table 6-6: Lines-of-code comparison for the three worked examples.

Example	Implementation	Notes	LOC
Dip (orders)	MATCH_RECOGNIZE statement	single declarative query	14
Dip (orders)	Hand-written Pandas function	loop / index / skip bookkeeping	23
Stock (stock_price)	MATCH_RECOGNIZE statement	single declarative query	14
Stock (stock_price)	Hand-written Pandas function	groupby/shift/index bookkeeping	25
V-shape (stock_price)	MATCH_RECOGNIZE statement	quantified declarative query	16
V-shape (stock_price)	Hand-written Pandas function	nested falling/rising run scan	31

The declarative form states *what* to find in one statement and lets the engine decide *how*. Each Pandas function hand-codes grouping, sorting, and skip logic, where one off-by-one silently changes the result. That is why the procedural baselines were more error-prone [6]. Against an external SQL engine the value is *where* it runs: in-process on the DataFrame, with no CREATE TABLE/INSERT round-trip and a footprint below 0.6 GB versus 2.0 GB (Oracle) and 14.6 GB (Trino). The trade-off is scope: the supported subset only, without a database query planner or distributed execution. This is a code-level comparison, not a user study (Chapter 7).

6.4 Coverage and Complexity Results

This section reports feature and pattern coverage, then analyzes theoretical complexity. Usability was covered with the worked examples.

6.4.1 SQL Feature Coverage

Two dimensions are reported: implemented clauses (See Table 6-7) and detected pattern types (See Table 6-8), both checked against the Trino-aligned reference tests. *Partially supported* means the common forms are implemented and verified; rarer variants are outside the validated scope.

Table 6-7: Support status of *MATCH_RECOGNIZE* features in the proposed engine

Feature	Status
PARTITION BY	supported
ORDER BY	supported
Quantifiers * + ? {i,j}	supported
Alternation (A B)	supported
Grouping / nesting	supported
Exclusion {- ...-}	supported
Anchors ^, \$	supported
PERMUTE	supported
DEFINE predicates	supported
Navigation PREV/NEXT/FIRST/LAST	supported
MEASURES core aggregates	supported
RUNNING/FINAL semantics	supported
SUBSET	supported
ONE ROW PER MATCH	supported
ALL ROWS PER MATCH	supported
AFTER MATCH SKIP modes	supported
MATCH_NUMBER(), CLASSIFIER()	supported

In summary, the engine supports the evaluated R010-style subset: partitioning; ordering; the listed PATTERN constructs; DEFINE predicates; SUBSET union variables; MEASURES with supported navigation functions and aggregates; RUNNING and FINAL semantics; both output modes; the evaluated AFTER MATCH SKIP modes; and MATCH_NUMBER()/CLASSIFIER().

Inside DEFINE and MEASURES it also evaluates arithmetic, comparisons, IN lists, NULL checks, and common scalar and string functions: LOWER lowercases a string, CAST converts a value to another type, COALESCE returns its first non-null argument, CASE picks a value by condition, and || joins two strings. Constant scalar subqueries without a FROM clause are supported inside tested expressions. Table-reading and correlated subqueries are not. Conditional and subquery ordering keys, general relational planning, and distributed execution are also outside the evaluated R010 path. The separate R020 window path is not evaluated in this chapter. These limits define the scope of Chapters 4 and 5.

At the pattern level, the engine detects ten row-pattern types, each listed with a regex example and a simple use case (See Table 6-8). A type counts as covered when at least one test case exercises it and produces output identical to the reference systems [21]. These ten are shapes the engine can express. The benchmark of Section 6.5.2 times the first five of them, and its 30 is those five run at six sizes. The table can also be read in reverse, from a use case to the pattern that expresses it, and Section 6.3 writes the same detection both as a Pandas function and as a query.

Table 6-8: Pattern-type coverage, with a regex example and a simple use case for each type

Pattern type	Pattern (regex)	Simple use case	Covered
Simple sequence	A+ B+	a run of one category then a run of another	yes
Alternation	A (B C)+ D	a fixed start, mixed intermediate states, a fixed end	yes
Bounded quantifier	A{1,5} B* C+	one to five of one event, optional filler, then one or more of another	yes
Optional parts	A+ B? C*	a required run with an optional middle and tail	yes
Grouping / nesting	(A B)+ (C{1,3} D*)+	repeated groups nested in a larger repeated pattern	yes
Rising / falling trend	A B+	values rising (or falling) over several consecutive rows	yes
Recovery (V-shape)	A B+ C+	a fall followed by a rise, a dip that recovers	yes
Exclusion	{- A -} B+	detect a context but omit some of its rows from the result	yes
Anchors	^A, A\$	the match must be at the partition start or end	yes
Permutation	PERMUTE(A, B, C)	the same set of events in any order	yes

6.4.2 Complexity Analysis

Complexity is analyzed twice: theoretically here, empirically in Section 6.5. Theory predicts scaling; experiments measure the implementation on a specific machine and dataset. The two views are set side by side (See Table 6-9).

Table 6-9: Theoretical and empirical views of complexity analysis

Theoretical Analysis	Empirical Analysis
Mathematical analysis.	Experimental measurement.
Independent of hardware and software environment.	Depends on the hardware, operating system, Python runtime, Pandas, and memory behavior.
Describes how the running time grows as the input grows, using asymptotic notation: Big-O for an upper bound, Big- Ω for a lower bound, and Big- Θ when both hold and the growth rate is therefore exact.	Uses actual execution time, throughput, and memory usage.
Predicts scalability as input size grows.	Measures real-world performance for the evaluated benchmark.

Theoretical analysis. Costs are stated in asymptotic notation, which describes how a cost grows with the input while ignoring constant factors and machine speed. $O(f(n))$ is an upper bound: the cost grows no faster than $f(n)$. $\Omega(f(n))$ is a lower bound: it grows at least that fast. $\Theta(f(n))$ is both at once, so it pins the growth rate exactly. This chapter uses Θ where the cost is known on both sides and O where only the ceiling is claimed. Five symbols recur: n input rows, m pattern symbols (a bounded quantifier $\{i, j\}$ counts as j copies of its fragment, since the builder clones the fragment per repetition), d DEFINE variables, k matches, ℓ average rows per match that are materialized in the output. Thus $\ell = 1$ for ONE ROW PER MATCH; it can be larger for ALL ROWS PER MATCH.

Running a query does five kinds of work. Parsing reads the query and extracts its MATCH_RECOGNIZE clauses. Automaton construction builds the pattern's automaton, NFA then DFA. Ordering sorts the rows. Condition checking tests the DEFINE conditions on every row. Matching scans for matches and builds the output.

Parsing and automaton construction depend on the pattern, not the data: NFA construction is $O(m)$, and the NFA-to-DFA conversion is $O(2^m)$ states in the worst case but a handful in practice. For a fixed pattern this is a one-time constant. *Ordering* is the only data-dependent cost above linear for the fixed row-local benchmark path. A full comparison sort costs $O(n \log n)$, done per partition. More precisely it is $\sum_h O(n_h \log n_h)$ over partitions of n_h rows, which the benchmark's single partition bounds by $O(n \log n)$. One partition is the worst case for this term, not a convenience: the same rows split into p groups would cost only $O(n \log(n/p))$. But the engine sorts only when needed. It first checks in $O(n)$ whether the rows are already ordered and, if so, skips the sort. *Condition checking* prepares one Boolean result per row and variable on the compiled row-local path, costing $O(nd)$. *Matching and output* then costs $O(n + k\ell)$ on the compiled default-skip path: one scan over the n rows, plus the ℓ rows materialized for each of the k matches. These bounds describe the path used by the main benchmark. They are not a bound for every supported expression and skip mode.

Combining the data-dependent work, the cost has *two independent axes*, and it is important to keep them apart. One is growth with the data size n for a fixed pattern. The other is growth with the pattern complexity for fixed data.

Axis 1: growth with the input size n , for a fixed compiled row-local pattern. This is the axis the benchmark scales. The automaton is fixed, and every data-dependent stage except ordering is linear, so only whether a sort is needed separates the two cases:

- **Best case** (input already ordered, as in the benchmark): the $O(n)$ check skips the sort, leaving $\Theta(nd + n + k\ell) = \Theta(n)$.
- **Worst case** (input unordered, a real sort required): $O(n \log n + nd + n + k\ell)$, which reduces to $O(n \log n)$.

Both reductions rest on the same two facts. d is fixed because it counts DEFINE variables, a property of the query rather than of the table. And $k\ell = O(n)$ because the default skip mode leaves matches non-overlapping, so the output cannot hold more rows than the input. These two bracket the engine's realistic scaling. The ordered and shuffled scenarios of Section 6.5.6 measure them directly. One reading note: across the measured 100 K–2.22 M range, $\log_2 n$ grows only from about 17 to 21. The $O(n \log n)$ worst case therefore looks almost straight in the plots: the two cases separate by a near-constant factor, not by a visible bend. Extra memory on this compiled path is $O(nd + n + k\ell)$, which reduces to $O(n)$ for the fixed benchmark and its linear-size output. Other modes can materialise more output or retain more search state.

The two cases are drawn side by side (See Figure 6-2).

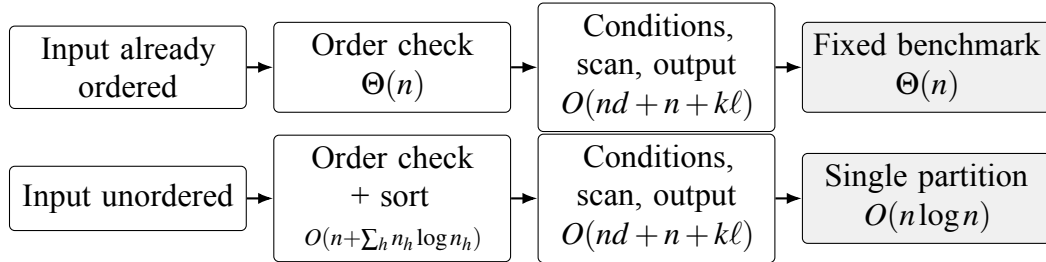


Figure 6-2: Data-size complexity for a fixed compiled row-local pattern.

For this fixed compiled path, ordering determines which branch applies.

Axis 2: growth with pattern complexity, for fixed data. Three constructs are expensive in the pattern size m rather than in the number of rows n : the NFA-to-DFA conversion, PERMUTE, and a DEFINE predicate that reads the match built so far. The distinction matters because m is what the analyst types, a handful of symbols, while n is the table, here up to 227.9 million rows. Each is either paid once or held in check by an explicit guard, so none appears as a growth curve in the data size.

The NFA-to-DFA conversion is $O(2^m)$ states in the worst case. But it is a one-time, cached compile step, capped at a state limit the engine derives from its own memory budget at roughly 64 KiB per state, with a conservative default ceiling of 50,000 states. The memory term is what tightens on a small machine; the ceiling is what holds on a

large one, and on the evaluation host it is the ceiling that binds. Reaching this limit raises an explicit compilation error, and the engine does not execute or cache a partial DFA. None of the evaluated patterns came close.

A PERMUTE of m variables expands to $O(m!)$ branches. Eight variables is the last size that fits under the estimated fifty-thousand-branch ceiling, so larger ones are rejected. A DEFINE predicate that reads the match built so far cannot use the row-local Boolean matrix. Ambiguous quantified choices may then require the exact preference-ordered search, whose worst case is exponential. Non-default skip modes are different: they disable some fused enumeration and output paths, but do not by themselves make a predicate state-dependent or force exact search.

The exact search has a partition-level search budget (Section 5.2.1) and uses an explicit stack. It therefore avoids Python’s recursion limit. If the search budget is exhausted, the query raises an error instead of returning a partial result. The main benchmark and the ordering scenarios stay on the compiled row-local path and exercise Axis 1. The state-dependent test in Section 6.5.6 uses a compiled incremental exact-search plan. It measures one important form, a running aggregate over the match so far, not the worst case of every state-dependent expression.

6.5 Empirical and Cross-System Analysis

This section compares the engine with Trino 473 and Oracle 21c EE on the Amazon UK Products dataset: five pattern families, six sizes (100 K to 2.22 M rows), 30 cells per system. Each cell is the IQR-filtered mean of twenty runs after five warmups: the twenty times are averaged after the $1.5 \times$ IQR rule of Section 6.1.1 drops the runs that fall far outside the middle half. All 90 cells completed. The engine averaged about 6.1 M rows/s. Its per-family time fits give $R^2 = 0.9904\text{--}0.9997$, so growth is close to linear over these six sizes. Complex nested patterns cost the most. Section 6.5.1 defines the patterns and the resource policy, Section 6.5.2 the results, Sections 6.5.3 and 6.5.4 throughput and memory, Section 6.5.5 the summary, and Sections 6.5.6–6.5.9 sensitivity, scalability, validity, and the key findings.

Positioning. The engine does not aim to replace Oracle, Trino, or streaming platforms. It brings a practical subset of row-pattern matching into in-memory Python and skips the load-into-a-server step. The target is bounded data that already sits in memory, not an unbounded stream. That choice shapes the design. The engine groups the frame and sorts each partition before it runs the automaton, so a match needs every row of its partition present. A streaming engine instead keeps incremental state per open match, and it must handle events that arrive late or out of order. In return it supports only the subset of Chapters 4–5, while the databases have mature optimizers, distributed execution, and broader SQL coverage. Against hand-written Pandas, though, the SQL form states partitioning, ordering, pattern, skip, and measures in one query.

6.5.1 Pattern Definitions

The five pattern families used throughout the benchmark are listed with the detection goal of each (See Table 6-10). They are the first five pattern types of the coverage table (See Table 6-8). Their cost rises from a single sequence to nested groups while the category column stays the same, so the six sizes measure growth rather than a change of

data. The other five types are covered by the correctness tests of Section 6.4.1, not by the timing matrix.

Table 6-10: SQL Pattern Definitions and Detection Goals

Pattern Name	SQL Pattern	Description & Goal
simple_sequence	A+ B+	Goal: Detect transitions from budget to low-mid price listings. Use: Identify runs where budget products (price $\leq$ £10) are immediately followed by low-mid products (£10–25) in the listing order, useful for locating price-band boundaries and clustered cheap listings.
alternation	A (B C)+ D	Goal: Detect budget-to-upper-mid transitions through mixed middle bands. Use: Find sequences that start with a budget product, continue with a mix of low-mid and mid products (£10–50), and end with an upper-mid product (£50–100). Exercises alternation inside a quantified group.
quantified	A{1,5} B* C+	Goal: Detect bounded budget runs followed by mid-band products. Use: Find sequences with 1–5 budget products, optionally followed by low-mid products, then one or more mid products (£25–50). Exercises bounded, optional, and unbounded quantifiers in one pattern.
optional_pattern	A+ B? C*	Goal: Flexible budget-run detection with optional band transitions. Use: Broad matching of budget runs with an optional low-mid product and optional mid products. High recall; produces the largest match sets in the benchmark.
complex_nested	(A B)+ (C{1,3} D*)+	Goal: Nested grouping of low-price runs followed by mid/upper-mid groups. Use: Detect runs of low-priced products (price $\leq$ £25) followed by one or more groups of 1–3 mid products with optional upper-mid products. Exercises nested alternation combined with bounded repetition; the most expensive pattern family.

All patterns run on a derived category column from the product price: A = budget ($\leq$ £10), B = low-mid ($>$ £10 and $\leq$ £25), C = mid ($>$ £25 and $\leq$ £50), D = upper-mid ($>$ £50 and $\leq$ £100), and E = premium ($>$ £100). Each row carries a sequential `seq_id` in source order, and the six sizes (100 K to 2,222,742 rows) are nested prefixes of the same source. Queries use `ORDER BY seq_id` and no `PARTITION BY`, so each dataset is

one logical partition. Multiple partitions are exercised elsewhere: the worked examples of Section 6.3 partition by customer and by company, and partitioned queries appear throughout the correctness suite [21]. One partition is kept here because it is the worst case for the sort term (Section 6.4.2) and because it leaves pattern and size as the only variables across the 90 cells. The sequence is a controlled listing order, not a temporal one; what matters is that all three systems process the identical rows.

Benchmark setup and resource policy. The benchmark is 90 cells (three systems, five families, six sizes; 1,800 measured executions) on the same CSVs and query bodies.

The proposed-engine values describe the fixed benchmark revision recorded in the appendix (See Appendix A.1). The Oracle and Trino values were retained from an earlier campaign under the same one-core, outer 32 GB, five-warmup, twenty-run protocol. The systems ran sequentially, not at the same time. The engine ran in its own control group; Trino and Oracle ran in limited containers that were restarted before each size. The same control-group counter measured all three memory footprints. Reported times are client-observed wall-clock values and include result materialization, but exclude data loading. The database client’s memory is outside the measured containers, which is a limitation of the memory comparison. Every system returned the same normalized rows (See Table A-1).

The engine’s throughput is stable across the range (a scan-dominated path) and it records the lowest time on all 30 cells. Memory uses the two unified metrics of Section 6.5.4: incremental peak (per-query working memory) and absolute peak (footprint).

6.5.2 Cross-System Comparison Under a Common Outer Resource Limit

The three systems run the same query bodies; they differ only in the source table. The benchmark protocol and the evidence used to reproduce the comparison are recorded in the appendix (See Appendix A.1). Match counts per cell are identical across the three systems, all of which use `ONE ROW PER MATCH` (See Table A-1). `optional_pattern` yields the most matches and `alternation` the fewest. Match volume feeds the output cost, but pattern structure drives most of the per-pattern variation below: `complex_nested` is the slowest family despite fewer matches than `simple_sequence`.

Each family is averaged over the six sizes (See Figure 6-3): throughput on the left (higher is better), time on the right (lower is better). The engine leads both measures on all five families. The widest gap is `complex_nested`, where Trino averages 5.28 s against the engine’s 0.34 s, a factor of 15.4. The values are listed in the appendix (See Table A-2).

The same data are grouped by size (See Figure 6-4). The engine is fastest at every size, holding 5.0–6.9 M rows/s as input grows. Per size it is about 9.3–13.3× faster than Trino and 4.3–7.0× faster than Oracle. The values are listed in the appendix (See Table A-3).

Each pattern has its own time-versus-size panel (See Figure 6-5). The engine (blue) is the lowest curve everywhere. The margin widens most on `complex_nested`, where the databases bend upward past 400 K rows while the engine stays near-linear (See Table A-4).

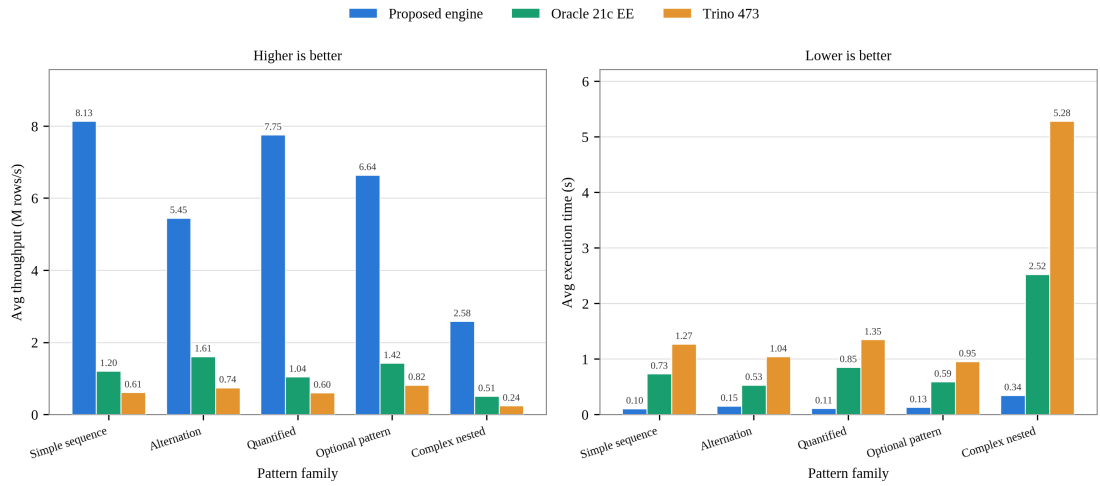


Figure 6-3: Performance summary by pattern family.

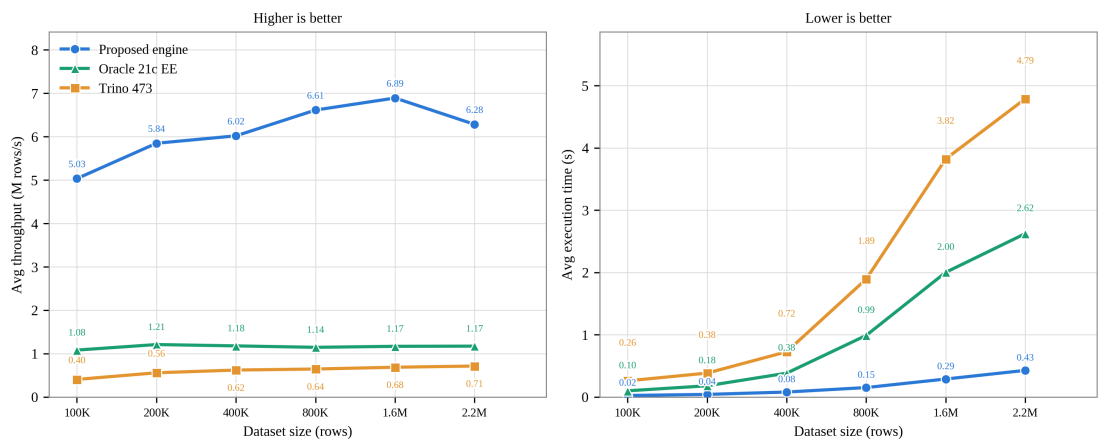


Figure 6-4: Performance summary by dataset size.

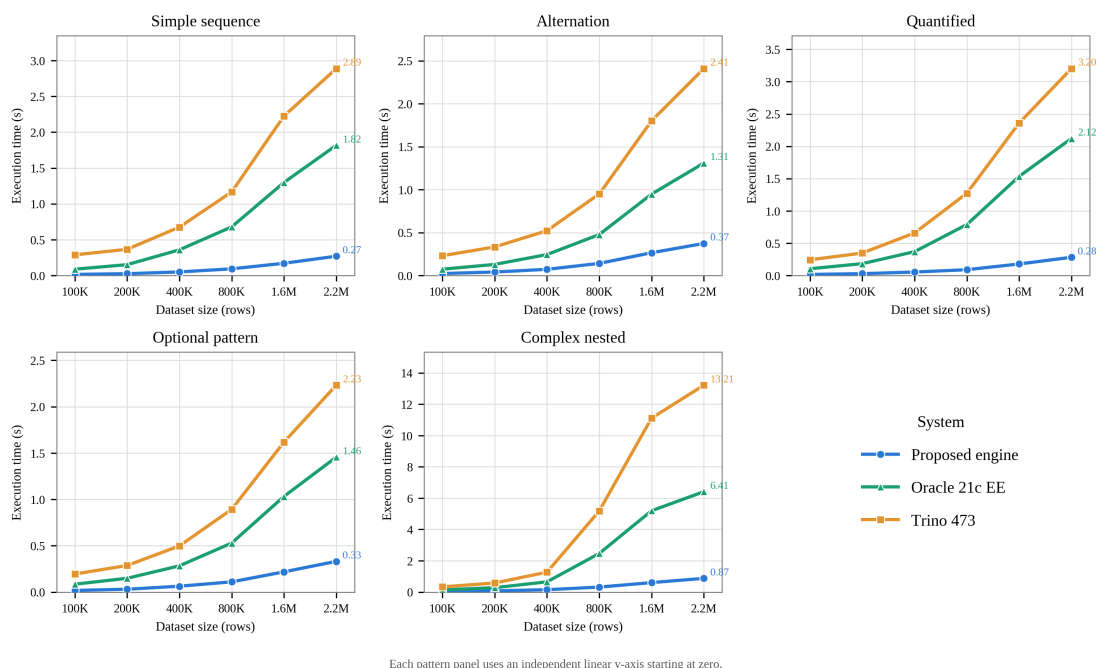


Figure 6-5: Execution time by pattern, size, and system.

6.5.3 Throughput by Pattern, Size, and System

Throughput is plotted with one panel per pattern; higher is better (See Figure 6-6). Throughput is computed for each measured run, as dataset size divided by that run's time, then filtered by the same $1.5 \times$ IQR rule and averaged. The engine is the highest curve in every panel (See Table A-5).

6.5.4 Memory Metrics by Pattern, Size, and System

For the main benchmark, both memory metrics use the same `cgroup v2 memory.current` counter. It measures the dedicated engine control group or the complete Trino or Oracle container every 50 ms. The boundaries are therefore operational rather than identical: the engine value covers its Python process, while each database value covers its server container and excludes the external client. Shorter allocation spikes can also fall between samples.

The scripts divide byte counters by 1024^2 and 1024^3 . For compact labels, MB and GB in the memory results therefore mean the binary units MiB and GiB.

Each measured run has a five-second idle window. Its average defines the *baseline*. The *absolute peak* is the largest query-time sample, and the *incremental peak* is that sample minus the baseline. The subtraction removes most fixed resident memory, such as the already-loaded DataFrame, Oracle System Global Area (SGA), and Trino JVM heap. It does not turn the values into an exact algorithm-only measurement. As with time, each reported memory value is the IQR-filtered mean over twenty measured runs.

Query memory (See Figure 6-7; Table A-6 in Appendix A) reports the incremental peak. *Footprint memory* (See Figure 6-8; Table A-7) reports the absolute peak. The

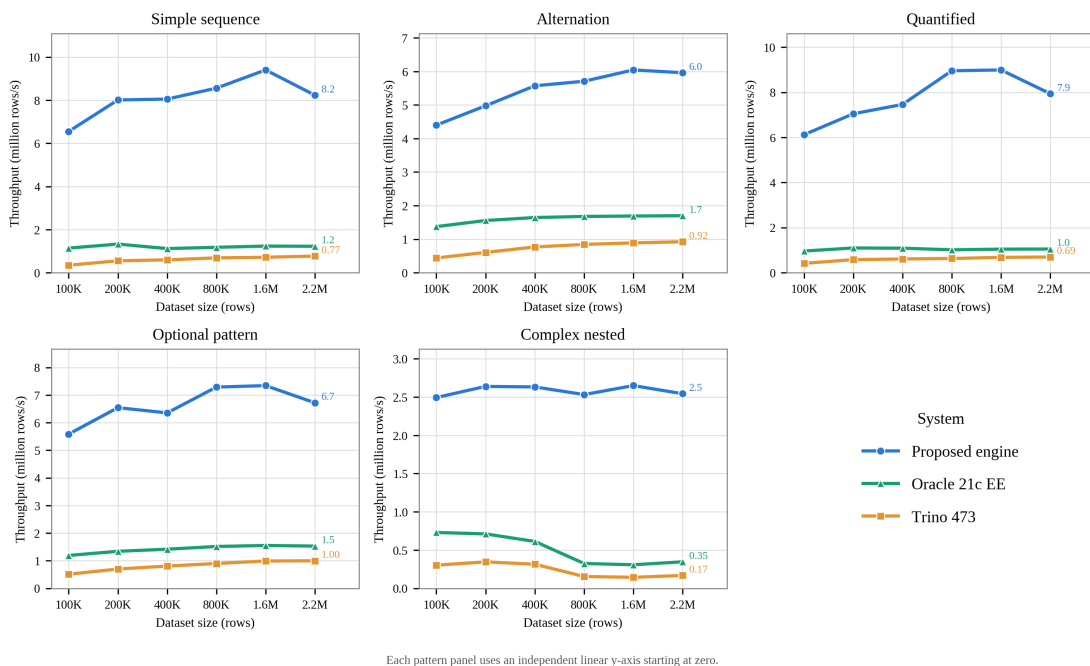


Figure 6-6: Throughput by pattern, size, and system.

footprint documents the practical deployment cost of the resident engine or container, and is *not* a per-query or algorithm-level metric.

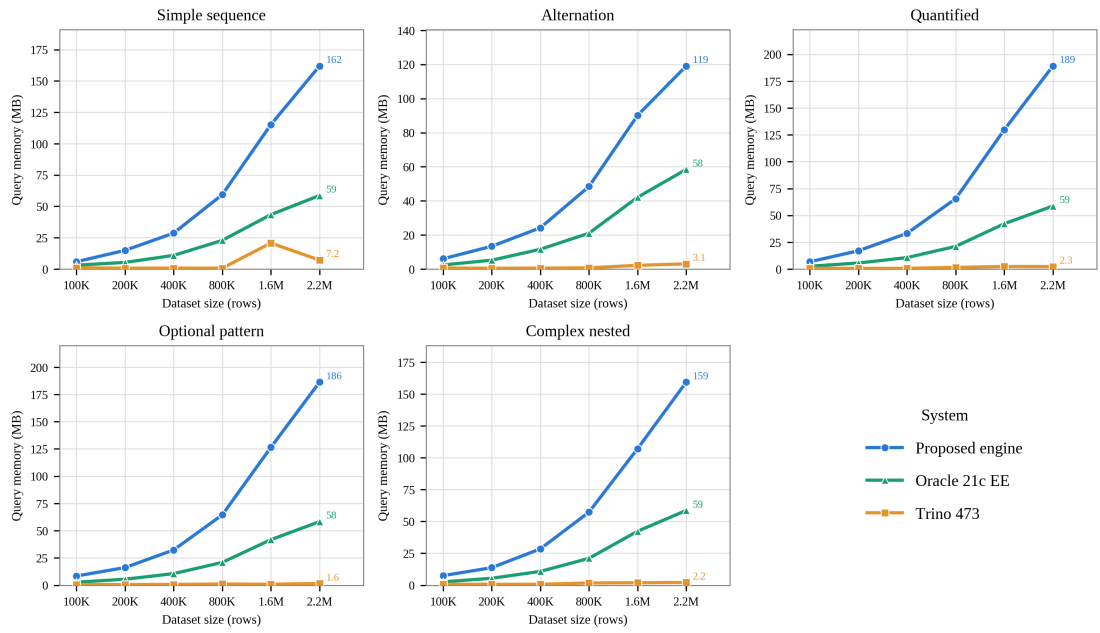
Two supporting values are recorded per cell in the benchmark evidence. The first is a *memory-per-million-rows* normalization: the incremental peak divided by the input size in millions, meaningful mainly at the larger sizes. The second is each database's own per-query counter, Trino's `peakMemoryBytes` and the high-water delta of Oracle's session Program Global Area (PGA), which serve as cross-checks below.

The engine's and Oracle's peaks rise roughly with size: the engine 5.9 MB to 189 MB, and Oracle 2.4 MB to 58.7 MB. The engine's incremental peak is the largest of the three (mean 64.5 MB versus Oracle's 23.6 MB, about $2.7\times$). This is consistent with the compiled path creating whole-column masks and intermediate arrays.

One caveat for Trino is that the incremental cgroup measure observes growth above an already-resident JVM heap. It stays below 2.5 MB in 27 of the 30 cells, with a largest value of 20.8 MB. Trino's native `peakMemoryBytes` counter ranges from 1.3 to 41.5 MB. The two counters have different scopes, so neither should be substituted for the common footprint comparison.

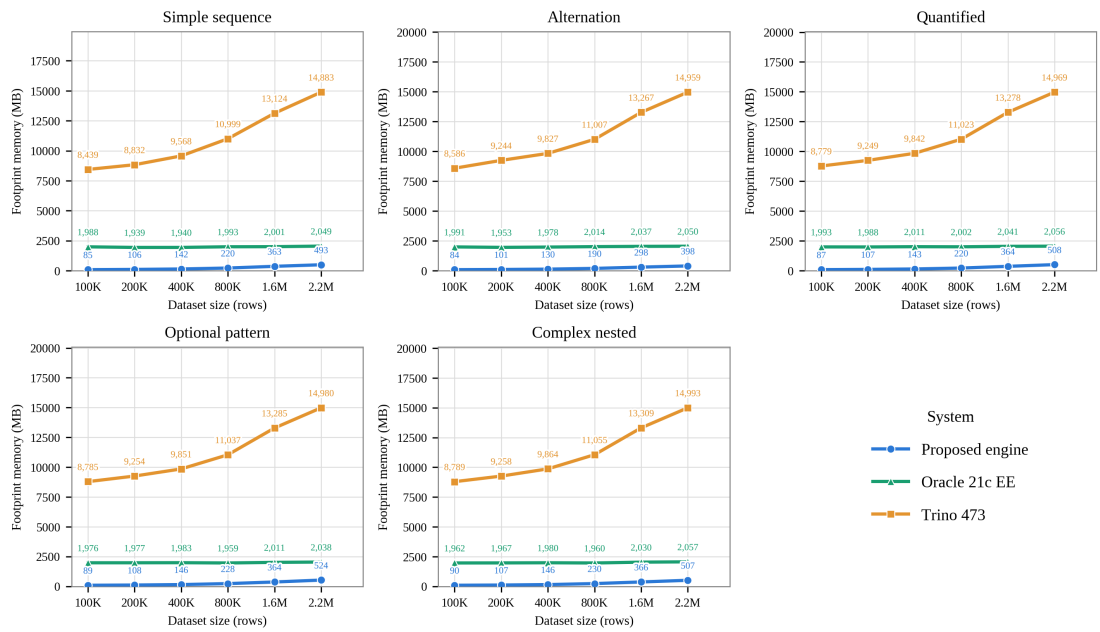
These deployment footprints have different boundaries. The engine is one Python process and peaks at 0.51 GB. The database values cover whole containers: Trino uses 8.2–14.6 GB because the table resides in its JVM heap, while Oracle stays near 2.0 GB. Per-size restarts make the measurements independent across sizes. They show deployment cost; per-query comparisons use the incremental peak instead (See Table A-6).

Chapter 6: Experimental Evaluation



Each pattern panel uses an independent linear y-axis starting at zero.

Figure 6-7: Query memory (incremental peak) by pattern, size, and system.



Each pattern panel uses an independent linear y-axis starting at zero.

Figure 6-8: Footprint memory (absolute peak) by pattern, size, and system.

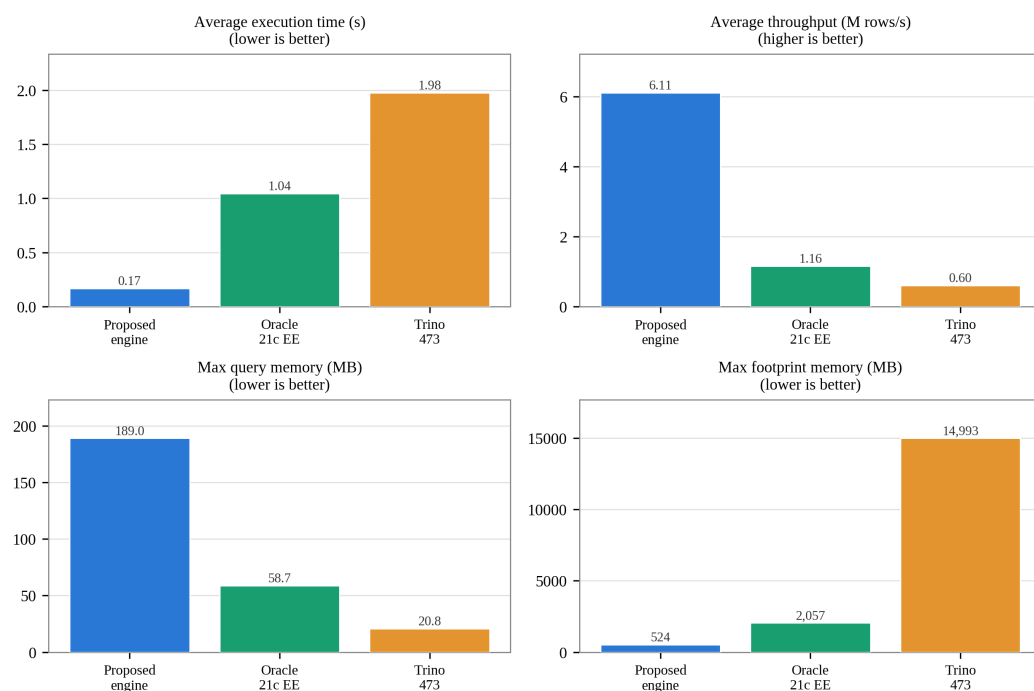


Figure 6-9: Overall benchmark statistics by system.

6.5.5 Cross-System Summary

The overall statistics have four panels, each with one bar per system (See Figure 6-9). *Average execution time* is the mean of the 30 cell means: 0.17 s for the engine, against 1.04 s for Oracle and 1.98 s for Trino. The engine runs inside the process that already holds the data, so it avoids the planning, storage, and client-transfer layers the databases still pay for. It also compiles supported conditions to column operations. *Average throughput* is the same 30 cells read as rates: 6.11 M rows/s, against 1.16 and 0.60. The rate does not fall as data grows, which is what near-linear time means, and the spread from 2.49 to 9.40 M rows/s tracks pattern structure rather than size.

Max query memory is the largest rise above the idle baseline. The engine is highest at 189 MB. The compiled path is vectorized: it builds whole-column masks and intermediate arrays over all input rows instead of walking row by row. The cost therefore follows the input, not the result. It stays between 56 and 90 bytes per input row in all 30 cells, and the 189 MB cell returns 95,650 rows while a 186 MB cell returns 276,776. This is the memory side of the design that gives the speed, so the higher figure is expected rather than a fault, and even its peak stays far below the 32 GB limit. Trino's 20.8 MB is growth above an already-resident heap, not a like-for-like figure. *Max footprint memory* is the largest total the process or container reached. Here the order reverses: 524 MB for the engine, 2,057 MB for Oracle, and 14,993 MB for Trino. The boundaries differ: one Python process against a complete database container, and Trino is the largest because the table is held in its JVM heap. It is a deployment cost, not a per-query one. The engine leads on time, throughput, and total footprint, and trades that for the highest per-query memory (See Table A-8).

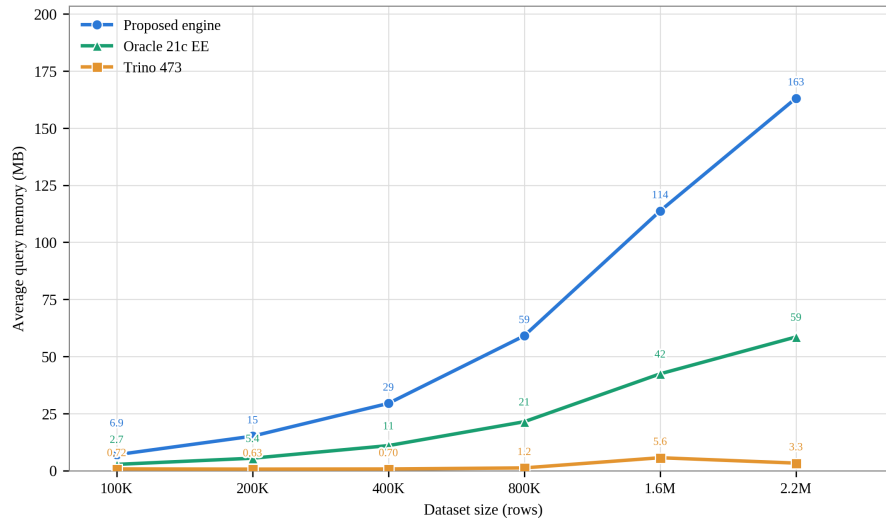


Figure 6-10: Average query memory by dataset size.

Averaged over the five patterns (See Figure 6-10), the engine's per-query memory rises with size. It stays the highest of the three (See Table A-9).

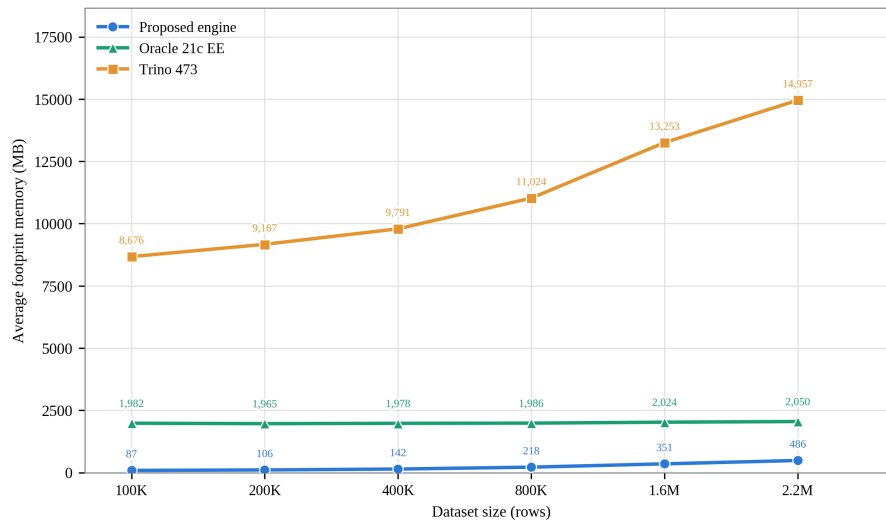


Figure 6-11: Average footprint memory by dataset size.

The footprint picture inverts (See Figure 6-11). The engine's single process stays well below the Trino and Oracle containers at every size (See Table A-10).

All 90 system cells completed, and all 30 pattern-size combinations agreed across the three systems. The engine had the lowest mean time (0.17 s, against 1.04 s for Oracle and 1.98 s for Trino). It also had the highest incremental query peak but the lowest total footprint. Run-to-run variation remained low after warmup and IQR filtering. The detailed values are reported once, in the appendix (See Appendix A). These results

describe the selected workload and resource configuration; they do not establish universal superiority.

The two databases are set against the engine, which is drawn as a $1\times$ reference (See Figure 6-12).

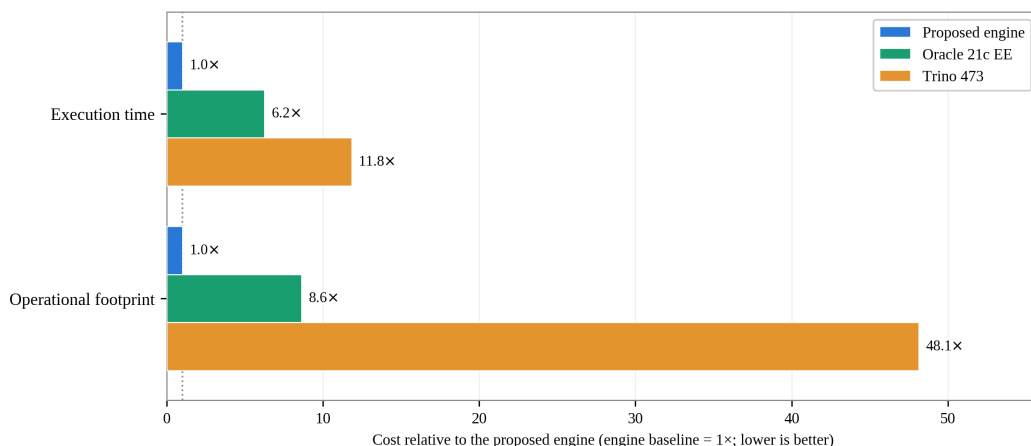


Figure 6-12: Cost of each database relative to the proposed engine (engine = $1\times$).

Each bar is a database’s average time or footprint as a multiple of the engine’s (blue $1\times$ line; lower is better). Oracle needs $6.2\times$ the time and $8.6\times$ the footprint; Trino $11.8\times$ and $48.1\times$. Throughput mirrors the time result. The engine’s one trade-off is higher incremental query memory, because the compiled path creates whole-column working arrays (about $2.7\times$ Oracle’s mean incremental peak). It is shown per cell (See Figure 6-7), and signed percentages are listed in the appendix (See Table A-12).

6.5.6 Sensitivity and Boundary Analysis

The main matrix uses ordered input and row-local predicates. Two sensitivity cases remove these advantages: shuffled input and a state-dependent predicate. Both are measured across all three systems.

Sensitivity case 1: shuffled input. The benchmark orders by an already-monotonic `seq_id`, so the engine’s $O(n)$ check skips its sort. The matrix was rerun with every input deterministically shuffled, which makes the engine enter its $O(n \log n)$ sorting branch. The query still restores the same logical order. All 90 system cells completed, and all 30 pattern–size combinations still agreed across systems. Scenario A is ordered and Scenario B is shuffled. The difference measures sensitivity to physical input order; it is not a direct profile of the sort operator inside each database.

The proposed-engine values describe the benchmark revision recorded in the appendix (See Appendix A.1). The Oracle and Trino values were retained from the earlier unordered campaign under the same one-core, 32 GB, five-warmup, twenty-run protocol. As in the main benchmark, the systems ran sequentially. The result is shown by system (See Figure 6-13).

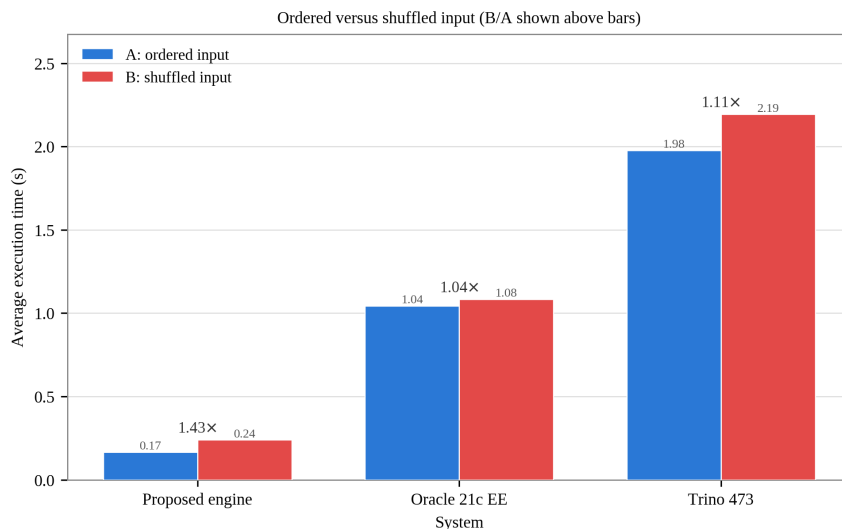


Figure 6-13: Average execution time on ordered vs shuffled input, by system.

Each pair is a system’s ordered (A) and shuffled (B) average; the number above is the B/A slowdown. Scenario A is the main matrix and Scenario B repeats it with every input shuffled. Each A/B comparison uses the same logical rows and query, but the changed physical layout can also affect caching and compression. The engine shows the largest slowdown (1.43×): 1.70× for `simple_sequence`, 1.69× for `quantified`, and 1.19× for `complex_nested`. Oracle and Trino change by 1.04–1.11×. Their internal plans were not profiled, so this smaller change is reported without assuming that they used the same sort plan in both scenarios. Most important, the ranking holds: at 2.22 M rows the engine is still about 4.5× faster than Oracle and 8.8× than Trino under shuffled input (against 6.2× and 11.2× ordered). So the advantage is not an artifact of the ordered shortcut: it survives when the engine must sort (See Tables A-13–A-14).

The engine is compared on ordered (no sort) and shuffled (forced sort) input (See Figure 6-14). Both grow close to linearly over these six sizes, exactly what the reading note of Section 6.4.2 predicts, since the log factor barely moves over this range. The shaded gap is the extra latency associated with shuffled input, including the engine’s sort. It is 0.20 s at 2.22 M rows (0.43 s ordered versus 0.63 s shuffled), and its normalization by $n \log_2 n$ stays within the same order of magnitude (See Table A-15). Over this measured range, the shuffled-input penalty remains smaller than the total latency.

Sensitivity case 2: a state-dependent predicate. Shuffling changes ordering work but leaves the predicates row-local. A state-dependent predicate cannot use the same row-local Boolean matrix. To show this boundary, the `simple_sequence` query’s `DEFINE` becomes `B AS category='B' AND price > AVG(A.price)`, a running aggregate that depends on the match so far. Trino and Oracle evaluate it natively. All three produce identical output, so the semantic comparison is valid.

The repetition counts differ from the main matrix. The recorded engine values use five warmups and twenty measured runs per size. The retained Trino and Oracle values use one warmup and two measured runs per size. The systems ran sequentially. This test

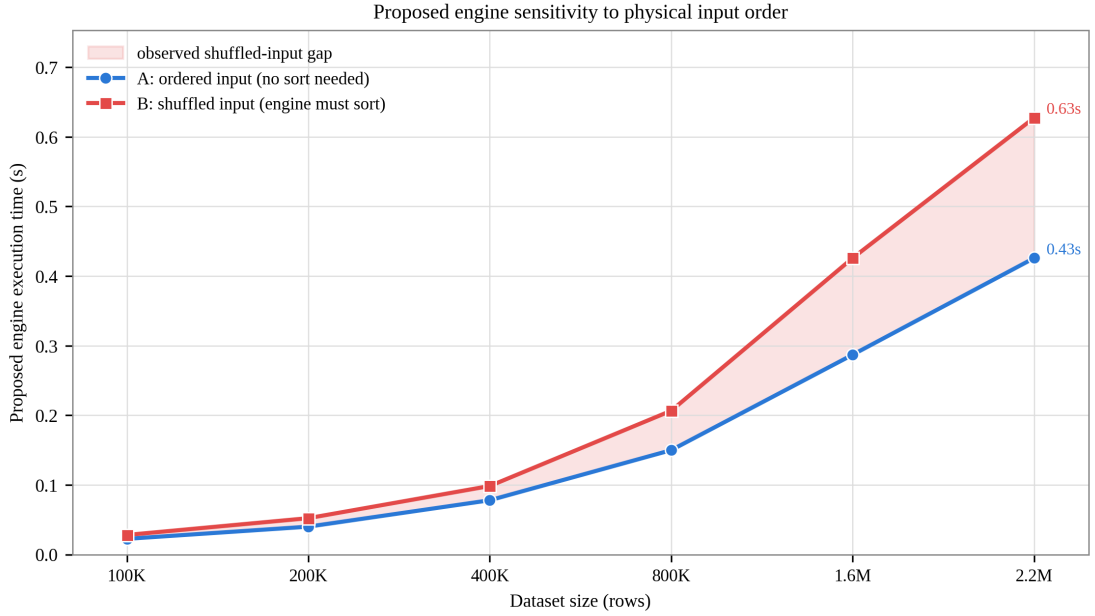


Figure 6-14: Engine time on ordered vs shuffled input, across sizes.

therefore shows the performance boundary, but its database means have less repetition than the main benchmark. The three systems are plotted for this test (See Figure 6-15).

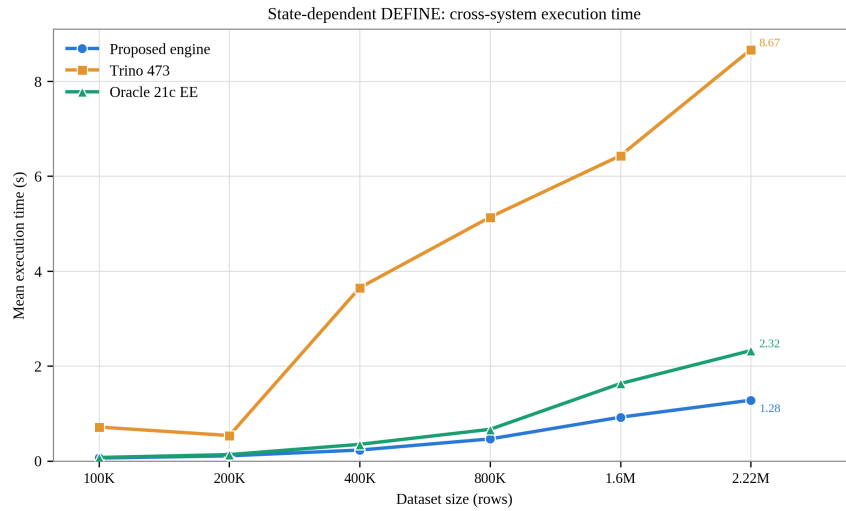


Figure 6-15: Execution time on a state-dependent *DEFINE* query, all systems.

An earlier recorded run reversed the ranking. The engine was the *slowest* system at every size, 64.41 s at 2.22 M rows, because the generic exact evaluator repeatedly recomputed `AVG(A.price)` as the tentative match grew. The exact matcher now compiles this form into an incremental plan. It updates the A prefix state on append and rollback instead of recomputing the aggregate. This evaluated query is linear, but it is not a row-local

vectorized predicate. It takes 1.28 s at 2.22 M rows, about $50\times$ faster than the earlier 64.41 s result, with identical output (181,147 matches). The engine is now fastest here too, $1.2\text{--}1.8\times$ ahead of Oracle and $5.0\text{--}15.9\times$ ahead of Trino (See Table A-16).

This is one compiled incremental form, not a general solution for all state-dependent expressions. Supported forms outside the compiled plans still use the generic exact evaluator, whose search can be exponential and can stop with an explicit resource-limit error.

The result is therefore scoped. The fixed compiled row-local benchmark is $\Theta(n)$ when ordered and $O(n \log n)$ when sorting is required. The tested running-AVG query is also linear after incremental compilation. No linear bound is claimed for generic exact search.

6.5.7 Scalability

Everything so far lives inside the range all three systems handle comfortably; this section asks how far each scales. The input is the real Amazon Reviews 2023 corpus. It uses nested prefixes of 5, 10, 20, 40, 80, 160, and 227.9 million rows, about $100\times$ the main dataset, with the five pattern families re-run on each. All three share one envelope at scale: one CPU core and 58 GB of RAM. Each reads the data in its own scalable mode. The engine uses RAM, Trino a disk-backed Hive/Parquet table, and Oracle 21c EE a native heap loaded by direct path. A query over the 600 s per-query budget is recorded as a wall; every completed database cell matched the engine’s output. Each stress-test cell uses two warmups and five measured runs. The proposed-engine values describe the benchmark revision recorded in the appendix (See Appendix A.1). The Oracle and Trino measurements were retained from the earlier campaign under the same one-core, 58 GB, two-warmup, five-run protocol. The systems ran sequentially. The results are plotted in two figures (See Figures 6-16 and 6-18); full values are in the appendix (See Tables A-17–A-18).

Time and throughput. Execution time is close to linear in n over the measured range. The per-pattern regressions give $R^2 = 0.9966\text{--}0.9998$ for the engine, $0.9824\text{--}0.9992$ for Oracle, and $0.9912\text{--}0.9999$ for Trino. Trino’s `complex_nested` fit uses its five completed sizes because the last two reached the time limit. These values quantify the observed trend; they do not replace the theoretical bounds in Section 6.4.2.

Left is execution time and right is absolute peak memory (See Figure 6-16). Each size average uses the same four families completed by all three systems at all seven sizes. The engine is fastest on every completed cell, about $5.1\text{--}10.1\times$ ahead of Oracle 21c EE and $11.4\text{--}13.8\times$ ahead of Trino on the size averages. It also finishes the full corpus on every pattern: 27.8 s for `simple_sequence`, 106.0 s for `complex_nested`. The engine and Oracle 21c EE each complete 35/35 cells. Trino completes 33/35: `complex_nested` crosses the 600 s time budget at 160 M and 227,899,533 rows. Because `complex_nested` is the slowest family for all three systems, a mean that kept it would count it for the engine and Oracle 21c EE at the two largest sizes but not for Trino. Trino’s average would then omit its hardest pattern while the others kept theirs, so the three would no longer describe the same workload, and the composition of the mean would change along the axis. The complete five-family values, `complex_nested` included, stay in the appendix (See Tables A-17–A-18).

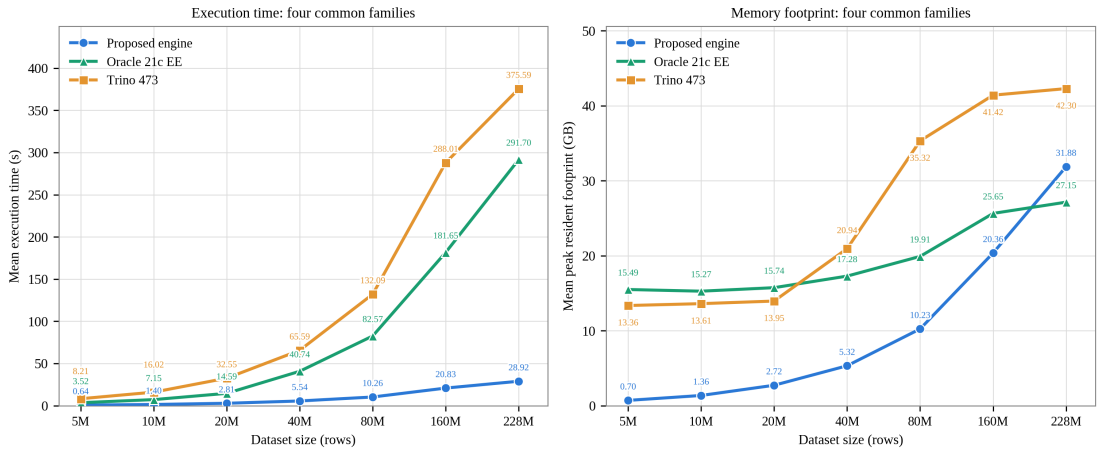


Figure 6-16: Volume scaling on real data: time (left) and peak memory (right).

Total resident memory tells a different story from the per-query figure below. The engine starts far leaner than either database: 0.7 GB at 5 M rows against 13.4 GB for Trino and 15.5 GB for Oracle. The database footprints include their resident JVM or SGA baselines. The engine stays the leanest of the three up to 160 M. Only at the full corpus does its resident data pass Oracle: 31.9 GB against Oracle 21c EE’s 27.1 GB, still well under Trino’s 42.3 GB (See Tables A-17–A-18).

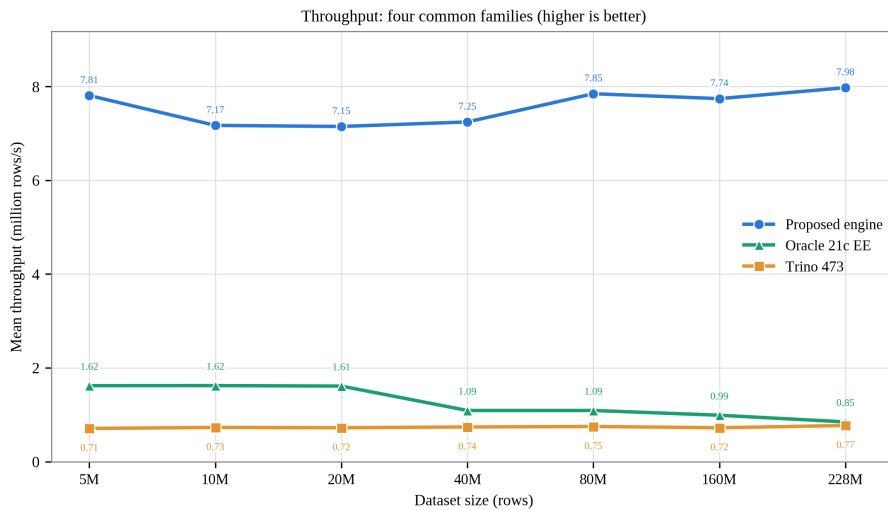


Figure 6-17: Throughput scaling on real data.

Throughput is again averaged over the same four common families (See Figure 6-17). The engine holds 7.1–8.0 M rows/s across the whole range. At the full size its patterns range from 2.2 M rows/s on `complex_nested` to 9.2 M rows/s on `alternation`. Oracle 21c EE records 0.8–1.6 M rows/s and 0.7–0.8 M for Trino. Oracle 21c EE drifts down as its fixed budgets spread over more data, while Trino stays near-flat, so the two converge at the full corpus. The per-pattern curves remain close to linear for all

three (See Figure A-1), and every completed cell returned identical match counts (See Tables A-19, A-22).

Ranking. At the full corpus, the four-family average ranks the engine first, Oracle 21c EE second, and Trino third. The order is not the same for every pattern.

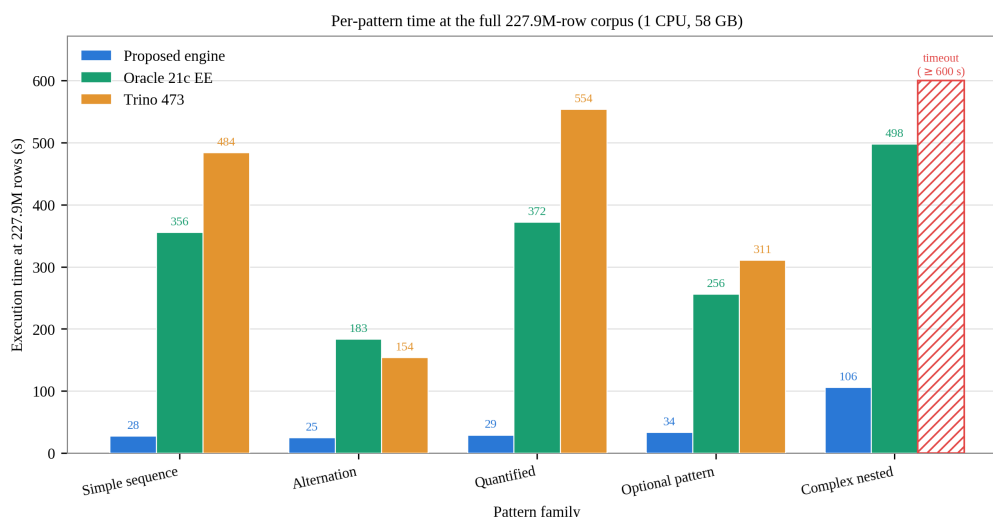
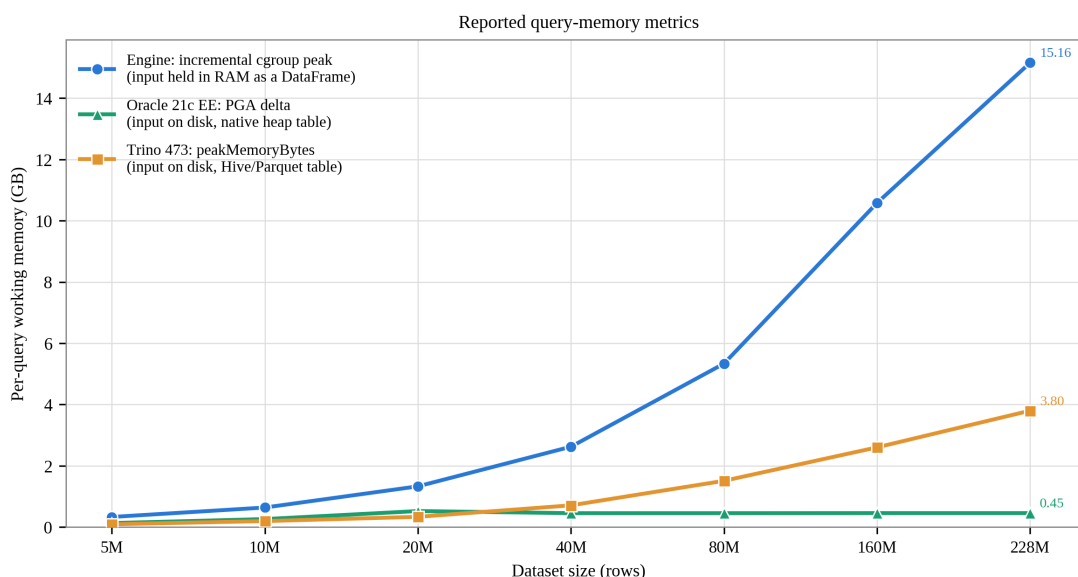


Figure 6-18: Per-pattern time at the full 227.9 M-row corpus.

The engine leads on all five patterns. Oracle 21c EE beats Trino on the three heavy families (`simple_sequence`, `quantified`, and `optional_pattern`); for `quantified`, the times are 372 s and 554 s. Oracle is also the only database to finish `complex_nested`, at 498 s. Trino wins only on the cheapest family, `alternation`, at 154 s versus 183 s. It reaches the time wall on `complex_nested`. Thus the compiled engine leads, Oracle 21c EE is the more robust database in this stress test, and Trino is the first to reach the time wall (See Table A-17).

Memory. The systems trade speed for memory in opposite directions.

The query-memory counters give a diagnostic view, not a same-instrument comparison (See Figure 6-19). The figure places the engine's cgroup incremental peak beside the database-native counters (Oracle PGA and Trino `peakMemoryBytes`). Each series is averaged over the same four common families. The engine reaches about 15 GB on average and 18.5 GB on the largest single cell at 227.9 M rows. Oracle stays near 0.5 GB and Trino below 4 GB according to their own counters. These counters have different accounting boundaries, so their magnitudes should not be read as an exact memory ratio. The same-instrument deployment comparison is the absolute cgroup footprint (See Figure 6-16 and Tables A-20–A-21). The largest observed engine footprint was 35.7 GB on one pattern, below the 58 GB limit. The experiment does not establish an exact maximum beyond this range.



All three systems share one envelope: 1 CPU core and 58 GB of RAM. Metric definitions and storage modes differ by system; values are reported without a causal interpretation.

Figure 6-19: Diagnostic query-memory counters; the accounting boundaries differ.

6.5.8 Evaluation Validity

A cross-system comparison only means something if the advantage cannot be an artifact of the measurement. Five checks define how the results should be read.

Correctness first. Times are compared only where the systems return the same answer; every cell was checked for row-level equality (See Table A-11). The 30/30 agreement is a correctness result and a precondition for comparing execution time.

Timer scope. Initial data loading is excluded for all systems. Query execution and result materialization are included, including database result transfer to the client. In the main matrix, five warmups reduce cold-start and JIT effects. The values are end-to-end query latencies, not isolated matcher-kernel times.

Stable measurements. Each main-matrix cell is the IQR-filtered mean of twenty runs. The median coefficient of variation, the standard deviation divided by the mean, is 3.5% (engine), 2.3% (Oracle), and 3.8% (Trino). The stress test uses fewer runs per cell, five after two warmups, but varies less: its median coefficient of variation is at most 1.0% for the three systems. This supports the stability of both sets of means.

Separate campaigns. The systems used the same host, logical queries, input rows, CPU allocation, and outer memory limit, but they were measured one after another rather than side by side. Every campaign ran on one CPU core, each with its own repetition count. The main benchmark uses five warmups and twenty measured runs per cell, under a 32 GB limit with resident inputs. The stress study uses two warmups and five runs, under 58 GB with system-appropriate scalable storage. The ordering test repeats the main protocol. The state-dependent case keeps twenty runs for the engine but two for the retained database values. No figure here comes from a concurrent head-to-head run.

Narrow scope. The claim is only that the engine is faster *on this workload* (one partition, in-memory input, the supported subset) on one machine. The systems had the

same outer CPU and memory limits in the main matrix. Their execution models and measured process boundaries also differ. Database tuning could change the gap.

6.5.9 Key Findings

The main findings are as follows.

Speed. The engine was fastest on all 30 cross-system cells, and it stayed $5.1\text{--}10.1\times$ ahead of Oracle 21c EE and $11.4\text{--}13.8\times$ ahead of Trino on the four common stress-test families to 227.9 M rows.

Why it is faster on this workload. The engine is specialized for in-process DataFrame matching. It avoids database planning, catalog, transaction, client-protocol, and data-transfer costs. Its compiler turns supported predicates into column operations and integer-code comparisons, then uses decision tables and grouped NumPy reductions to reduce per-row Python work. Ordered input also lets it skip sorting, although Section 6.5.6 shows that it remains fastest when sorting is forced. This advantage applies to the supported single-node workload, not to every pattern or deployment.

Correctness. All 30 main pattern–size combinations matched across the three systems. Every completed stress database cell also matched the engine. The engine and Oracle completed 35/35 stress cells; Trino completed 33/35.

Scalability. Execution time was close to linear for all three systems over the tested range. The engine’s measured memory was also close to linear, and it finished the full 227.9 M-row corpus. It did not reach an internal size or memory limit; RAM remains the likely boundary for larger in-memory inputs. Oracle 21c EE also reached the full size; Trino’s hardest pattern exceeded the time budget at 160 M and 227.9 M rows.

The trade-off. The engine keeps its DataFrame in RAM and builds whole-column working arrays. The input is already part of the memory baseline, while the incremental peak measures additional query-time growth. Its total footprint is lower than the resident database containers in the main benchmark, but that relationship reverses against Oracle at the largest stress-test size.

Honest limits. The lead is bounded by what the compiler can turn into column operations. The state-dependent DEFINE that once made both databases faster now uses a compiled incremental plan. This solves one aggregate form, not the full class. Other state-dependent or ambiguous forms can still use the generic exact evaluator and may reach its explicit work limit. At scale, the resident DataFrame makes RAM a likely boundary. The study maps where each system’s usable region lies; it does not crown a universal winner.

6.6 Chapter Summary

This chapter evaluated correctness, coverage, complexity, usability, performance, scalability, and memory. All three systems agreed in the 30 main pattern–size combinations, and every completed stress database cell matched the engine. The engine and Oracle completed all 35 stress cells; Trino completed 33. The engine recorded the lowest time on the evaluated workload and scaled close to linearly, while its query-time memory grew with the input. These findings apply to the supported single-node workload and stated resource limits. The next chapter summarizes the contributions, limitations, and future work.

Chapter 7: Conclusions and Future Work

This chapter brings the findings together. It interprets the results, states the limits of the work, and identifies the next steps. Section 7.1 discusses the evidence. Section 7.2 maps the main claims to that evidence, and Section 7.3 defines their boundaries. Sections 7.4 and 7.5 give the final conclusion and future work.

7.1 Discussion

The results show that a practical subset of SQL row-pattern recognition can run directly on Pandas DataFrames. For the reported correctness queries, the engine agreed with the applicable Trino 473, Oracle, and hand-written Pandas baselines. The cases covered partitioning, ordering, implicit DEFINE conditions, navigation, match numbering, and measure output. This supports the evaluated subset. It does not claim full SQL:2016 or vendor parity.

The usability comparison gives a second result. The three SQL examples used 14–16 lines of detection logic, compared with 23–31 lines for their hand-written Pandas versions. Those functions also needed explicit grouping, shifting, candidate filtering, index tracking, and skip handling. The SQL form keeps this logic in one declarative statement. That lowers developer effort, makes the pattern easier to read, and travels between notebook analysis and SQL workflows, as long as the query stays inside the supported subset.

The performance benchmark contained 30 pattern–size combinations per system: five pattern families over six sizes from 100 K to the full 2.22 M rows, so 90 cells across the three systems. Each cell is the mean of twenty measured runs after five warmups and the stated $1.5\times$ IQR outlier rule. Each system used the same input and query logic under one CPU core and the same outer 32 GB memory limit. Across those 30 cells the proposed engine averaged 0.17 s of execution time per cell and 6 110 392 rows/s of throughput. Oracle averaged 1.04 s and Trino 1.98 s per cell. The proposed engine had the lowest time in all 30 cells, and all 30 normalized outputs agreed across the three systems. Pattern structure remained important: `complex_nested` was slower than the simpler families.

Memory shows the trade-off. Across the main benchmark, the maximum incremental query peaks were 189.05 MB for the engine, 58.71 MB for Oracle, and 20.75 MB for Trino. The maximum total footprints showed the reverse order: 523.97 MB, 2,057.17 MB, and 14,992.90 MB, respectively. All six values use the same instrument: the Linux control-group (cgroup) counter that reports the memory of a set of processes. But the process boundaries differ: one engine process versus a complete database container. Vendor-native counters have different scopes and are not treated as equivalent measurements.

This result has a clear boundary. The engine is specialized for one in-process DataFrame workload. It avoids database planning, storage, and client-transfer layers, and it compiles supported conditions to column operations. The evaluated running-average predicate is also updated incrementally. Supported expressions outside a compiled form use the general evaluator; unsupported forms are rejected. State-dependent conditions

and some ambiguous pattern choices may require exact search. In the separate state-dependent test, the engine processed the full 2.22 M rows in 1.28 s and preserved the reference output. These results describe the tested subset on one machine. They are not a general ranking of SQL systems (Sections 6.5.9 and 6.5.6).

A separate scalability test used Amazon Reviews 2023 from 5 M to 227.9 M rows. Execution time was close to linear for all three systems. The engine’s measured memory was also close to linear. On the size averages, the engine remained $5.1\text{--}10.1\times$ faster than Oracle 21c EE and $11.4\text{--}13.8\times$ faster than Trino over the four pattern families completed by all three systems. Oracle 21c EE completed every cell. Trino’s `complex_nested` pattern reached the 600 s limit at the last two sizes. The engine completed the full corpus, but its maximum process footprint reached about 35.7 GB. Speed and memory residency must therefore be read together. The engine values describe the fixed benchmark revision recorded in the appendix (See Appendix A.1). The retained Oracle and Trino measurements came from separate campaigns under the same stated protocol. They were not run simultaneously.

The architecture separates parsing, validation, logical planning, automata construction, matching, and output assembly. The optimized paths use row-local masks, compact DFA and run-length plans, compiled measures, fused output, bounded caches, lazy row access, and conservative output projection. Partition processing remains sequential. The study therefore evaluates a single-node Pandas engine, not a distributed SQL system. Trino and Oracle retain wider SQL coverage, mature optimizers, and production execution infrastructure.

Every reported measurement has all of these optimized paths enabled, each activating where it applies, so the results describe the combined configuration. Differential tests compare each eligible fast path with the general path on the same input. The evidence is therefore that none of them changes the answer, not a measured contribution per technique. Two cases in Section 6.5.6 do isolate a path: the first forces the sort, and the second uses a predicate the row-local path cannot compile. A full ablation, each technique alone and in combination, is left to future work.

7.2 Claims and Evidence

The main claims of the thesis are listed with the evidence behind each and the boundary within which it holds (See Table 7-1). The purpose is to keep every conclusion inside what the evidence actually shows.

Table 7-1: Each thesis claim, the evidence behind it, and the scope within which it holds

Claim area	Supporting evidence	Validated conclusion	Scope boundary
Correctness	Selected comparisons with Trino, Oracle, and hand-written Pandas reported in Chapter 6.	The reported reference queries produce the same rows and measure values across the compared systems.	Selected supported scenarios only.

Table 7-1 (continued): Each thesis claim, the evidence behind it, and the scope within which it holds

Claim area	Supporting evidence	Validated conclusion	Scope boundary
Feature support	The coverage tables and reference tests in Chapter 6, supported by the design in Chapters 4 and 5.	The system supports a practical row-pattern subset suitable for DataFrame-centered analysis.	This is not full SQL:2016 or vendor coverage.
Performance	The repeated 30-cell benchmark to 2.22 M rows and the separate scalability sweep to 227.9 M rows.	The engine has the lowest time on the evaluated cells and shows near-linear measured scaling.	One host, selected patterns, and different repetition protocols for the main and scalability studies.
Memory use	Incremental-peak and total-footprint measurements in both benchmarks.	The measurements show the memory cost of the evaluated workloads.	Empirical measurements, not a formal upper bound.
Developer effort	Lines of detection code in the three worked examples.	The SQL form uses fewer lines and removes manual traversal logic.	Code comparison only, not a user study.
Architecture	The five-stage design and its automated tests.	The implementation separates parsing, planning, compilation, and execution concerns.	The system remains a Pandas-centered prototype.

Three validity limits remain. *Internal validity*, whether the measured differences come from the systems themselves, depends on the reference outputs and their normalization. *Construct validity*, whether the chosen metrics stand for the claims made from them, rests on row equality, time, throughput, memory, and line count. *External validity*, how far the results carry beyond this setup, is limited by one workstation, two performance datasets, and the selected patterns. The main benchmark uses twenty measured runs per cell; the larger sweep uses five. Neither campaign was collected in one simultaneous rerun. Trino and Oracle were not expert-tuned, and neither was the engine: all three ran with their default settings under the stated limits. Other configurations may narrow the measured gaps.

7.3 Limitations

The following limitations define how far the results can be generalized.

Supported subset, not exhaustive SQL. The engine targets a substantial but bounded slice of MATCH_RECOGNIZE: PARTITION BY, ORDER BY, PATTERN, DEFINE, selected MEASURES, supported skip and output modes, navigation, subsets, exclusion blocks, anchors, and PERMUTE. It does not claim full SQL:2016 coverage or parity with Trino or

Oracle. Conditional and subquery ordering keys, vendor-specific extensions, and other unsupported expression forms remain outside the validated scope.

Complex pattern and quantifier interactions. The automata design handles ordinary sequences, alternation, grouping, bounded repetition, and common quantifiers. Deeply nested expressions with several greedy or unbounded quantifiers can blow up the state and transition count, and PERMUTE is especially costly because its semantics are factorial. The engine therefore uses explicit NFA, DFA, and PERMUTE guards. If complete construction exceeds the state budget, compilation fails. It does not execute or cache an incomplete automaton. This preserves correctness, but a very large pattern may be rejected.

Predicates set a second limit. A DEFINE that cannot become column operations may need the exact search, which is the engine’s slowest path. The evaluated running AVG form is updated incrementally, but this does not cover every state-dependent expression. Exact search also has an explicit search budget. Its exhaustion is reported as an error rather than a changed result (Section 6.5.6).

Measure and aggregate scope. The measure evaluator supports common functions: SUM, COUNT, MIN, MAX, AVG, statistical and collection/string aggregates, conditional aggregates, and supported RUNNING/FINAL forms. Arbitrary user-defined aggregates and every possible expression combination remain out of scope. This is a natural limit of an in-memory engine with its own expression evaluator rather than a full SQL engine.

Performance boundaries. The core benchmark is predictable from 100 K to 2.22 M rows, and a separate sweep confirms near-linear execution time to 227.9 M rows. Measured engine memory also increased close to linearly on one workstation and one CPU core. Wider schemas, many small partitions, and highly ambiguous patterns were not tested at that scale. For large row-local scans, available RAM is the practical limit. Pattern compilation and exact search also have explicit algorithmic guards.

Memory behavior. Memory stayed within the configured limits, and query memory increased with input size. Small deltas still vary with allocator reuse, garbage collection, result materialization, and the 50 ms sampling interval. The reported values are therefore empirical measurements, not a formal upper bound. The input DataFrame and result must remain resident, so the current engine is not an out-of-core system.

Optimizer integration. The engine runs independently of database optimizers. It does no cost-based plan selection, predicate pushdown into external storage, join reordering, or distributed scheduling, so some plan-level optimizations that databases offer are simply absent here.

7.4 Conclusion

Pandas has no native row-pattern primitive: detecting a multi-row pattern means shifted columns, manual loops, and custom state handling, or a round trip through an external SQL engine. This thesis addressed that gap with a SQL-in-Pandas implementation that runs a supported subset of MATCH_RECOGNIZE over DataFrames.

The work makes three contributions. First, it provides an in-process Pandas engine with verified semantics for the evaluated R010-style subset. Second, it provides an optimized and guarded execution design for conditions, matching, measures, caching, output construction, and memory use. Third, it provides theoretical and empirical evaluation of correctness, coverage, usability, performance, and scalability.

The evidence supports three conclusions. The selected correctness scenarios agree with Trino, Oracle, and hand-written Pandas. The declarative examples use less detection code than their procedural baselines. The main and scalability benchmarks show predictable growth under the stated protocols, with an increasing in-memory cost.

The approach is a bridge between SQL row-pattern recognition and DataFrame analytics. It does not replace mature SQL engines; it brings a useful slice of their row-pattern power into Python workflows where the data already lives in Pandas. The conclusion holds within the limits stated in Section 7.3.

7.5 Future Work

Five directions would extend the work.

Broader SQL and dialect coverage. The most direct step is wider SQL:2016 and vendor coverage: more navigation combinations, broader expressions, richer aggregates, fuller RUNNING/FINAL semantics, and a larger conformance suite drawn from Trino, Oracle, and the standard.

Parallel and distributed execution. Independent partitions are a natural unit of parallel work. The current sequential partition loop could use independent workers with separate matcher state while preserving deterministic output order. Distributed execution on Dask or Spark would also need a rule for a partition split across chunks.

Backend abstraction and columnar acceleration. The path is Pandas-centered today. A backend abstraction could target Polars or Apache Arrow, pushing more predicate evaluation into compiled vectorized kernels and reducing Python iteration. An out-of-core backend could also reduce the resident-memory limit, but it would have to preserve partition order and matches across batch boundaries.

Optimization and diagnostics. The automata pipeline can gain from stronger state pruning, guard pushdown, better selectivity estimates, and a faster exact search for state-dependent forms not covered by compiled plans. A per-technique ablation would also quantify what each optimized path contributes on its own and alongside the others. This remains a performance boundary (Section 6.5.6). Better EXPLAIN output, match traces, and per-pattern counters would make each execution path easier to verify.

Usability and integration. Clearer error messages, notebook-friendly visualizations, reusable test fixtures, and integration with existing DataFrame query interfaces would make the engine easier to adopt in exploratory analysis and teaching.

7.6 Chapter Summary

This chapter drew the findings together in a scoped context. It read the evaluation in practical terms: supported semantics, developer effort, performance, and memory. It also separated measured results from their limits.

The conclusion is that a practical subset of SQL row-pattern recognition can run directly over DataFrames. This is supported by the reported correctness and performance evidence, not by a claim of full SQL parity. Future work follows the remaining limits: wider conformance, parallel execution, alternative backends, exact-search optimization, and better diagnostics.

References

- [1] D. Petković, “Identifying possible financial frauds using SQL row pattern recognition,” *International Journal of Computer Applications*, vol. 184, no. 35, pp. 31–34, Nov. 2022, ISSN: 0975-8887. DOI: 10.5120/ijca2022922446 [Online]. Available: <https://ijcaonline.org/archives/volume184/number35/32542-2022922446/>
- [2] M. Winand, *match_recognize — regular expressions over rows*, Modern SQL: A lot has changed since SQL-92. Accessed: Jun. 15, 2025. [Online]. Available: https://modern-sql.com/feature/match_recognize
- [3] F. Zemke, A. Witkowski, M. Cherniack, and L. Colby, “Pattern matching in sequences of rows,” ANSI Standard Proposal, Technical Report, 2007.
- [4] J. Melton and A. Eisenberg, “SQL:2016 support for row pattern recognition,” *SIGMOD Record*, vol. 46, no. 2, pp. 39–48, 2017. DOI: 10.1145/3105906.3105914
- [5] D. Petković, “Specification of row pattern recognition in the SQL standard and its implementations,” *Datenbank-Spektrum*, vol. 22, no. 2, pp. 163–174, Jul. 2022, ISSN: 1610-1995. DOI: 10.1007/s13222-022-00404-3
- [6] M. Ashraf, *Python udf example for row pattern recognition*, GitHub repository, 2025. [Online]. Available: https://github.com/MonierLawande/Row_match_recognize/blob/master/Python_Examples/python_udf.ipynb
- [7] Y. Cao, C.-Y. Chan, J. Li, and K.-L. Tan, *Optimization of analytic window functions*, 2012. arXiv: 1208.0086 [cs.DB]. [Online]. Available: <https://arxiv.org/abs/1208.0086>
- [8] Apache Flink, *MATCH_RECOGNIZE*, https://nightlies.apache.org/flink/flink-docs-release-1.20/docs/dev/table/sql/queries/match_recognize/, 2026.
- [9] International Organization for Standardization, *ISO/IEC TR 19075-5:2016 information technology – database languages – sql technical reports – part 5: Row pattern recognition in sql, feature r010: Row pattern recognition in from clause*, <https://www.iso.org/standard/65143.html>, 2016.
- [10] International Organization for Standardization and International Electrotechnical Commission, *Information technology — guidance for the use of database language sql — part 5: Row pattern recognition*, <https://www.iso.org/standard/78936.html>, ISO/IEC 19075-5:2021, Edition 1, 2021.
- [11] Oracle Corporation, *Pattern matching in oracle database 12c*, <https://docs.oracle.com/database/121/DWHSG/pattern.htm>, Accessed: 2024, 2013.

REFERENCES

- [12] Oracle, *Sql pattern matching in oracle database*, <https://docs.oracle.com/en/database/oracle/oracle-database/21/dwhsg/sql-pattern-matching-data-warehouses.html>, Oracle Database 21c Data Warehousing Guide, 2021. Accessed: Jun. 15, 2025.
- [13] K. Hansen, *Practical Oracle SQL: Mastering the Full Power of Oracle Database*. Apress Berkeley, CA, Jan. 2020, ISBN: 978-1-4842-5616-9. DOI: 10.1007/978-1-4842-5617-6
- [14] Trino, *MATCH_RECOGNIZE*, <https://trino.io/docs/current/sql/match-recognize.html>, 2022.
- [15] Trino, *Row pattern recognition with MATCH_RECOGNIZE*, https://trino.io/blog/2021/05/19/row_pattern_matching.html, 2021.
- [16] Snowflake Inc., *MATCH_RECOGNIZE clause*, Snowflake Documentation, 2025. Accessed: Jun. 18, 2025. [Online]. Available: https://docs.snowflake.com/en/sql-reference/constructs/match_recognize
- [17] PostgreSQL Global Development Group, *Unsupported features*, PostgreSQL 18 documentation, 2025. Accessed: Jul. 21, 2026. [Online]. Available: <https://www.postgresql.org/docs/18/unsupported-features-sql-standard.html>
- [18] DuckDB Foundation, *Development roadmap*, 2026. Accessed: Jul. 21, 2026. [Online]. Available: <https://duckdb.org/roadmap>
- [19] W. McKinney, “Data structures for statistical computing in Python,” in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61. DOI: 10.25080/Majora-92bf1922-00a
- [20] J. Michels et al., “The new and improved SQL: 2016 standard,” *ACM SIGMOD Record*, vol. 47, no. 2, pp. 51–60, 2018.
- [21] MonierAshraf, *Row_match_recognize: Regression test suite*, GitHub repository, 2025. [Online]. Available: https://github.com/MonierLawande/Row_match_recognize/tree/master/tests
- [22] M. Lawande, M. El-helw, and A. Awad, “Row pattern recognition for pandas: Bringing match_recognize to python dataframes,” in *2026 IEEE 23rd Mediterranean Electrotechnical Conference (MELECON)*, IEEE, 2026. DOI: 10.1109/MELECON64486.2026.11418885
- [23] ISO/IEC, “ISO/IEC 9075-2:2016 information technology — database languages — SQL — part 2: Foundation (SQL/Foundation),” International Organization for Standardization, International Standard, 2016.
- [24] F. Hoffa, *Funnel analytics with SQL: MATCH_RECOGNIZE() on Snowflake*, Medium, Towards Data Science, Mar. 2021. Accessed: Jun. 15, 2025. [Online]. Available: <https://medium.com/data-science/funnel-analytics-with-sql-match-recognize-on-snowflake-8bd576d9b7b1>
- [25] M. Winand, *What’s new in SQL:2016*, Modern SQL: A lot has changed since SQL-92, Jun. 2017. Accessed: Jun. 15, 2025. [Online]. Available: <https://modern-sql.com/blog/2017-06/whats-new-in-sql-2016>

REFERENCES

- [26] Oracle, *Pattern recognition with MATCH_RECOGNIZE*, https://docs.oracle.com/en/middleware/fusion-middleware/osa/19.1/cqlreference/pattern-recognition-match_recognize.html, 2022.
- [27] G. Howe, *Any interest in adding match_recognize?* PostgreSQL General Mailing List, Message-ID: 7e495cb8-ca03-4e3e-aa7e-45bf75212da8@Spark, 2020. Accessed: Jun. 18, 2025. [Online]. Available: <https://www.postgresql.org/message-id/7e495cb8-ca03-4e3e-aa7e-45bf75212da8@Spark>
- [28] T. Findling, *Bringing row pattern recognition to DuckDB: Translating MATCH_RECOGNIZE to WITH RECURSIVE*, Bachelor’s thesis, Eberhard Karls Universität Tübingen, 2025.
- [29] University of Tübingen, Database Systems Group, *Match_Recognize research project*, Database Systems Teaching Notes. Accessed: Jul. 7, 2025. [Online]. Available: <https://db.cs.uni-tuebingen.de/teaching/research-projects/match-recognize/>
- [30] University of Tübingen, Database Systems Group, *Student research projects*, Database Systems Teaching Notes. Accessed: Jul. 7, 2025. [Online]. Available: <https://db.cs.uni-tuebingen.de/teaching/research-projects/>
- [31] L. Lambrecht, T. Findling, S. Heid, M. Knüdel, and T. Grust, “Democratize MATCH_RECOGNIZE!” *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 5251–5254, 2025. doi: 10.14778/3750601.3750644
- [32] Google Cloud, *MATCH_RECOGNIZE clause in GoogleSQL*, BigQuery documentation, 2025. Accessed: Jul. 21, 2026. [Online]. Available: https://cloud.google.com/bigquery/docs/reference/standard-sql/query-syntax#match_recognize_clause
- [33] Databricks, *Databricks SQL release notes 2026*, MATCH_RECOGNIZE beta in Databricks SQL and Runtime 19.0, 2026. Accessed: Jul. 21, 2026. [Online]. Available: <https://docs.databricks.com/gcp/en/sql/release-notes/2026#match-recognize-is-now-in-beta>
- [34] Trino Project, *Trino documentation: Pattern recognition in window functions*, <https://trino.io/docs/current/sql/pattern-recognition-in-window.html>, Accessed: 2024, 2023.
- [35] Š. Šprem, N. Tomažin, J. Matečić, and M. Horvat, “Building advanced web applications using data ingestion and data processing tools,” *Electronics*, vol. 13, no. 4, p. 709, 2024.
- [36] E. Georgiou, *Evaluating Apache Flink, Kafka Streams, and Spark Streaming for scalable machine learning workflows*, 2025.
- [37] M. Schule, H. Lang, M. Springer, A. Kemper, T. Neumann, and S. Gunnemann, “In-database machine learning with sql on gpus,” in *Proceedings of the 33rd International Conference on Scientific and Statistical Database Management*, ser. SSDBM ’21, New York, NY, USA: Association for Computing Machinery, 2021, pp. 25–36, ISBN: 9781450384131. doi: 10.1145/3468791.3468840 [Online]. Available: <https://doi.org/10.1145/3468791.3468840>

REFERENCES

- [38] N. Makrynioti, R. Ley-Wild, and V. Vassalos, “Sql4ml: A declarative end-to-end workflow for machine learning,” *arXiv preprint arXiv:1907.12415*, 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:198968138>
- [39] D. Naphade, “The evolution and modernization of data pipeline architectures,” *European Journal of Computer Science and Information Technology*, vol. 13, no. 6, pp. 42–53, 2025.
- [40] M. Raasveldt and H. Mühleisen, “DuckDB: An embeddable analytical database,” in *Proceedings of the 2019 International Conference on Management of Data*, Amsterdam, Netherlands: Association for Computing Machinery, 2019, pp. 1981–1984. doi: 10.1145/3299869.3320212
- [41] DuckDB Foundation, *SQL on pandas*, DuckDB documentation, 2026. Accessed: Jul. 21, 2026. [Online]. Available: https://duckdb.org/docs/current/guides/python/sql_on_pandas
- [42] T. Æ. Mogensen, *Introduction to Compiler Design*, 3rd. Springer Nature, 2024, pp. 105–113.
- [43] E. Zhu, S. Huang, and S. Chaudhuri, “High-performance row pattern recognition using joins,” *Proceedings of the VLDB Endowment*, vol. 16, no. 5, pp. 1181–1194, 2023, ISSN: 2150-8097. doi: 10.14778/3579075.3579090
- [44] J. Lucas, Y. Idris, B. Contreras-Rojas, J.-A. Quiane-Ruiz, and S. Chawla, “Rheem-studio: Cross-platform data analytics made easy,” in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*, 2018, pp. 1573–1576. doi: 10.1109/ICDE.2018.00179
- [45] A. Mozzillo, L. Zecchini, L. Gagliardelli, A. Aslam, S. Bergamaschi, and G. Simonini, *Evaluation of dataframe libraries for data preparation on a single machine*, 2025. doi: 10.48786/EDBT.2025.27 [Online]. Available: <https://openproceedings.org/2025/conf/edbt/paper-96.pdf>
- [46] S. Chawla, B. Contreras Rojas, Z. Kaoudi, S. Kruse, and J.-A. Quiané-Ruiz, “Building your cross-platform application with RHEEM,” *arXiv preprint arXiv:1805.11723*, 2018. doi: 10.48550/arXiv.1805.11723 [Online]. Available: <https://arxiv.org/abs/1805.11723>
- [47] The pandas development team, *pandas 2.3.3 API Reference*, Official pandas documentation, 2025. Accessed: Jul. 22, 2026. [Online]. Available: <https://pandas.pydata.org/pandas-docs/version/2.3/reference/index.html>
- [48] asaniczka, *Amazon UK Products Dataset 2023*, Kaggle dataset, <https://www.kaggle.com/datasets/asaniczka/amazon-uk-products-dataset-2023> (accessed 2025-07-16), 2023.
- [49] Y. Hou, J. Li, Z. He, A. Yan, X. Chen, and J. McAuley, *Bridging Language and Items for Retrieval and Recommendation*, Amazon Reviews 2023 dataset, McAuley Lab, UC San Diego, <https://amazon-reviews-2023.github.io/> (accessed 2026-07-19), 2024.
- [50] MonierAshraf, *Row_match_recognize*, GitHub repository, 2025. [Online]. Available: https://github.com/MonierLawande/Row_match_recognize/blob/master/Examples%20Match_reocginze/Examples.ipynb

REFERENCES

- [51] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd. Boston, MA, USA: Addison-Wesley, 2006, ISBN: 978-0-321-45536-9.
- [52] A. Albrecht and contributors, *Python-sqlparse documentation*, <https://sqlparse.readthedocs.io/en/latest/>, Accessed: 25 June 2026, 2026.
- [53] T. Mao and contributors, *SQLGlot: Python sql parser and transpiler*, <https://github.com/tobymao/sqlglot>, Accessed: 25 June 2026, 2026.
- [54] E. Klahn and contributors, *mo-sql-parsing: More sql parsing*, <https://github.com/klahnakoski/mo-sql-parsing>, Accessed: 25 June 2026, 2026.
- [55] L. Gaifax and contributors, *pglast: Postgresql languages ast and statements prettifier*, <https://pglast.readthedocs.io/en/latest/>, Accessed: 25 June 2026, 2026.
- [56] ANTLR Team and Contributors, *Trino sql grammar for antlr*, <https://github.com/antlr/grammars-v4/tree/master/sql/trino>, 2022.
- [57] T. Parr, *The Definitive ANTLR 4 Reference*. Lewisville, TX, USA: Pragmatic Bookshelf, 2013, ISBN: 978-1-934356-99-9.
- [58] T. Parr, S. Harwell, and K. Fisher, “Adaptive LL(*) parsing: The power of dynamic analysis,” in *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications*, ser. OOPSLA ’14, ACM, 2014. DOI: 10.1145/2660193.2660202
- [59] E. Robson and E. Freeman, *Entwurfsmuster von Kopf bis Fuß*. O’Reilly, 2022, pp. 614–615.
- [60] K. Thompson, “Programming techniques: Regular expression search algorithm,” *Communications of the ACM*, vol. 11, no. 6, pp. 419–422, 1968. DOI: 10.1145/363347.363387
- [61] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd. Boston, MA, USA: Addison-Wesley, 2006, ISBN: 978-0-321-48681-3.
- [62] MonierAshraf, *SQL MATCH_RECOGNIZE on Pandas: Package documentation*, Package documentation; released on PyPI as <https://pypi.org/project/pandas-match-recognize/>, 2026. [Online]. Available: https://github.com/MonierLawande/Row_match_recognize/blob/master/README.md
- [63] M. Ashraf, *Oracle MATCH_RECOGNIZE — stock price examples*, GitHub repository, 2025. [Online]. Available: https://github.com/MonierLawande/Row_match_recognize/blob/master/Examples%20Match_reocginze/Oracle_data_tables.sql

Appendix A: Detailed Benchmark Tables

This appendix presents the detailed benchmark tables used in Chapter 6. The chapter explains the main results, while this appendix keeps the full values for verification. Unless stated otherwise, the tables use the Amazon UK Products benchmark and the cross-system protocol below.

A.1 Measurement Provenance and Reproducibility

All systems ran sequentially, one at a time. No two systems ran together, so they never competed for the same CPU core or memory. Proposed-engine runs were refreshed on revision f8b052d. Trino and Oracle values were retained from protocol-aligned earlier campaigns; the state-dependent repetition difference is disclosed below. The final regression run passed 709 tests.

Benchmark protocols. The evaluation used four fixed protocols:

- **Cross-system matrix.** One CPU core, an outer 32 GB memory limit, five warm-up runs, and twenty measured runs. The input was ordered, and the warm-up runs preceded measurement. The matrix contains five patterns and six sizes (100 K–2 222 742 rows), giving 30 cells per system. All 90 cells completed. The 30 proposed-engine cells are from the recorded engine campaign. The 60 database cells are retained results.
- **Stress test.** One CPU core, a 58 GB memory limit, two warm-up runs, and five measured runs. Each query had a 600 s limit. The test contains five patterns and seven sizes (5 M–227 899 533 rows). The proposed engine and Oracle completed 35 of 35 cells. Trino completed 33 of 35; `complex_nested` reached the time limit at 160 M and 227 899 533 rows.
- **Ordering test.** The matrix protocol was repeated after a deterministic shuffle. This removed physical pre-ordering while keeping the same logical ORDER BY key. All 90 shuffled cells completed.
- **State-dependent test.** One CPU core and a 32 GB memory limit were used with ordered input and `B AS category='B' AND price > AVG(A.price)`. The proposed engine used five warm-up runs and twenty measured runs per size. The retained Trino and Oracle reference values contain two measured runs per size. Their means are useful for comparison, but provide less evidence about variation. All 18 cells completed.

Memory was sampled every 50 ms from `memory.current`, the total that version 2 of the Linux control-group (cgroup) interface reports for a group of processes. The incremental peak subtracts a five-second pre-query baseline. Timing includes query execution and result construction or retrieval. For the proposed engine, the absolute peak covers its process. For Trino and Oracle, it covers the full database container. Database-native query counters are reported separately where available. Data generation

and loading are excluded. The scripts divide byte counters by 1024^2 and 1024^3 ; MB and GB in the memory tables therefore mean MiB and GiB.

The saved benchmark bundle contains the result tables and provenance records for the main matrix, ordering, state-dependent, and stress campaigns.

Measurement exclusions. No complete benchmark cell was omitted. The stated $1.5 \times$ IQR rule could remove individual samples from a cell. The number removed was retained with the measurements. A complete cell may be rejected only when there is recorded evidence of an invalid run, such as a wrong revision, incorrect resource limits, host contention, sampler failure, process failure, or incomplete output.

Known limitations. The engine is single-node and in-memory. Its practical limit depends on available RAM, input width, output size, and pattern structure. The executor selects among compiled row-local predicates, supported incremental aggregate paths, and general exact search. The selected path depends on both the expression and the pattern. The absence of a simple column operation does not by itself imply exact search.

A.2 Match Counts

Runtime and memory also depend on output size. The returned rows are therefore reported under ONE ROW PER MATCH (See Table A-1). The counts are identical across the three systems. In this dataset, optional_pattern produces the largest output and alternation the smallest.

Table A-1: Returned rows by pattern and size under ONE ROW PER MATCH

Size (rows)	simple_sequence	alternation	quantified	optional_pattern	complex_nested
100,000	9,067	1,828	5,643	15,247	9,420
200,000	16,093	3,692	10,064	27,172	17,759
400,000	34,391	5,803	19,012	52,950	32,360
800,000	66,330	9,782	32,804	99,214	58,616
1,600,000	130,338	20,276	67,358	197,253	119,299
2,222,742	181,147	28,776	95,650	276,776	167,641

A.3 Performance Summary by Pattern and by Dataset Size

The same 30 cells are summarized by pattern and by size (See Tables A-2 and A-3). The first groups results by pattern. The second groups them by dataset size. Each displayed time is an arithmetic mean of cell mean times. Each displayed throughput is an arithmetic mean of cell throughputs; it is not recomputed from the displayed mean time.

Table A-2: Performance summary by pattern across the three systems

Pattern	Proposed engine		Trino 473		Oracle 21c EE	
	Thr. (rows/s)	Time (s)	Thr. (rows/s)	Time (s)	Thr. (rows/s)	Time (s)
simple_sequence	8,134,451	0.10	610,551	1.27	1,202,967	0.73
alternation	5,445,348	0.15	744,335	1.04	1,606,719	0.53
quantified	7,751,889	0.11	600,410	1.35	1,041,152	0.85
optional_pattern	6,637,948	0.13	816,795	0.95	1,424,079	0.59
complex_nested	2,582,321	0.34	238,402	5.28	505,612	2.52

Table A-3: Performance summary by dataset size across the three systems

Size (rows)	Proposed engine		Trino 473		Oracle 21c EE	
	Thr. (rows/s)	Time (s)	Thr. (rows/s)	Time (s)	Thr. (rows/s)	Time (s)
100,000	5,029,576	0.02	402,647	0.26	1,078,819	0.10
200,000	5,843,626	0.04	555,411	0.38	1,206,014	0.18
400,000	6,015,052	0.08	618,312	0.72	1,175,201	0.38
800,000	6,608,913	0.15	642,407	1.89	1,141,999	0.99
1,600,000	6,886,390	0.29	683,152	3.82	1,165,356	2.00
2,222,742	6,278,793	0.43	710,662	4.79	1,169,247	2.62

A.4 Detailed Cross-System Benchmark Matrices

This section gives the full values behind the benchmark figures in Chapter 6. Execution time includes result construction or retrieval. For each measured run, throughput is input size divided by that run’s time. Cell throughput is the arithmetic mean of the retained per-run rates after applying the same $1.5 \times$ IQR rule independently to throughput. It is not input size divided by the cell’s mean time. Query memory is the incremental cgroup peak above the five-second baseline. Footprint memory is the absolute cgroup peak. Time and memory are also filtered independently and reported as means of the retained samples from twenty measured runs.

Table A-4: Execution time by pattern, size, and system (seconds; mean $\pm$ sample standard deviation)

Pattern	Size (rows)	Proposed	Trino 473	Oracle 21c EE
simple_sequence	100,000	0.015 $\pm$ 0.001	0.288 $\pm$ 0.040	0.088 $\pm$ 0.004
simple_sequence	200,000	0.025 $\pm$ 0.001	0.366 $\pm$ 0.033	0.151 $\pm$ 0.004
simple_sequence	400,000	0.049 $\pm$ 0.002	0.675 $\pm$ 0.040	0.358 $\pm$ 0.004
simple_sequence	800,000	0.093 $\pm$ 0.007	1.166 $\pm$ 0.029	0.678 $\pm$ 0.012
simple_sequence	1,600,000	0.170 $\pm$ 0.004	2.225 $\pm$ 0.144	1.298 $\pm$ 0.011
simple_sequence	2,222,742	0.269 $\pm$ 0.015	2.887 $\pm$ 0.117	1.816 $\pm$ 0.015
alternation	100,000	0.023 $\pm$ 0.001	0.230 $\pm$ 0.027	0.073 $\pm$ 0.006
alternation	200,000	0.040 $\pm$ 0.001	0.333 $\pm$ 0.026	0.129 $\pm$ 0.006
alternation	400,000	0.072 $\pm$ 0.001	0.521 $\pm$ 0.013	0.244 $\pm$ 0.009
alternation	800,000	0.141 $\pm$ 0.009	0.950 $\pm$ 0.016	0.477 $\pm$ 0.006
alternation	1,600,000	0.265 $\pm$ 0.005	1.800 $\pm$ 0.024	0.947 $\pm$ 0.011
alternation	2,222,742	0.373 $\pm$ 0.006	2.409 $\pm$ 0.015	1.309 $\pm$ 0.013
quantified	100,000	0.016 $\pm$ 0.001	0.242 $\pm$ 0.019	0.105 $\pm$ 0.006
quantified	200,000	0.028 $\pm$ 0.001	0.347 $\pm$ 0.015	0.183 $\pm$ 0.008
quantified	400,000	0.053 $\pm$ 0.004	0.659 $\pm$ 0.030	0.369 $\pm$ 0.009
quantified	800,000	0.089 $\pm$ 0.001	1.272 $\pm$ 0.032	0.788 $\pm$ 0.008
quantified	1,600,000	0.178 $\pm$ 0.004	2.360 $\pm$ 0.049	1.534 $\pm$ 0.017
quantified	2,222,742	0.281 $\pm$ 0.020	3.202 $\pm$ 0.060	2.121 $\pm$ 0.014
optional_pattern	100,000	0.018 $\pm$ 0.001	0.195 $\pm$ 0.011	0.085 $\pm$ 0.008
optional_pattern	200,000	0.031 $\pm$ 0.001	0.287 $\pm$ 0.010	0.149 $\pm$ 0.007
optional_pattern	400,000	0.063 $\pm$ 0.003	0.498 $\pm$ 0.026	0.283 $\pm$ 0.012
optional_pattern	800,000	0.110 $\pm$ 0.005	0.892 $\pm$ 0.046	0.528 $\pm$ 0.016
optional_pattern	1,600,000	0.218 $\pm$ 0.003	1.617 $\pm$ 0.019	1.031 $\pm$ 0.027
optional_pattern	2,222,742	0.331 $\pm$ 0.011	2.231 $\pm$ 0.035	1.455 $\pm$ 0.041
complex_nested	100,000	0.040 $\pm$ 0.001	0.332 $\pm$ 0.025	0.138 $\pm$ 0.010
complex_nested	200,000	0.076 $\pm$ 0.002	0.579 $\pm$ 0.027	0.281 $\pm$ 0.006
complex_nested	400,000	0.152 $\pm$ 0.005	1.271 $\pm$ 0.026	0.653 $\pm$ 0.010
complex_nested	800,000	0.317 $\pm$ 0.018	5.180 $\pm$ 0.177	2.466 $\pm$ 0.023
complex_nested	1,600,000	0.604 $\pm$ 0.013	11.108 $\pm$ 0.094	5.193 $\pm$ 0.032
complex_nested	2,222,742	0.875 $\pm$ 0.029	13.213 $\pm$ 0.050	6.411 $\pm$ 0.045

Appendix A: Detailed Benchmark Tables

Table A-5: Throughput (rows/s) by pattern, dataset size, and system

Pattern	Size (rows)	Proposed	Trino 473	Oracle 21c EE
simple_sequence	100,000	6,548,744	345,160	1,139,312
simple_sequence	200,000	8,011,331	550,642	1,326,602
simple_sequence	400,000	8,052,822	594,845	1,117,414
simple_sequence	800,000	8,559,369	686,654	1,177,723
simple_sequence	1,600,000	9,403,104	714,822	1,232,833
simple_sequence	2,222,742	8,231,338	771,181	1,223,919
alternation	100,000	4,400,765	439,363	1,374,150
alternation	200,000	4,977,878	604,704	1,555,973
alternation	400,000	5,573,944	767,586	1,644,350
alternation	800,000	5,709,821	842,631	1,677,404
alternation	1,600,000	6,046,363	888,861	1,689,728
alternation	2,222,742	5,963,317	922,867	1,698,711
quantified	100,000	6,119,628	414,663	959,927
quantified	200,000	7,046,915	577,203	1,095,056
quantified	400,000	7,462,262	608,481	1,085,087
quantified	800,000	8,955,091	629,296	1,015,188
quantified	1,600,000	8,990,669	678,342	1,043,489
quantified	2,222,742	7,936,771	694,474	1,048,168
optional_pattern	100,000	5,584,402	511,386	1,190,721
optional_pattern	200,000	6,542,233	698,482	1,340,712
optional_pattern	400,000	6,353,474	805,792	1,416,523
optional_pattern	800,000	7,287,498	898,860	1,515,186
optional_pattern	1,600,000	7,341,539	989,684	1,552,628
optional_pattern	2,222,742	6,718,543	996,563	1,528,702
complex_nested	100,000	2,494,341	302,662	729,985
complex_nested	200,000	2,639,771	346,023	711,728
complex_nested	400,000	2,632,758	314,855	612,631
complex_nested	800,000	2,532,787	154,594	324,492
complex_nested	1,600,000	2,650,277	144,049	308,102
complex_nested	2,222,742	2,543,994	168,226	346,736

Table A-6: Incremental query memory by pattern, size, and system (MB)

Pattern	Size (rows)	Proposed	Trino 473	Oracle 21c EE
simple_sequence	100,000	5.87	1.05	3.14
simple_sequence	200,000	14.94	0.73	5.32
simple_sequence	400,000	28.68	0.79	10.91
simple_sequence	800,000	59.37	0.71	22.88
simple_sequence	1,600,000	115.20	20.75	43.45
simple_sequence	2,222,742	161.89	7.20	58.65
alternation	100,000	6.06	0.64	2.42
alternation	200,000	13.38	0.49	5.22
alternation	400,000	24.22	0.56	11.62
alternation	800,000	48.40	0.73	21.03
alternation	1,600,000	90.12	2.20	42.12
alternation	2,222,742	119.05	3.05	58.41
quantified	100,000	6.83	0.68	2.65
quantified	200,000	17.07	0.68	5.65
quantified	400,000	33.31	0.73	10.69
quantified	800,000	65.55	1.55	21.13
quantified	1,600,000	129.68	2.42	42.21
quantified	2,222,742	189.05	2.33	58.71
optional_pattern	100,000	8.45	0.53	2.72
optional_pattern	200,000	16.17	0.57	5.49
optional_pattern	400,000	32.35	0.71	10.62
optional_pattern	800,000	64.51	1.12	20.88
optional_pattern	1,600,000	126.43	0.86	41.76
optional_pattern	2,222,742	186.24	1.59	58.26
complex_nested	100,000	7.50	0.71	2.63

Appendix A: Detailed Benchmark Tables

Table A-6 (continued): Query memory (MB) by pattern, size, and system

Pattern	Size (rows)	Proposed	Trino 473	Oracle 21c EE
complex_nested	200,000	13.77	0.70	5.42
complex_nested	400,000	28.52	0.71	10.84
complex_nested	800,000	57.29	1.75	21.07
complex_nested	1,600,000	107.09	1.92	42.33
complex_nested	2,222,742	159.30	2.17	58.61

Table A-7: Absolute footprint memory by pattern, size, and system (MB)

Pattern	Size (rows)	Proposed	Trino 473	Oracle 21c EE
simple_sequence	100,000	84.69	8,439.12	1,988.23
simple_sequence	200,000	105.82	8,832.13	1,938.77
simple_sequence	400,000	142.06	9,567.85	1,939.79
simple_sequence	800,000	219.85	10,999.08	1,993.00
simple_sequence	1,600,000	363.39	13,124.03	2,000.77
simple_sequence	2,222,742	492.84	14,882.81	2,048.95
alternation	100,000	84.35	8,585.77	1,991.35
alternation	200,000	101.29	9,243.71	1,953.46
alternation	400,000	129.50	9,826.87	1,978.45
alternation	800,000	190.33	11,007.22	2,013.57
alternation	1,600,000	298.20	13,266.95	2,037.13
alternation	2,222,742	397.95	14,958.94	2,049.51
quantified	100,000	86.84	8,778.67	1,992.95
quantified	200,000	107.49	9,248.96	1,988.29
quantified	400,000	143.42	9,842.49	2,011.17
quantified	800,000	219.66	11,023.44	2,001.70
quantified	1,600,000	363.83	13,277.69	2,040.85
quantified	2,222,742	507.62	14,969.31	2,055.91
optional_pattern	100,000	89.03	8,785.11	1,975.58
optional_pattern	200,000	107.90	9,253.51	1,976.71
optional_pattern	400,000	146.19	9,851.34	1,983.32
optional_pattern	800,000	228.40	11,037.41	1,959.17
optional_pattern	1,600,000	363.50	13,284.68	2,010.72
optional_pattern	2,222,742	523.97	14,980.42	2,037.57
complex_nested	100,000	89.84	8,789.27	1,961.78
complex_nested	200,000	107.47	9,257.82	1,966.53
complex_nested	400,000	146.49	9,864.07	1,979.70
complex_nested	800,000	229.89	11,054.94	1,960.23
complex_nested	1,600,000	366.45	13,309.19	2,030.26
complex_nested	2,222,742	507.50	14,992.90	2,057.17

The engine footprint is its process peak. The Trino and Oracle footprints cover their full containers. This difference matters when reading the absolute values (See Tables A-8 and A-10).

Table A-8: Overall benchmark statistics by system

System	Cells	Cell-Mean Time Total (s)	Avg Time (s)	Avg Thr. (rows/s)	Thr. Range (rows/s)	Max Query Mem. (MB)	Max Footpr. Mem. (MB)
Proposed engine	30	5.02	0.17	6,110,392	2,494,341–9,403,104	189.05	523.97
Trino 473	30	59.33	1.98	602,098	144,049–996,563	20.75	14,992.90
Oracle 21c EE	30	31.34	1.04	1,156,106	308,102–1,698,711	58.71	2,057.17

The cell-mean time total adds the 30 cell mean times; it is not the duration of one workload run. Average time is the arithmetic mean of those 30 cell means. Average throughput is the arithmetic mean of the 30 cell throughputs defined above.

Table A-9: Average query memory by dataset size, all three systems

Size (rows)	Proposed (MB)	Trino 473 (MB)	Oracle 21c EE (MB)
100,000	6.94	0.72	2.71
200,000	15.07	0.63	5.42
400,000	29.42	0.70	10.93
800,000	59.02	1.17	21.40
1,600,000	113.70	5.63	42.37
2,222,742	163.11	3.27	58.53

Table A-10: Average footprint memory by dataset size, all three systems

Size (rows)	Proposed (MB)	Trino 473 (MB)	Oracle 21c EE (MB)
100,000	86.95	8,675.59	1,981.98
200,000	106.00	9,167.22	1,964.75
400,000	141.53	9,790.52	1,978.49
800,000	217.63	11,024.42	1,985.53
1,600,000	351.08	13,252.51	2,023.95
2,222,742	485.97	14,956.88	2,049.82

Table A-11: Cross-system correctness by pattern across six dataset sizes

Pattern	Proposed engine	Trino 473	Oracle 21c EE
simple_sequence	reference result	6/6 identical	6/6 identical
alternation	reference result	6/6 identical	6/6 identical
quantified	reference result	6/6 identical	6/6 identical
optional_pattern	reference result	6/6 identical	6/6 identical
complex_nested	reference result	6/6 identical	6/6 identical

For each metric, the relative value is the ratio of the two arithmetic means across the 30 cells:

$$100 \left(\bar{x}_{\text{proposed}} / \bar{x}_{\text{reference}} - 1 \right).$$

It is not the arithmetic mean of 30 cell-wise percentage differences. Lower execution time and memory are better. Higher throughput is better.

Table A-12: Relative difference between the proposed engine and each SQL engine

Metric	vs. Trino 473	vs. Oracle 21c EE
Execution time	−92%	−84%
Throughput	+915%	+429%
Query memory	+3093%	+174%
Operational footprint	−98%	−88%

A.5 Ordered vs. Shuffled Input (Sorting Scenarios)

These tables support the ordering analysis in Section 6.5.6. Scenario A uses input that is already ordered by seq_id. Scenario B shuffles the physical rows before the same logical ordering request. The logical order and match counts remain unchanged in all 90 system–pattern–size cells.

Table A-13: Average execution time for ordered and shuffled input

System	A ordered (s)	B shuffled (s)	Slowdown
Proposed engine	0.167	0.240	1.43×
Trino 473	1.978	2.193	1.11×
Oracle 21c EE	1.045	1.083	1.04×

Table A-14: Execution time and slowdown by size for ordered (“ord”) and shuffled (“shuf”) input

Size (rows)	Proposed engine			Trino 473			Oracle 21c EE		
	ord	shuf	slow	ord	shuf	slow	ord	shuf	slow
100,000	0.022	0.028	1.25×	0.257	0.283	1.10×	0.098	0.086	0.88×
200,000	0.040	0.052	1.30×	0.382	0.402	1.05×	0.178	0.166	0.93×
400,000	0.078	0.098	1.26×	0.725	0.775	1.07×	0.381	0.377	0.99×
800,000	0.150	0.207	1.38×	1.892	1.973	1.04×	0.987	0.986	1.00×
1,600,000	0.287	0.426	1.48×	3.822	4.180	1.09×	2.001	2.077	1.04×
2,222,742	0.426	0.627	1.47×	4.788	5.546	1.16×	2.622	2.810	1.07×

Table A-15: Observed ordered–shuffled time difference for the proposed engine by dataset size

Size (rows)	A (s)	B (s)	Δt (ms)	$\Delta t/n$ (ns)	$\Delta t/(n \log_2 n)$ (ns)
100,000	0.022	0.028	5.6	56.0	3.37
200,000	0.040	0.052	12.2	61.0	3.46
400,000	0.078	0.098	20.5	51.2	2.75
800,000	0.150	0.207	56.6	70.8	3.61
1,600,000	0.287	0.426	138.9	86.8	4.21
2,222,742	0.426	0.627	201.5	90.7	4.30

The difference between Scenarios A and B estimates the extra cost associated with shuffled input. Ordering is the main added operation in the proposed engine, but cache and row-layout effects may also contribute. The difference per row rises slightly, while the value divided by $n \log_2 n$ stays in a narrow range. This is consistent with comparison-based sorting over the tested sizes; it does not prove the asymptotic bound. The Oracle value below one at 100 K rows is within small-run variation and does not show that shuffling improves execution.

Table A-16: Mean execution time for the state-dependent aggregate pattern

Size (rows)	Proposed engine (s)	Trino 473 (s)	Oracle 21c EE (s)
100,000	0.060	0.713	0.073
200,000	0.107	0.535	0.133
400,000	0.229	3.649	0.349
800,000	0.463	5.136	0.666
1,600,000	0.921	6.432	1.630
2,222,742	1.279	8.665	2.324

This pattern uses `price > AVG(A.price)` in DEFINE. For this supported form, the engine evaluates the running aggregate incrementally in one scan. Ordered input lets it skip sorting after checking the order. All three systems return the same output.

A.6 Stress-Test Results

The stress test in Section 6.5.7 uses real Amazon Reviews 2023 data from 5 M to 227.9 M rows. Within this campaign, the three systems use equivalent queries and logical rows. The storage representations also differ. The proposed engine holds a DataFrame in RAM. Trino scans a Hive/Parquet table. Oracle 21c EE scans a native table. Loading time is excluded. Every completed database result was checked against the engine output. The recorded proposed-engine measurements and retained database measurements were collected in separate campaigns under the same stress-test protocol. In total, 103 of 105 cells completed: 35 proposed-engine cells, 35 Oracle cells, and 33 Trino cells. The two incomplete Trino cells reached the stated time limit.

Appendix A: Detailed Benchmark Tables

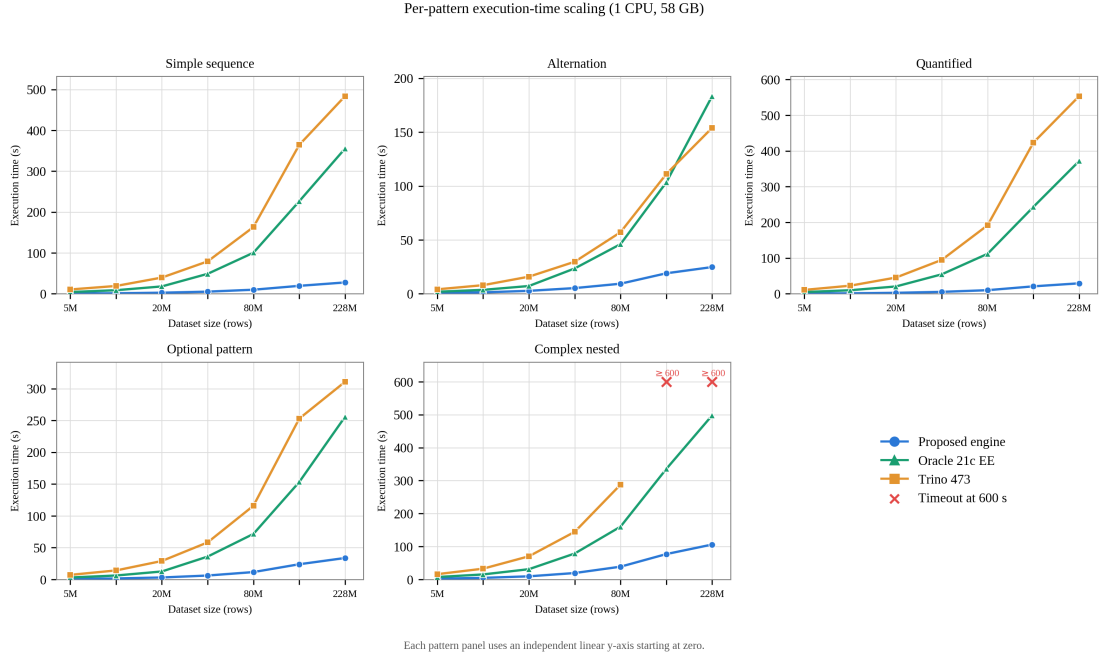


Figure A-1: Execution-time scaling by pattern under one CPU core and a 58 GB memory limit. Red $\times$ symbols mark Trino runs that reached the 600-second limit.

Execution time is reported as the mean and sample standard deviation of retained samples from five measured runs. Stress-test throughput is the arithmetic mean of the retained per-run rates, using the same independent $1.5 \times$ IQR rule. *wall* means that the query reached the 600 s limit. Absolute memory includes the idle engine or database baseline. It covers the proposed-engine process or the full database container. Incremental memory subtracts that baseline. Oracle’s native value is its session PGA high-water mark. Trino’s native value is its reported query peak. A small incremental Trino value can therefore mean that memory was already reserved by the JVM; it does not mean that the server had no memory footprint.

Appendix A: Detailed Benchmark Tables

Table A-17: Stress-test execution time (seconds; mean $\pm$ SD)

Size (rows)	simple_sequence	alternation	quantified	optional_pattern	complex_nested
<i>Proposed engine</i>					
5,000,000	0.60 $\pm$ 0.01	0.67 $\pm$ 0.00	0.60 $\pm$ 0.01	0.70 $\pm$ 0.01	2.34 $\pm$ 0.02
10,000,000	1.30 $\pm$ 0.00	1.36 $\pm$ 0.01	1.37 $\pm$ 0.02	1.57 $\pm$ 0.00	4.81 $\pm$ 0.09
20,000,000	2.67 $\pm$ 0.02	2.73 $\pm$ 0.02	2.72 $\pm$ 0.03	3.11 $\pm$ 0.02	9.54 $\pm$ 0.06
40,000,000	5.26 $\pm$ 0.01	5.29 $\pm$ 0.01	5.44 $\pm$ 0.05	6.19 $\pm$ 0.07	19.50 $\pm$ 0.38
80,000,000	9.97 $\pm$ 0.14	9.36 $\pm$ 0.02	10.09 $\pm$ 0.25	11.62 $\pm$ 0.09	38.78 $\pm$ 1.38
160,000,000	19.49 $\pm$ 0.02	19.04 $\pm$ 0.27	20.87 $\pm$ 0.19	23.94 $\pm$ 0.06	77.03 $\pm$ 2.24
227,899,533	27.79 $\pm$ 0.04	24.81 $\pm$ 0.15	29.34 $\pm$ 0.08	33.76 $\pm$ 0.06	105.99 $\pm$ 2.28
<i>Oracle 21c EE</i>					
5,000,000	4.27 $\pm$ 0.05	1.88 $\pm$ 0.01	4.83 $\pm$ 0.02	3.10 $\pm$ 0.02	7.42 $\pm$ 0.03
10,000,000	8.78 $\pm$ 0.00	3.65 $\pm$ 0.02	9.95 $\pm$ 0.11	6.22 $\pm$ 0.01	15.25 $\pm$ 0.12
20,000,000	18.05 $\pm$ 0.08	7.22 $\pm$ 0.02	20.55 $\pm$ 0.05	12.55 $\pm$ 0.05	31.18 $\pm$ 0.12
40,000,000	48.89 $\pm$ 0.41	23.54 $\pm$ 0.63	54.54 $\pm$ 0.66	35.98 $\pm$ 0.45	78.75 $\pm$ 0.72
80,000,000	100.55 $\pm$ 1.16	46.08 $\pm$ 1.26	111.99 $\pm$ 1.15	71.67 $\pm$ 1.11	160.26 $\pm$ 1.43
160,000,000	226.56 $\pm$ 2.02	103.42 $\pm$ 3.83	243.30 $\pm$ 4.42	153.30 $\pm$ 3.38	335.53 $\pm$ 4.38
227,899,533	355.62 $\pm$ 8.88	183.46 $\pm$ 9.08	371.81 $\pm$ 8.03	255.91 $\pm$ 7.55	498.04 $\pm$ 9.54
<i>Trino 473 (disk connector)</i>					
5,000,000	10.58 $\pm$ 0.17	4.15 $\pm$ 0.18	10.90 $\pm$ 0.32	7.20 $\pm$ 0.09	16.05 $\pm$ 0.05
10,000,000	19.17 $\pm$ 0.17	8.00 $\pm$ 0.14	22.70 $\pm$ 0.51	14.19 $\pm$ 0.10	32.72 $\pm$ 0.34
20,000,000	39.75 $\pm$ 0.39	15.89 $\pm$ 0.08	45.53 $\pm$ 0.15	29.04 $\pm$ 0.09	70.32 $\pm$ 0.06
40,000,000	79.35 $\pm$ 0.49	29.75 $\pm$ 0.06	94.96 $\pm$ 0.47	58.31 $\pm$ 0.22	144.54 $\pm$ 2.84
80,000,000	163.39 $\pm$ 0.86	57.13 $\pm$ 0.14	191.85 $\pm$ 0.39	115.97 $\pm$ 0.24	287.75 $\pm$ 0.55
160,000,000	364.58 $\pm$ 2.02	111.17 $\pm$ 0.65	423.30 $\pm$ 1.02	253.00 $\pm$ 6.50	wall
227,899,533	483.85 $\pm$ 17.26	154.04 $\pm$ 0.90	553.56 $\pm$ 2.83	310.91 $\pm$ 1.58	wall

Table A-18: Stress-test absolute peak memory (GB)

Size (rows)	simple_sequence	alternation	quantified	optional_pattern	complex_nested
<i>Proposed engine</i>					
5,000,000	0.66	0.59	0.75	0.80	0.77
10,000,000	1.26	1.21	1.44	1.52	1.47
20,000,000	2.53	2.45	2.84	3.06	3.03
40,000,000	4.93	4.78	5.57	6.00	5.97
80,000,000	9.51	9.03	10.75	11.64	9.90
160,000,000	18.92	17.97	21.49	23.06	19.71
227,899,533	30.08	28.36	33.40	35.67	30.89
<i>Oracle 21c EE</i>					
5,000,000	15.49	15.49	15.50	15.50	15.50
10,000,000	15.25	15.28	15.28	15.28	15.26
20,000,000	15.74	15.73	15.76	15.72	15.88
40,000,000	17.18	17.19	17.36	17.39	17.40
80,000,000	19.78	19.94	19.99	19.93	19.99
160,000,000	25.63	25.63	25.63	25.70	25.69
227,899,533	27.02	26.87	26.98	27.72	27.71
<i>Trino 473 (disk connector)</i>					
5,000,000	13.32	13.35	13.38	13.39	13.40
10,000,000	13.59	13.60	13.61	13.62	13.83
20,000,000	13.88	13.90	13.92	14.12	16.23
40,000,000	18.99	20.48	21.45	22.86	27.20
80,000,000	25.31	30.07	38.40	47.51	47.55
160,000,000	30.60	39.23	47.87	47.97	wall
227,899,533	37.83	47.46	47.48	36.42	wall

Appendix A: Detailed Benchmark Tables

Table A-19: Stress-test throughput (million rows/s)

Size (rows)	simple_sequence	alternation	quantified	optional_pattern	complex_nested
<i>Proposed engine</i>					
5,000,000	8.33	7.43	8.31	7.16	2.14
10,000,000	7.68	7.34	7.28	6.39	2.08
20,000,000	7.50	7.32	7.34	6.43	2.10
40,000,000	7.60	7.56	7.36	6.47	2.05
80,000,000	8.03	8.55	7.93	6.88	2.07
160,000,000	8.21	8.41	7.67	6.68	2.08
227,899,533	8.20	9.19	7.77	6.75	2.15
<i>Oracle 21c EE</i>					
5,000,000	1.17	2.66	1.03	1.62	0.67
10,000,000	1.14	2.74	1.00	1.61	0.66
20,000,000	1.11	2.77	0.97	1.59	0.64
40,000,000	0.82	1.70	0.73	1.11	0.51
80,000,000	0.80	1.74	0.71	1.12	0.50
160,000,000	0.71	1.55	0.66	1.04	0.48
227,899,533	0.64	1.24	0.61	0.89	0.46
<i>Trino 473 (disk connector)</i>					
5,000,000	0.47	1.21	0.46	0.69	0.31
10,000,000	0.52	1.25	0.44	0.70	0.31
20,000,000	0.50	1.26	0.44	0.69	0.28
40,000,000	0.50	1.34	0.42	0.69	0.28
80,000,000	0.49	1.40	0.42	0.69	0.28
160,000,000	0.44	1.44	0.38	0.63	wall
227,899,533	0.46	1.48	0.41	0.73	wall

Table A-20: Stress-test incremental peak memory above the pre-query baseline (GB)

Size (rows)	simple_sequence	alternation	quantified	optional_pattern	complex_nested
<i>Proposed engine</i>					
5,000,000	0.321	0.204	0.374	0.417	0.317
10,000,000	0.582	0.468	0.711	0.787	0.620
20,000,000	1.213	1.019	1.457	1.627	1.332
40,000,000	2.373	2.018	2.890	3.208	2.663
80,000,000	4.915	4.023	5.876	6.532	4.246
160,000,000	9.631	7.988	11.758	12.946	8.436
227,899,533	14.018	11.409	16.739	18.459	12.048
<i>Oracle 21c EE</i>					
5,000,000	0.127	0.128	0.128	0.128	0.128
10,000,000	0.257	0.256	0.256	0.256	0.260
20,000,000	0.516	0.512	0.514	0.512	0.524
40,000,000	0.448	0.452	0.448	0.449	0.449
80,000,000	0.463	0.455	0.460	0.452	0.458
160,000,000	0.468	0.455	0.463	0.472	0.487
227,899,533	0.676	0.648	0.641	0.546	0.588
<i>Trino 473 (disk connector)</i>					
5,000,000	0.007	0.003	0.004	0.002	0.004
10,000,000	0.006	0.008	0.004	0.008	0.005
20,000,000	0.006	0.006	0.006	0.004	0.357
40,000,000	0.729	0.007	0.451	0.020	0.719
80,000,000	1.540	0.016	1.532	0.011	0.013
160,000,000	1.605	2.630	0.029	0.015	wall
227,899,533	2.658	1.374	0.776	1.545	wall

Appendix A: Detailed Benchmark Tables

Table A-21: Database-native per-query memory counters (GB)

Size (rows)	simple_sequence	alternation	quantified	optional_pattern	complex_nested
<i>Oracle 21c EE</i>					
5,000,000	0.129	0.129	0.129	0.129	0.129
10,000,000	0.260	0.260	0.260	0.260	0.260
20,000,000	0.522	0.522	0.522	0.522	0.522
40,000,000	0.453	0.453	0.453	0.453	0.453
80,000,000	0.453	0.453	0.453	0.453	0.453
160,000,000	0.453	0.453	0.453	0.460	0.460
227,899,533	0.453	0.453	0.453	0.460	0.460
<i>Trino 473 (disk connector)</i>					
5,000,000	0.091	0.091	0.091	0.091	0.091
10,000,000	0.195	0.194	0.194	0.194	0.194
20,000,000	0.334	0.333	0.334	0.333	0.333
40,000,000	0.708	0.708	0.708	0.708	0.708
80,000,000	1.510	1.509	1.509	1.509	1.509
160,000,000	2.601	2.600	2.601	2.604	wall
227,899,533	3.799	3.798	3.798	3.798	wall

Table A-22: Stress-test returned rows by pattern and size under ONE ROW PER MATCH

Size (rows)	simple_sequence	alternation	quantified	optional_pattern	complex_nested
5,000,000	591,476	103,496	323,805	309,050	351,661
10,000,000	1,150,566	193,114	631,723	605,909	681,288
20,000,000	2,289,658	369,406	1,272,734	1,224,714	1,361,317
40,000,000	4,399,482	665,442	2,470,577	2,389,804	2,617,208
80,000,000	8,216,302	1,190,587	4,727,306	4,597,581	4,974,189
160,000,000	14,293,355	1,960,129	8,716,637	8,536,256	9,102,417
227,899,533	18,836,350	2,656,560	12,355,865	12,134,528	12,900,322